

2.Un-Answered UI Q&A

05/22/2026 11:43:55

<https://claude.ai/chat/65882e08-cc47-441d-8075-83a2d52c5b40>

Answer these questions with explanations and and simple example

Now let me compile all the questions from the images and create a comprehensi...

Don't create any document just write it here

I'll answer all the questions from your notes here. Let me go through them...

As you said react fiber added asynchronous and it work in this way like work pau...

Great question! This is a very common confusion. Let me clear it up. --- ## The...

Answer this

📖 Quoreka Questions --- **Q1. How do you show error or exception in UI /...
react internals

React Internals — Deep Dive --- ## 1. How React Works — The Big Picture ``...
memoization vs lazy loading

Memoization vs Lazy Loading These two are completely different concepts....

1. input - (function immediateA(a) { return (function immediateB(b) { console.log(...

Let me answer all of these one by one. --- ## Output Questions --- **Q1.** ``js...

Answer this

TechM Questions --- ## Q1. How we can call Backend API from UI in React?...

Answer birlasoft questions

Birlasoft Questions --- **Q1. What is Callback Queue?** The Callback Queue...

Is this promise fetch("https://jsonplaceholder.typicode.com/users") .then(res => { ...

Yes! This is exactly a Promise chain. Let me break it down completely. --- ## Yes,...

Fallbackque va call stack

Callback Queue vs Call Stack --- ## Simple One Line Difference `` Call Stack...

2.defaultProps in React ? 3.props vs defaultProps 10.rules to use Hooks ? (UI) 1...

Answers --- ## Q2. defaultProps in React? Default values for props when par...

how to make UI faster in react how do u make async function without using asyn...

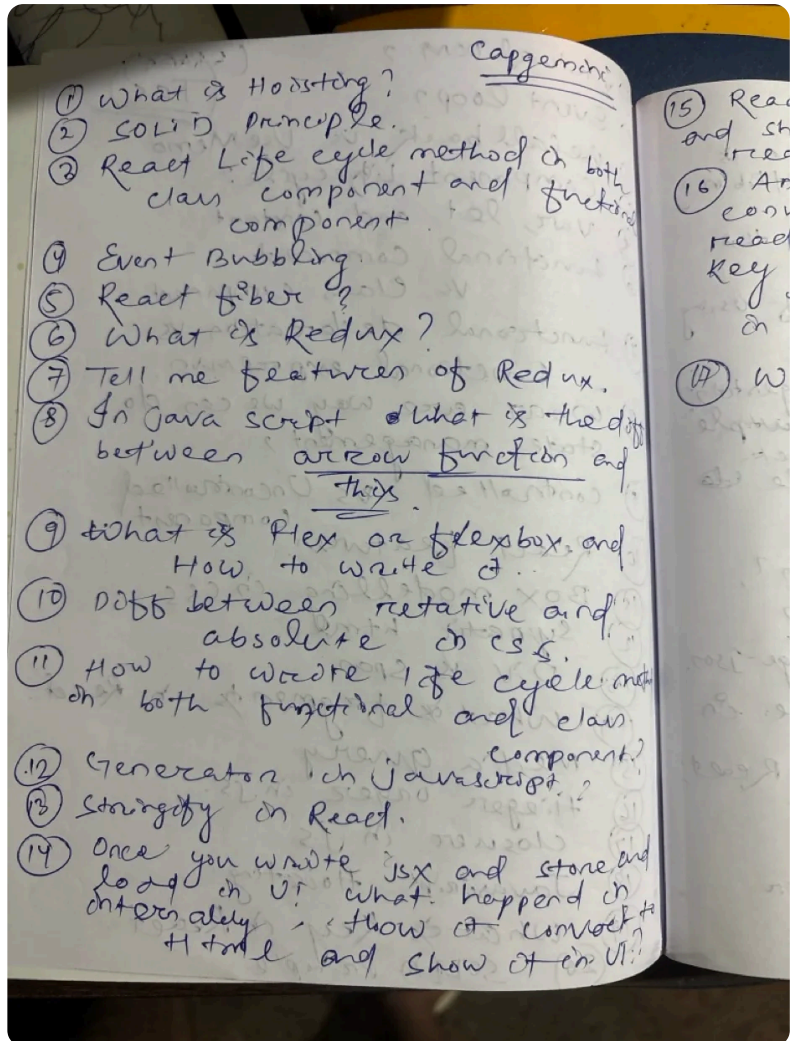
All Questions Answered --- ## 1. How to Make UI Faster in React ### Level 1 ...

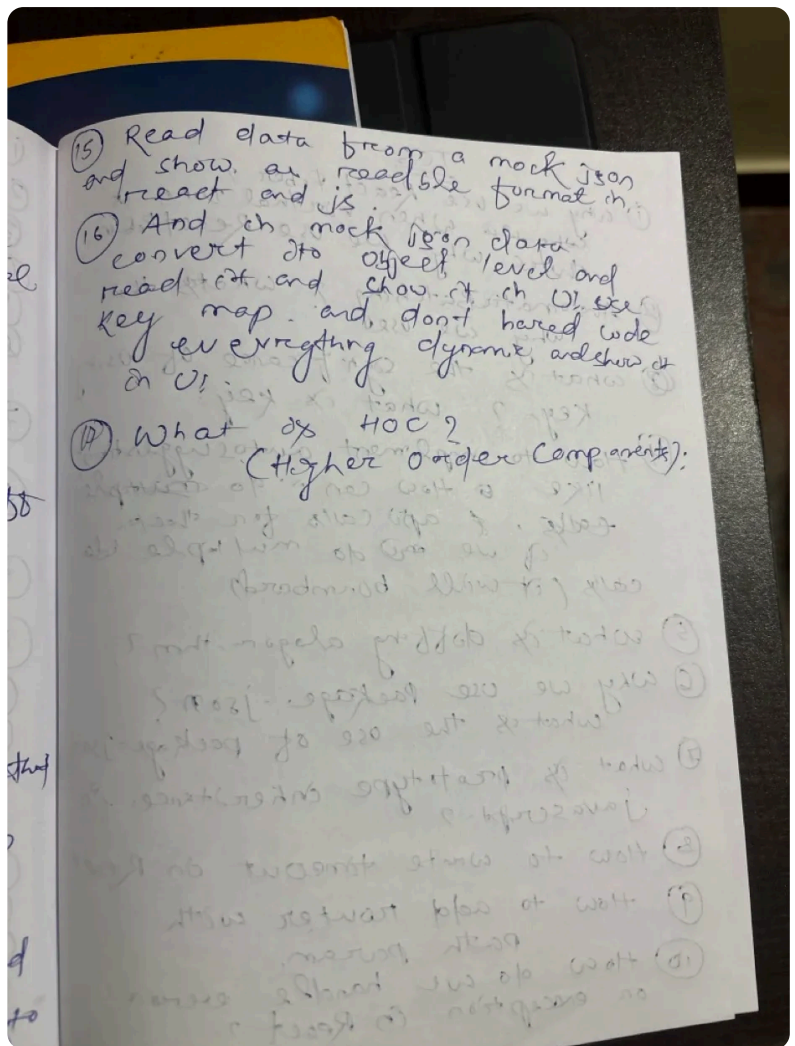
what is sync and async functions in javascript how do you write in css align...

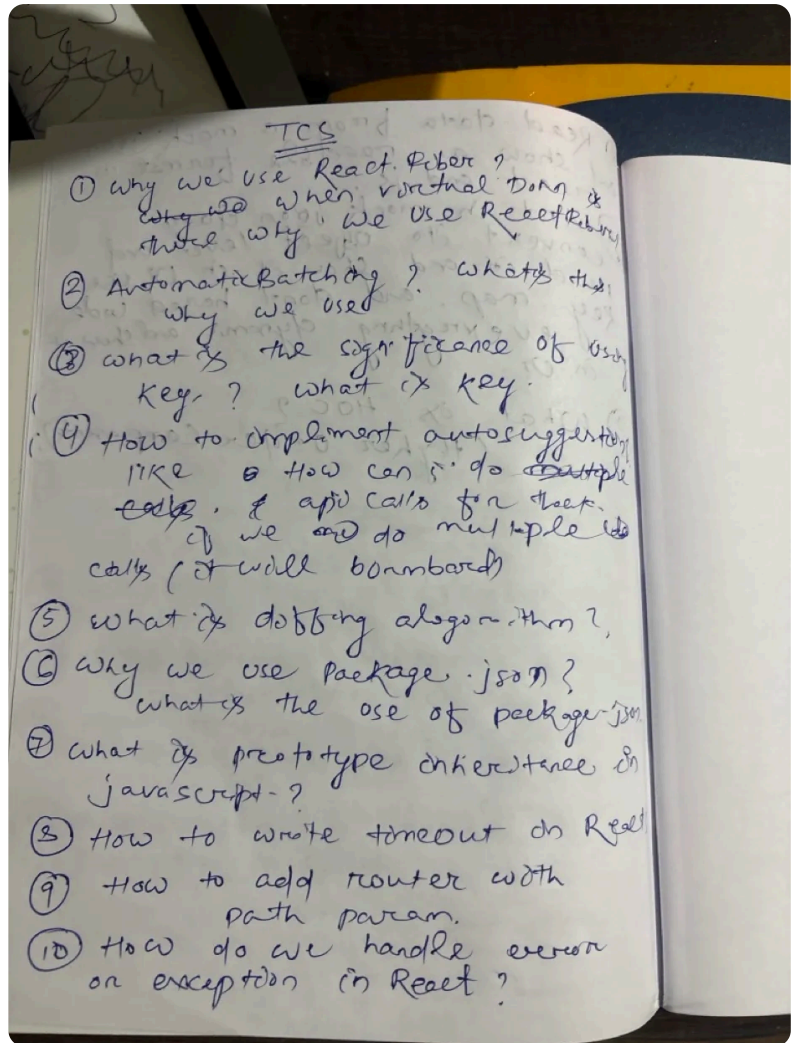
1. Sync vs Async in JavaScript --- ### Synchronous (Sync) One thing at a time...
How do we fix this above project
I can see the full project. Let me identify all bugs and implement all the required...
HTML basic questions and answers what is redux, how we use redux and explain...
HTML Basic Questions & Answers --- **Q. What is HTML?** HTML (HyperText...
what is pure component ? and what is non pure component ? what is useReducer...
All Answers --- ## 1. Pure Component A component that **only re-renders wh...
What is error boundary ? explain in details (why we used ? where we used and...
1. Error Boundary — Complete Guide --- ### What is it? A React component...
Search/Filter Bar: Add a search input to your Todo app that filters the list in real-...
Here are all 4 implementations: --- ## 1. Todo App with Search/Filter Bar --- ## 2....
you missed to add code for this Todo App with Search/Filter Bar
Here's the full React code for the Todo App with Search/Filter Bar: ``jsx import...
maybank / MBBLabs ===== 2. what is useReducer ?...
MBBLabs / Maybank Interview Questions --- ## Q2. What is useReducer? Wh...
vaiesika company UI ===== how to use this below json data using javascript...
All Answers --- ## Q1. findDetails — Access Nested JSON with Dot Notation...
Wipro ===== what is React ? is it library or framework ? what is...
Wipro Interview Questions --- ## Q1. What is React? Library or Framework?...
-mphasis What is event loop in js - [] What is micro task - [] and what is macro...
Mphasis Interview Questions --- ## Q1. What is Event Loop in JS? JavaScript...
Bell Company (UI) ===== what is the output of this ...
Bell Company UI Questions --- ## Q1. Output of this code? ``js...
a5econsulting - UI ===== what is reconciliation in...
a5econsulting UI Questions --- ## Q1. What is Reconciliation in React? Alrea...
wipro - UI ===== write js code to filter this below array with o...
Wipro UI Questions --- ## Q1. Filter Odd Numbers and Double Them ``js con...



- ① Virtual DOM ?
- ② Event Loop ?
- ③ UseCallback vs UseMemo (SPREAD)
Tcs
- ④ Component Lifecycle ?
- ⑤ var, let and constant
- ⑥ Functional component
vs Class component
- ⑦ Functional declaration vs
Functional expression
- ⑧ What are way we can do
state management ?
- ⑨ controlled vs Uncontrolled
Component
- ⑩ React features
- ⑪ Box modelling in CSS
- ⑫ syntactic html
- ⑬ Div vs span
- ⑭ What are fragments in React
- ⑮ media query
- ⑯ Higher order in JS
- ⑰ closure in JS
- ⑱ JavaScript Hoisting
- ⑲ What is Key in React
- ⑳ SOLID Principle







Answer these questions with explanations and and simple example

3/10/2026, 7:34:34 PM



Viewed file ▾

📄 Check available skills

✓ Done

Now let me compile all the questions from the images and create a comprehensive document answering them all.

Ran command, created file ▾

📄 Setup docx environment

📄 Creating file

✓ Done

3/10/2026, 7:36:59 PM



Don't create any document just write it here

3/10/2026, 7:37:14 PM



I'll answer all the questions from your notes here. Let me go through them systematically.

Page 1 (TCS Questions)

Q1. Virtual DOM?

The Virtual DOM is a lightweight JS copy of the real DOM kept in memory. When state changes, React creates a new Virtual DOM, diffs it with the old one (reconciliation), and updates only the changed parts in the real DOM.

```
function Counter() {  
  const [count, setCount] = useState(0);  
  return <button onClick={() => setCount(count + 1)}>Count: {count}  
</button>;  
}  
  
// Only the count text updates in real DOM, not the whole page
```

Q2. Event Loop?

JS is single-threaded. The Event Loop manages async operations via: Call Stack → Microtasks (Promises) → Macrotasks (setTimeout).

```
console.log("1");  
setTimeout(() => console.log("3"), 0);  
Promise.resolve().then(() => console.log("2"));
```

```
// Output: 1, 2, 3 → Promise (microtask) runs before setTimeout (macrotask)
```

Q3. useCallback vs useMemo?

- **useCallback** – memoizes a **function reference**, returns same function unless deps change
- **useMemo** – memoizes a **computed value**, recomputes only when deps change

```
const handleClick = useCallback(() => doSomething(id), [id]); // memoized function

const sorted = useMemo(() => list.sort((a, b) => a - b), [list]); // memoized value
```

Q4. Component Lifecycle?

Three phases: **Mounting** → **Updating** → **Unmounting**

```
// Class component
componentDidMount()    // after mount
componentDidUpdate()   // after update
componentWillUnmount() // before removal

// Functional equivalent
useEffect(() => {
  fetchData();          // componentDidMount
  return () => cleanup(); // componentWillUnmount
}, []);
```

Q5. var, let, const?

	var	let	const
Scope	Function	Block	Block
Hoisted	Yes (undefined)	Yes (TDZ)	Yes (TDZ)
Reassign	✓	✓	✗
Redeclare	✓	✗	✗

```

if (true) {
  var x = 10; // leaks outside block
  let y = 20; // stays inside block
}
console.log(x); // 10
console.log(y); // ReferenceError

```

Q6. Functional vs Class Component?

```

// Class (old way)
class Greet extends React.Component {
  render() { return <h1>Hello {this.props.name}</h1>; }
}

// Functional (modern, preferred)
const Greet = ({ name }) => <h1>Hello {name}</h1>;

```

Functional components are simpler, support Hooks, and are the modern standard.

Q7. Function Declaration vs Function Expression?

```

// Declaration - fully hoisted, can call before definition
sayHi(); // ✓ works
function sayHi() { console.log("Hi"); }

```

```
// Expression - NOT hoisted
greet(); // ❌ TypeError
const greet = function() { console.log("Hello"); };

// Arrow function expression
const add = (a, b) => a + b;
```

Q8. Ways to do State Management?

1. `useState` / `useReducer` – local state
2. **Context API** – share state without prop drilling
3. **Redux / Redux Toolkit** – global predictable state
4. **Zustand** – lightweight global state
5. **React Query / SWR** – server/async state
6. **Recoil / Jotai** – atomic state

```
// Context API
const ThemeContext = createContext();
<ThemeContext.Provider value="dark">
  <Child />
</ThemeContext.Provider>

// In Child:
const theme = useContext(ThemeContext); // "dark"
```

Q9. Controlled vs Uncontrolled Components?

```
// Controlled - React controls the value (source of truth = state)
const [val, setVal] = useState("");
<input value={val} onChange={e => setVal(e.target.value)} />

// Uncontrolled - DOM controls the value (source of truth = DOM)
```

```
const inputRef = useRef();  
<input ref={inputRef} />  
// Read: inputRef.current.value
```

Q10. React Features?

- Virtual DOM
- Component-based architecture
- JSX syntax
- Unidirectional data flow
- Hooks (useState, useEffect, etc.)
- React Fiber (concurrent rendering)
- Lazy loading / Code splitting
- SSR support (Next.js)

Q11. Box Model in CSS?

Content → Padding → Border → Margin (inside to outside)

```
div {  
  width: 200px;  
  padding: 10px;      /* inside element */  
  border: 2px solid;  
  margin: 20px;       /* outside element */  
  box-sizing: border-box; /* padding+border included in width */  
}
```

Q12. Semantic HTML?

Elements that describe their meaning, not just their appearance.

```
<!-- Non-semantic -->
<div id="nav"> <div class="header">

<!-- Semantic -->
<nav> <header> <main> <article> <section> <footer>
```

Benefits: better SEO, accessibility (screen readers), readable code.

Q13. div vs span?

- **div** – **block-level**, takes full width, starts new line → use for layout/sections
- **span** – **inline**, only takes content width, stays in line → use for styling text snippets

```
<div style="background:yellow">This is a block</div>
<p>This is <span style="color:red">red text</span> inline.</p>
```

Q14. Fragments in React?

Lets you group elements without adding an extra DOM node.

```
// Without fragment - adds unnecessary div
return <div><h1>Title</h1><p>Text</p></div>;

// With fragment - no extra DOM node
return (
  <>
    <h1>Title</h1>
    <p>Text</p>
  </>
);
```

Q15. Media Query in CSS?

Apply styles based on screen size — foundation of responsive design.

```
/* Mobile (default) */
.text { font-size: 14px; }

/* Tablet */
@media (min-width: 768px) { .text { font-size: 16px; } }

/* Desktop */
@media (min-width: 1024px) { .text { font-size: 18px; } }
```

Q16. Higher Order Functions in JS?

Functions that take or return other functions. Common: `map`, `filter`, `reduce`.

```
const nums = [1, 2, 3, 4, 5];
nums.map(n => n * 2);           // [2, 4, 6, 8, 10]
nums.filter(n => n % 2 === 0);  // [2, 4]
nums.reduce((acc, n) => acc + n, 0); // 15
```

Q17. Closure in JS?

An inner function that **remembers** variables from its outer scope even after the outer function has finished.

```
function counter() {
  let count = 0;
  return function() {
    count++;
    console.log(count);
  };
}
const inc = counter();
inc(); // 1
```

```
inc(); // 2
inc(); // 3 ← count persists in closure
```

Q18. JavaScript Hoisting?

JS moves declarations to the top of their scope before execution.

```
console.log(a); // undefined (var hoisted, initialized as undefined)
var a = 5;

console.log(b); // ReferenceError (let in TDZ)
let b = 5;

greet(); // "Hello" (function declaration fully hoisted)
function greet() { console.log("Hello"); }
```

Q19. What is Key in React?

Keys help React identify which list items changed, were added, or removed.

```
// ❌ Bad - index as key (breaks on reorder/delete)
items.map((item, i) => <li key={i}>{item}</li>)

// ✅ Good - stable unique id
items.map(item => <li key={item.id}>{item.name}</li>)
```

Q20. SOLID Principles?

- **S** – Single Responsibility: one class/function = one job
- **O** – Open/Closed: open for extension, closed for modification
- **L** – Liskov Substitution: subclass can replace its parent
- **I** – Interface Segregation: don't force unused methods

- **D** – Dependency Inversion: depend on abstractions, not concretions

Page 2 (Capgemini Questions)

Q1. React Life Cycle in Class & Functional Component?

```
// Class
componentDidMount()           // after first render
componentDidUpdate(pp, ps)    // after every update
componentWillUnmount()        // before removal

// Functional
useEffect(() => {
  // mount logic
  return () => { /* unmount cleanup */ };
}, []);                       // [] = only on mount

useEffect(() => {
  // update logic
}, [dependency]);             // runs when dependency changes
```

Q2. Event Bubbling?

Event fires on child, then propagates up through ancestors.

```
<div onClick={() => console.log("div")}>
  <button onClick={() => console.log("button")}>Click</button>
</div>
// Click button → logs "button" then "div"

// Stop bubbling:
<button onClick={e => { e.stopPropagation(); console.log("button
only"); }}>
```

Q3. React Fiber?

React's reimplemented reconciliation engine (React 16+). Enables:

- Splitting rendering work into small chunks
- Pausing, aborting, reusing work
- Update prioritization
- Concurrent Mode (React 18)

Before Fiber, rendering was synchronous and could freeze the UI. Fiber makes it interruptible.

Q4. What is Redux?

A predictable global state management library. All state lives in one store, changed only via dispatched actions processed by pure reducers.

```
// Action
{ type: 'INCREMENT' }

// Reducer
function reducer(state = { count: 0 }, action) {
  if (action.type === 'INCREMENT') return { count: state.count + 1 };
  return state;
}

store.dispatch({ type: 'INCREMENT' });
store.getState(); // { count: 1 }
```

Q5. Features of Redux?

- Single source of truth (one store)
- State is read-only (change only via actions)

- Pure reducer functions
- DevTools with time-travel debugging
- Middleware support (think for async, saga)
- Redux Toolkit (RTK) reduces boilerplate

Q6. Arrow Function vs Regular Function (`this`)?

```
const obj = {
  name: "React",
  regular: function() { return this.name; }, // this = obj ✅
  arrow: () => this.name                     // this = outer scope
  ❌
};

obj.regular(); // "React"
obj.arrow();   // undefined

// In class components, arrow avoids manual binding:
handleClick = () => { this.setState({...}); } // 'this' always =
component
```

Q7. Flexbox – What is it and how to write it?

```
.container {
  display: flex;
  flex-direction: row;      /* main axis: row | column */
  justify-content: center;  /* main axis alignment */
  align-items: center;      /* cross axis alignment */
  flex-wrap: wrap;
  gap: 16px;
}

.item {
```

```
flex: 1;          /* grow and shrink equally */  
}
```

Q8. relative vs absolute in CSS?

```
/* relative - offset from normal position, still takes up space */  
.box { position: relative; top: 10px; left: 20px; }  
  
/* absolute - removed from flow, positioned to nearest positioned ancestor */  
.parent { position: relative; }  
.child { position: absolute; top: 0; right: 0; } /* top-right of parent */
```

Q9. Generator in JavaScript?

A function that can **pause and resume** using `yield`.

```
function* gen() {  
  yield 1;  
  yield 2;  
  yield 3;  
}  
const g = gen();  
g.next(); // { value: 1, done: false }  
g.next(); // { value: 2, done: false }  
g.next(); // { value: 3, done: false }  
g.next(); // { value: undefined, done: true }
```

Q10. Stringify in React?

```
const user = { name: "Alice", age: 25 };
```

```
JSON.stringify(user);           // '{"name":"Alice","age":25}'
JSON.parse('{"name":"Alice"}'); // { name: "Alice" }

// Debug state in JSX:
<pre>{JSON.stringify(state, null, 2)}</pre>
```

Q11. JSX → HTML: What happens internally?

1. You write JSX: `<h1 className="title">Hello</h1>`
2. Babel transpiles it → `React.createElement("h1", { className: "title" }, "Hello")`
3. Returns a plain JS object (React Element / Virtual DOM node)
4. React Fiber reconciles old vs new Virtual DOM
5. ReactDOM patches only the changed parts in real DOM
6. Browser renders it as HTML

```
// JSX compiles to:
React.createElement("h1", { className: "title" }, "Hello");
// Result: { type: "h1", props: { className: "title", children:
"Hello" } }
```

Q12. Read mock JSON and show in React?

```
const data = [{ id: 1, name: "Alice" }, { id: 2, name: "Bob" }];

function UserList() {
  return (
    <ul>
      {data.map(user => <li key={user.id}>{user.name}</li>)}
    </ul>
  );
}
```

Q13. Convert mock JSON to object level, show in UI dynamically?

```
const data = {
  users: [
    { id: 1, name: "Alice", age: 25 },
    { id: 2, name: "Bob", age: 30 }
  ]
};

function DynamicTable({ data }) {
  const headers = Object.keys(data.users[0]); // ["id", "name", "age"]
  return (
    <table>
      <thead>
        <tr>{headers.map(h => <th key={h}>{h}</th>)}</tr>
      </thead>
      <tbody>
        {data.users.map(row => (
          <tr key={row.id}>
            {headers.map(h => <td key={h}>{row[h]}</td>)}
          </tr>
        ))}
      </tbody>
    </table>
  );
}
```

No hardcoded column names — fully dynamic via `Object.keys()`.

Q14. What is HOC (Higher Order Component)?

A function that takes a component and returns a new enhanced component. Used to reuse logic (auth checks, logging, theming).

```
function withAuth(WrappedComponent) {
  return function(props) {
    const isLoggedIn = checkAuth();
    if (!isLoggedIn) return <Redirect to="/login" />;
    return <WrappedComponent {...props} />;
  };
}

const ProtectedDashboard = withAuth(Dashboard);
```

Page 4 (TCS – Additional)

Q1. Why we use React Fiber, Virtual DOM, and ReactDOM?

- **Virtual DOM** → minimizes expensive real DOM updates by diffing
- **Fiber** → makes rendering interruptible and prioritized (Concurrent Mode)
- **ReactDOM** → bridge between React's Virtual DOM and the actual browser DOM

```
import ReactDOM from 'react-dom/client';
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

Q2. Automatic Batching?

Grouping multiple state updates into a **single re-render**. React 18 extended this to work everywhere (timeouts, Promises, native events).

```
// React 18 - all 3 updates batched → only 1 re-render
setTimeout(() => {
  setCount(c => c + 1);
  setFlag(f => !f);
  setName("Alice");
```

```
// Only 1 re-render happens!  
}, 1000);
```

Q3. Significance of Key in React?

See Q19 above. Key lets React track list items efficiently during reconciliation — without it, React re-renders the entire list on any change.

Q4. How to implement Autosuggest / Debouncing for multiple API calls?

```
function SearchBox() {  
  const [query, setQuery] = useState("");  
  
  const debouncedSearch = useCallback(  
    debounce((q) => fetchSuggestions(q), 500), // wait 500ms after  
    user stops typing  
    []);  
  
  const handleChange = (e) => {  
    setQuery(e.target.value);  
    debouncedSearch(e.target.value);  
  };  
  
  return <input value={query} onChange={handleChange} />;  
}
```

Debouncing delays the API call until the user stops typing — prevents bombarding the server with requests.

Q5. Diffing Algorithm in React?

React's reconciliation algorithm that compares old vs new Virtual DOM trees. Key rules:

- Elements of different types → rebuild entire subtree

- Same type → update only changed attributes
- Lists → use `key` to match elements efficiently

Time complexity: $O(n)$ instead of $O(n^3)$ due to smart assumptions.

Q6. Why we use package.json?

```
{
  "name": "my-app",
  "version": "1.0.0",
  "dependencies": { "react": "^18.0.0" },
  "scripts": { "start": "react-scripts start" }
}
```

- Lists all project dependencies
- Defines npm scripts (start, build, test)
- Locks version ranges
- Enables anyone to run `npm install` and get the same packages

Q7. Prototype Inheritance in JavaScript?

Every JS object has a `__proto__` (prototype). When you access a property, JS looks up the prototype chain.

```
function Animal(name) { this.name = name; }
Animal.prototype.speak = function() { return `${this.name} speaks`; };

const dog = new Animal("Rex");
dog.speak(); // "Rex speaks" ← found on prototype, not on dog itself

// ES6 Class (same thing, cleaner syntax)
```



```
class Animal {
  constructor(name) { this.name = name; }
  speak() { return `${this.name} speaks`; }
}
class Dog extends Animal {
  bark() { return "Woof!"; }
}
```

Q8. How to write timeout in React?

```
useEffect(() => {
  const timer = setTimeout(() => {
    console.log("Runs after 2 seconds");
    setVisible(false);
  }, 2000);

  return () => clearTimeout(timer); // cleanup on unmount!
}, []);
```

Always clear timeout in the cleanup function to avoid memory leaks.

Q9. How to add router with path params?

```
// Setup
import { BrowserRouter, Routes, Route, useParams } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/user/:id" element={<UserProfile />} />
      </Routes>
    </BrowserRouter>
  );
}
```

```

    </BrowserRouter>
  );
}

// Read the param
function UserProfile() {
  const { id } = useParams(); // extracts :id from URL
  return <h1>User ID: {id}</h1>;
}

// URL: /user/42 → displays "User ID: 42"

```

Q10. How to handle errors / exceptions in React?

```

// 1. Error Boundaries (class component - catches render errors)
class ErrorBoundary extends React.Component {
  state = { hasError: false };
  static getDerivedStateFromError() { return { hasError: true }; }
  render() {
    return this.state.hasError ? <h1>Something went wrong</h1> :
    this.props.children;
  }
}

<ErrorBoundary><MyComponent /></ErrorBoundary>

// 2. try/catch in async functions
async function fetchData() {
  try {
    const res = await fetch('/api/data');
    const data = await res.json();
  } catch (error) {
    console.error("Error:", error.message);
  }
}

```

```
// 3. .catch() with Promises  
fetch('/api').then(res => res.json()).catch(err => setError(err));
```

That covers all questions from all 4 pages! Let me know if you want any topic explained in more depth. 🚀

3/10/2026, 7:39:05 PM



As you said react fiber added asynchronous and it work in this way like work pause work pause and asynchronous then it will take longer time and it become performance issue then why they introduce react fiber

3/10/2026, 10:30:30 PM



Great question! This is a very common confusion. Let me clear it up.

The Misconception

You're thinking: *"If Fiber pauses and resumes work, won't that take MORE time than just doing it all at once?"*

The answer: Total work time is almost the same. What changes is WHO GETS BLOCKED.

The Real Problem Before Fiber (React 15)

Before Fiber, rendering was **synchronous and un interruptible**.

```
User clicks button
→ React starts rendering (big component tree)
→ Takes 500ms to finish
→ During these 500ms... NOTHING ELSE CAN HAPPEN
→ User clicks, scrolls, types → ALL IGNORED
→ Browser freezes / janky UI
```

Imagine a chef who **refuses to stop cooking** until the entire 10-course meal is done — even if the restaurant catches fire.

What Fiber Actually Does

Fiber doesn't make rendering **faster in total time**. It makes rendering **non-blocking**.

```
User clicks button
→ React starts rendering
→ Does a small chunk of work (a few ms)
→ PAUSES → checks: "Is there anything urgent? (user typing,
animation?)"
→ If YES → handles urgent task first
→ Then RESUMES rendering
→ Total time: slightly same or even slightly more
→ BUT UI never freezes
```

The Real Performance Win: Priority System

Fiber introduced **priority levels** for updates:

Priority	Example
 Immediate	Error boundaries
 High	User typing, button click
 Normal	Data fetching result
 Low	Analytics, logging
 Idle	Pre-rendering offscreen content

Before Fiber:

```
→ Heavy data update (low priority) BLOCKS button click (high
priority)
→ User feels lag
```

After Fiber:

```
→ Heavy data update starts
→ User clicks button (high priority detected)
```

- Fiber INTERRUPTS data **update**
- Handles button click **FIRST** (feels instant)
- **Then** goes back **to** data **update**

Real World Analogy

Before Fiber (bad):

You're on a phone call with a customer (low priority). Boss walks in (high priority). You say "wait, let me finish this call first" — boss is frustrated.

After Fiber (good):

You're on a phone call. Boss walks in. You put customer on hold, handle boss immediately, then return to call.

Total time of both tasks? Almost same. But **important things get done first**.

Concurrent Mode (React 18) — Fiber's Full Power

```
// React 18 - useTransition marks an update as LOW priority
const [isPending, startTransition] = useTransition();

function handleSearch(e) {
  setInputValue(e.target.value);           // HIGH priority → instant
  // UI update

  startTransition(() => {
    setSearchResults(filterData(e.target.value)); // LOW priority →
    // can be interrupted
  });
}
```

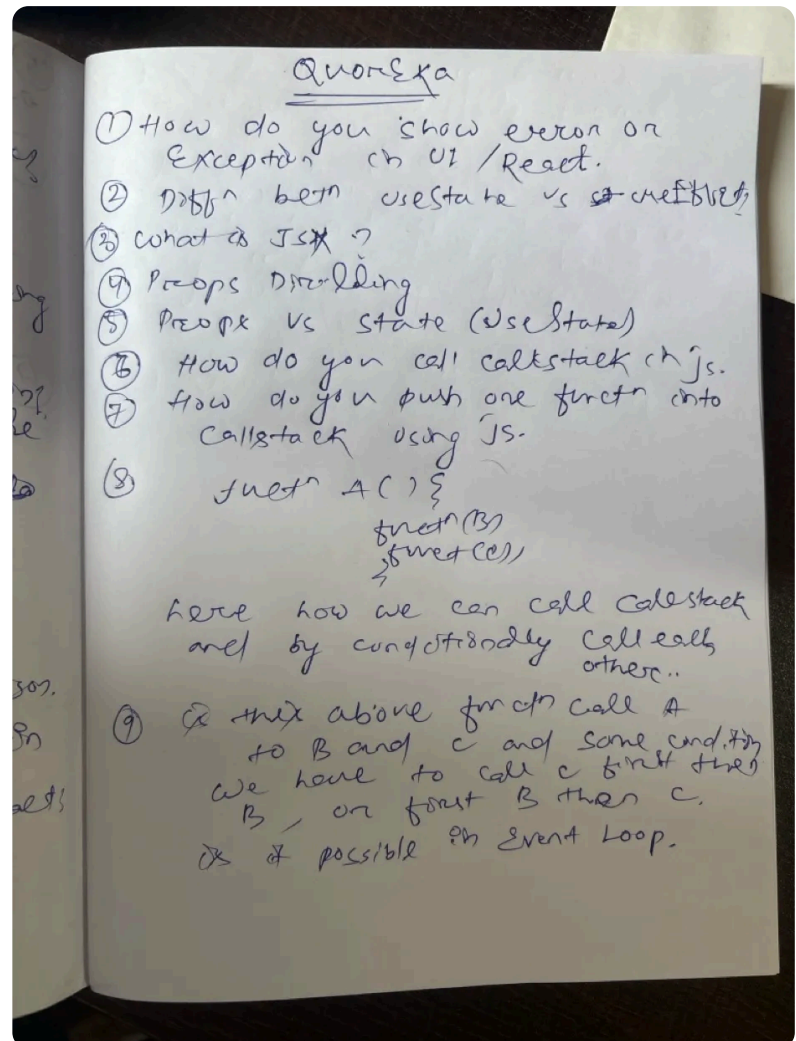
The input feels **instantly responsive** even while filtering 10,000 items in background.

Summary

	Before Fiber	After Fiber
Rendering	Synchronous, all-or-nothing	Chunked, interruptible
UI during heavy render	Freezes ❌	Stays responsive ✅
Priority handling	None — first come first served	High priority tasks go first ✅
Total render time	Fast	Same or slightly more
User experience	Janky	Smooth

Fiber trades a tiny bit of total time for a much better user experience. The goal was never speed — it was responsiveness.

3/10/2026, 10:30:54 PM



Answer this

3/12/2026, 1:08:39 PM



Quoreka Questions

Q1. How do you show error or exception in UI / React?

Three ways:

1. Error Boundary (catches render errors)


```

class ErrorBoundary extends React.Component {
  state = { hasError: false, message: "" };

  static getDerivedStateFromError(error) {
    return { hasError: true, message: error.message };
  }

  render() {
    if (this.state.hasError)
      return <h2 style={{color:"red"}}>Error: {this.state.message}</h2>;
    return this.props.children;
  }
}

// Usage
<ErrorBoundary>
  <MyComponent />
</ErrorBoundary>

```

2. try/catch with state (catches async errors)

```

const [error, setError] = useState(null);

async function fetchData() {
  try {
    const res = await fetch("/api/data");
    const data = await res.json();
  } catch (err) {
    setError(err.message); // store error in state
  }
}

// Show in UI

```

```
{error} && <p style={{color: "red"}}>Something went wrong: {error}</p>}
```

3. Conditional rendering

```
{error} ? <ErrorMessage msg={error} /> : <DataComponent />
```

Q2. Difference between useState vs useEffect?

	useState	useEffect
Purpose	Store & update data	Perform side effects
When it runs	On every state change	After render (or when deps change)
Returns	[value, setter]	cleanup function (optional)
Examples	counter, form input	API calls, timers, subscriptions

```
const [count, setCount] = useState(0);           // stores data

useEffect(() => {                                // runs AFTER render
  document.title = `Count: ${count}`;           // side effect
}, [count]);                                     // re-runs when count
changes
```

Key difference: **useState** is about **what to show**, **useEffect** is about **what to do after showing**.

Q3. What is JSX?

JSX (JavaScript XML) is a syntax extension that lets you write HTML-like code inside JavaScript. It is NOT valid JS — Babel compiles it to **React.createElement()** calls.

```
// JSX (what you write)
const el = <h1 className="title">Hello {name}</h1>;

// What Babel compiles it to
const el = React.createElement("h1", { className: "title" }, "Hello
" + name);

// Rules:
// 1. Return single root element (or use <>...</>)
// 2. Use className instead of class
// 3. Self-close tags: <img /> <input />
// 4. JS expressions go inside {}
const ui = <p>{2 + 2}</p>; // renders "4"
```

Q4. Props Drilling?

When you pass props through many intermediate components that **don't need it**, just to reach a deeply nested child.

```
// ❌ Props Drilling - middle components just pass it down
function App() {
  const user = "Alice";
  return <Parent user={user} />;
}
function Parent({ user }) {
  return <Child user={user} />; // Parent doesn't use user, just
passes it
}
function Child({ user }) {
  return <h1>{user}</h1>; // only Child actually needs it
}

// ✅ Solution - Context API (no drilling)
const UserContext = createContext();
```

```
function App() {
  return (
    <UserContext.Provider value="Alice">
      <Parent />
    </UserContext.Provider>
  );
}
function Child() {
  const user = useContext(UserContext); // directly access without drilling
  return <h1>{user}</h1>;
}
```

Q5. Props vs State (useState)?

	Props	State
Who owns it	Parent passes it	Component owns it
Mutable?	 Read-only	 Can be changed
Who updates it	Parent	Component itself
Triggers re-render	Yes (when parent updates)	Yes (when setState called)

// Props - passed from parent, cannot change inside Child

```
function Child({ name }) {
  // name = read-only, can't do setName here
  return <h1>Hello {name}</h1>;
}
```

// State - owned and changed by the component itself

```
function Counter() {
  const [count, setCount] = useState(0); // owned here
  return <button onClick={() => setCount(count + 1)}>{count}</button>
```

```
</button>;  
}
```

Q6. How do you call the Call Stack in JS?

The Call Stack is not something you call manually — it's managed automatically by the JS engine. Every time you call a function, JS **pushes** it onto the stack. When it returns, it **pops** it off.

```
function C() { console.log("C running"); }  
function B() { C(); }  
function A() { B(); }  
  
A();
```

```
// Call Stack sequence:  
// 1. A() pushed  
// 2. B() pushed (A called B)  
// 3. C() pushed (B called C)  
// 4. C() popped (C finished)  
// 5. B() popped (B finished)  
// 6. A() popped (A finished)  
// Stack is empty
```

You can see the call stack in **Chrome DevTools** → **Sources** → **Call Stack panel** when paused on a breakpoint.

Q7. How do you push one function into Call Stack using JS?

Again, you don't push manually — calling a function automatically pushes it. But here's how to **control the order**:

```
function A() { console.log("A"); }  
function B() { console.log("B"); }  
function C() { console.log("C"); }
```

```
// Direct call - pushes A immediately onto stack
A();

// setTimeout - pushes to macrotask queue, goes to stack AFTER
current stack clears
setTimeout(A, 0);

// Promise - pushes to microtask queue (runs before setTimeout)
Promise.resolve().then(A);
```

Q8. fnA() calls fnB(), fnB() calls fnC() — how does Call Stack work, and conditionally call each other?

```
function C() {
  console.log("C executed");
}

function B() {
  console.log("B executed");
  C(); // B calls C → C pushed onto stack
}

function A() {
  console.log("A executed");
  B(); // A calls B → B pushed onto stack
}

A();

// Stack visualization:
// [A]
// [A, B]      ← B pushed when A calls B
// [A, B, C]    ← C pushed when B calls C
```

```
// [A, B]      ← C popped (finished)
// [A]         ← B popped (finished)
// []          ← A popped (finished)
```

With conditions:

```
function C() { console.log("C executed"); }

function B(callC) {
  console.log("B executed");
  if (callC) C(); // conditionally call C
}

function A(condition) {
  console.log("A executed");
  B(condition); // pass condition down
}

A(true); // A → B → C
A(false); // A → B only
```

Q9. Function A calls B and C. On some condition we need C first, then B — is it possible in Event Loop?

Yes! You can control execution order using the Event Loop with `setTimeout`, `Promise`, or `async/await`.

```
function A() {
  console.log("A started");

  const condition = true; // some condition

  if (condition) {
    // C first, then B
  }
}
```

```

    Promise.resolve()
      .then(() => C()) // microtask - runs first
      .then(() => B()); // runs after C
  } else {
    // Normal order: B first, then C
    Promise.resolve()
      .then(() => B())
      .then(() => C());
  }
}

function B() { console.log("B executed"); }
function C() { console.log("C executed"); }

A();
// Output when condition = true:
// "A started"
// "C executed"
// "B executed"

```

Using async/await (cleaner):

```

async function A(condition) {
  console.log("A started");

  if (condition) {
    await C(); // wait for C to finish first
    await B(); // then B
  } else {
    await B();
    await C();
  }
}

async function B() { console.log("B"); }

```



```
async function C() { console.log("C"); }
```

```
A(true); // A → C → B
```

```
A(false); // A → B → C
```

Using setTimeout (macrotask queue):

```
function A() {  
  setTimeout(() => B(), 100); // B after 100ms  
  setTimeout(() => C(), 0);   // C after 0ms → C runs first!  
}  
  
// Output: C → B (lower timeout = runs first)
```

The Event Loop gives you full control over execution order through microtasks (Promises) and macrotasks (setTimeout). 

3/12/2026, 1:09:29 PM



React Internals — Deep Dive

1. How React Works — The Big Picture

```
You write JSX
  ↓
Babel compiles to React.createElement()
  ↓
React Elements (plain JS objects)
  ↓
React Fiber (reconciliation engine)
  ↓
Virtual DOM diffing
  ↓
ReactDOM commits changes to real DOM
  ↓
Browser paints the screen
```

2. JSX → React.createElement → React Element

```
// You write
const el = <h1 className="title">Hello</h1>;

// Babel compiles to
const el = React.createElement("h1", { className: "title" },
```

```

"Hello");

// React.createElement returns a plain JS object (React Element)
{
  type: "h1",
  props: {
    className: "title",
    children: "Hello"
  },
  key: null,
  ref: null,
  $$typeof: Symbol(react.element) // security - prevents XSS
}

```

React Elements are **immutable, cheap, plain objects** — not real DOM nodes. Just a description of what to render.

3. React Fiber — The Core Engine

Every React Element gets a corresponding **Fiber node** — a JS object that holds all the work React needs to do.

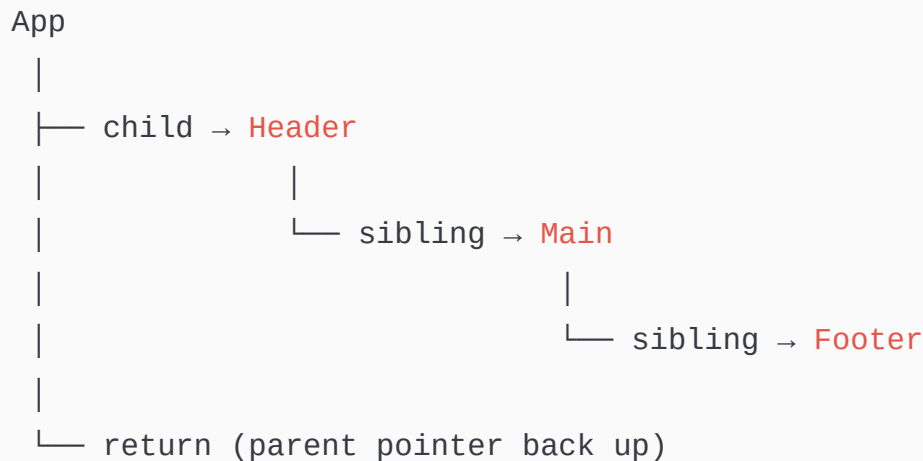
```

// A Fiber node looks like this internally
{
  type: "h1",           // component type
  key: null,
  child: Fiber,         // first child
  sibling: Fiber,        // next sibling
  return: Fiber,        // parent fiber
  pendingProps: {},     // new props
  memoizedProps: {},    // last rendered props
  memoizedState: {},    // hooks state lives here
  effectTag: "UPDATE",  // what needs to happen
  (INSERT/UPDATE/DELETE)
  alternate: Fiber,     // pointer to old fiber (for diffing)
}

```

```
lanes: Lanes, // priority of this work
}
```

Fiber Tree is a Linked List, not a Tree



This linked list structure is why Fiber can **pause and resume** — it knows exactly where to continue.

4. Two Phases of Rendering

Phase 1: Render Phase (async, interruptible)

React traverses the fiber tree and figures out **what changed**. No DOM changes yet.

```
beginWork(fiber) → process fiber, create child fibers
↓
completeWork(fiber) → create DOM nodes (but don't insert yet)
↓
move to next fiber (child → sibling → parent)
```

Phase 2: Commit Phase (synchronous, cannot be interrupted)

React applies all changes to the real DOM in 3 sub-phases:

1. Before Mutation → `getSnapshotBeforeUpdate()`, `useLayoutEffect` cleanup
2. Mutation → INSERT / UPDATE / DELETE actual DOM nodes
3. Layout → `useLayoutEffect()`, `componentDidMount/Update`
↓
(Browser paints screen)
↓
4. Passive Effects → `useEffect()` runs here (after paint)

5. Reconciliation — How Diffing Works

React compares **current fiber tree** vs **work-in-progress fiber tree**.

Rule 1: Different element type → destroy and rebuild

```
// Old tree      New tree
<div>           → <span>
  <Child />      <Child />
</div>          </span>

// React destroys entire div subtree, builds span subtree fresh
// Child unmounts and remounts
```

Rule 2: Same element type → update only changed props

```
// Old      New
<div className="old" />  <div className="new" />

// React keeps the DOM node, just updates className
// No unmount/remount
```

Rule 3: Lists use Keys to match elements

```
// Without keys – React matches by position
// Old: [A, B, C] New: [X, A, B, C]
// React thinks A changed to X, B→A, C→B, inserts C
// Re-renders all 4! ❌

// With keys – React matches by identity
// Old: [A(key=a), B(key=b)] New: [X(key=x), A(key=a), B(key=b)]
// React knows A and B didn't change, just inserts X
// Only 1 new render! ✅
```

6. Double Buffering — current vs work-in-progress

React maintains **two fiber trees** at all times:

```
current tree          work-in-progress tree
(what's on screen)   (what React is building)
    |                  |
    └─── alternate ───┘
    (each fiber points to its mirror)
```

1. React builds work-in-progress tree
2. Once complete → work-in-progress becomes current
3. Old current becomes the next work-in-progress buffer
4. (Reuses memory, avoids garbage collection)

7. How Hooks Work Internally

Hooks are stored as a **linked list on the fiber node** — order matters, which is why you can't use hooks inside conditionals.

```
// Your component
function Counter() {
  const [count, setCount] = useState(0); // Hook 1
```

```

    const [name, setName] = useState(""); // Hook 2
    useEffect(() => {}, [count]); // Hook 3
  }

  // Internally stored on fiber.memoizedState:
  Hook1 → Hook2 → Hook3 → null
  {      {      {
    state:0, state:"", effect: fn,
    next: —→ next: —→ next: null
  }      }      }

```

Why hooks can't be inside if/else:

```

// ❌ WRONG - breaks hook order
if (condition) {
  const [x, setX] = useState(0); // sometimes Hook1, sometimes
  missing!
}

// On first render: Hook1=useState, Hook2=useEffect
// On re-render(condition=false): Hook1=useEffect ← MISMATCH! React
breaks

```

8. useState Internals

```

// Simplified internal implementation
function useState(initialValue) {
  const fiber = getCurrentFiber();
  const hook = getOrCreateHook(fiber); // gets from linked list

  // First render
  if (hook.state === undefined) {
    hook.state = initialValue;
  }
}

```

```

}

// setState queues an update
const setState = (newValue) => {
  hook.queue.push(newValue);      // add to update queue
  scheduleWork(fiber);            // tell React to re-render
};

// Process queued updates
hook.state = processQueue(hook.queue, hook.state);

return [hook.state, setState];
}

```

9. useEffect Internals

```

// useEffect runs AFTER browser paint (async)
// useEffect runs BEFORE browser paint (sync)

function useEffect(fn, deps) {
  const hook = getOrCreateHook();
  const prevDeps = hook.deps;

  // Compare deps (Object.is shallow comparison)
  const depsChanged = prevDeps
    ? deps.some((d, i) => !Object.is(d, prevDeps[i]))
    : true; // no prev deps = first render = always run

  if (depsChanged) {
    hook.effect = fn; // schedule effect
    hook.deps = deps;
  }
}

```



```
// After commit phase:  
// 1. Run cleanup of previous effect (return fn)  
// 2. Run new effect fn
```

```
useEffect(() => {  
  const sub = subscribe(id); // effect  
  return () => sub.unsubscribe(); // cleanup runs BEFORE next effect  
}, [id]);
```

10. React Scheduler — Priority Lanes

React 18 introduced **Lanes** — a bitmask system for update priority:

```
// Lanes (simplified)  
const SyncLane      = 0b0001; // highest - user input  
const InputLane     = 0b0010; // typing, clicking  
const TransitionLane = 0b0100; // startTransition()  
const IdleLane      = 0b1000; // lowest - offscreen
```

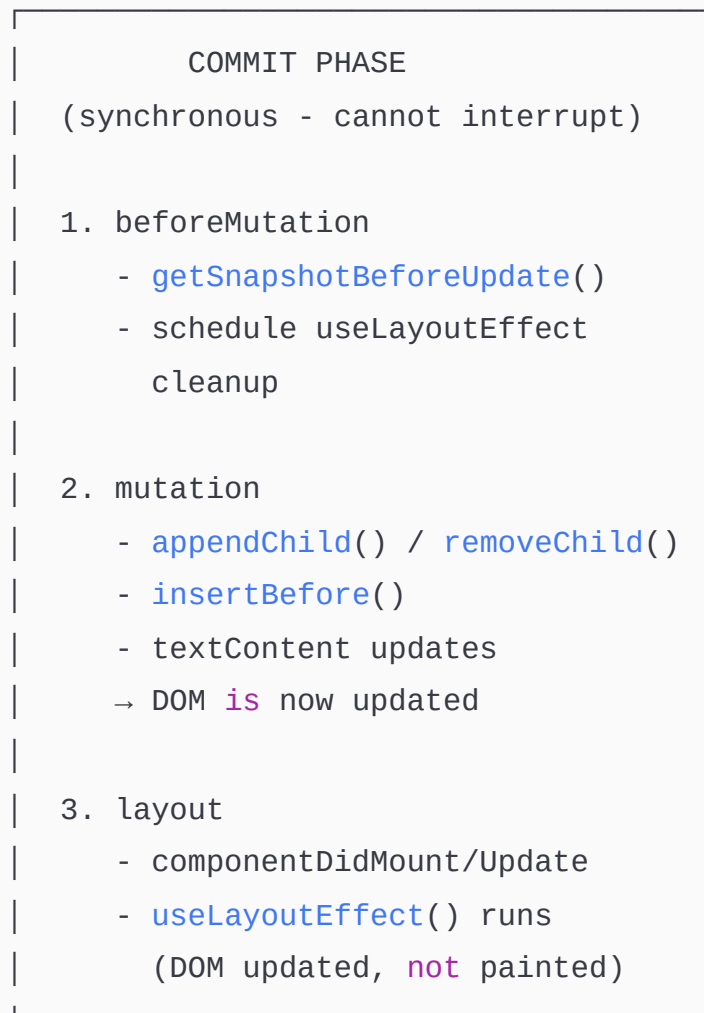
```
// Multiple updates merged with bitwise OR  
const pendingLanes = SyncLane | TransitionLane; // 0b0101
```

```
// You control priority with useTransition  
const [isPending, startTransition] = useTransition();  
  
function handleInput(e) {  
  setInput(e.target.value); // SyncLane → runs immediately  
  
  startTransition(() => {  
    setResults(filter(e.target.value)); // TransitionLane → can be interrupted  
  });  
}
```

11. Commit Phase in Detail

Fiber `tree` (work-in-progress) `is` fully built

↓



↓

Browser paints screen

↓

`useEffect()` `runs` (passive effects)

12. Batching Internals

```
// React 17 - only batches inside React event handlers
setTimeout(() => {
```

```
    setA(1); // re-render 1
    setB(2); // re-render 2 (2 renders!)
  }, 0);

// React 18 - batches EVERYWHERE automatically
setTimeout(() => {
  setA(1);
  setB(2);
  // React batches both → only 1 re-render ✅
}, 0);

// How: React wraps all updates in a unit of work
// flushes them together in one commit
```

13. Context Internals

```
const MyContext = createContext(defaultValue);
// Creates: { Provider, Consumer, _currentValue }

// Provider updates _currentValue on the context object
// When value changes, React walks the fiber tree
// looking for consumers (useContext / Consumer)
// and marks them as needing re-render

// This is why Context causes ALL consumers to re-render
// even if only part of the value changed ← performance gotcha

// Fix: split contexts, or use useMemo
const value = useMemo(() => ({ user, theme }), [user, theme]);
```

14. Full Re-render Flow Summary

1. Event fires (click, input, etc.)
↓
2. setState called → update queued on fiber
↓
3. Scheduler picks up work (based on priority lane)
↓
4. RENDER PHASE begins
 - Clone current fiber tree → work-in-progress
 - Call your function component (re-executes!)
 - Process hooks in order (linked list)
 - React.createElement → new React elements
 - Diff old vs new (reconciliation)
 - Tag fibers: INSERT / UPDATE / DELETE↓
5. COMMIT PHASE begins (sync)
 - Apply DOM mutations
 - Run useEffect↓
6. Browser paints
↓
7. useEffect runs
↓
8. Done 

Key Takeaways

Concept	What it is
React Element	Plain JS object describing UI
Fiber	JS object tracking work + state + effects
current tree	What's currently on screen
work-in-progress	What React is building next
Render phase	Figure out what changed (async)

Concept	What it is
Commit phase	Apply changes to DOM (sync)
Hooks	Linked list stored on fiber node
Lanes	Priority bitmask for scheduling
Batching	Multiple setState → 1 re-render

This is the engine running under every React app you build. 🚀

3/13/2026, 12:50:58 PM



Memoization vs Lazy Loading

These two are completely different concepts. People confuse them because both are "optimization techniques."

Simple One-Line Difference

	What it solves
Memoization	Prevents unnecessary re-computation / re-rendering
Lazy Loading	Prevents loading code/data before it's needed

Memoization

Core idea: Cache the result. If input is same, return cached result instead of recomputing.

Problem it solves:

```
// This component re-renders every time parent re-renders
// Even if props didn't change → wasted work
function Parent() {
  const [count, setCount] = useState(0);

  return (
    <>
      <button onClick={() => setCount(count + 1)}>Click</button>
    </>
  );
}
```

```
        <ExpensiveChild name="Alice" /> {/* re-renders on every click!
    */}
    </>
  );
}
```

3 Tools for Memoization in React

1. React.memo — memoize a component

```
// Without memo - re-renders every time parent re-renders
function Child({ name }) {
  console.log("Child rendered");
  return <h1>{name}</h1>;
}

// With memo - only re-renders if props actually change
const Child = React.memo(function Child({ name }) {
  console.log("Child rendered");
  return <h1>{name}</h1>;
});

// Now clicking the button won't re-render Child
// because name="Alice" never changes
```

2. useMemo — memoize a computed value

```
function ProductList({ products, filter }) {
  // ❌ Without useMemo - recalculates on EVERY render
  const filtered = products.filter(p => p.category === filter);

  // ✅ With useMemo - only recalculates when products or filter
  changes
  const filtered = useMemo(() => {
```

```

    return products.filter(p => p.category === filter);
  }, [products, filter]);

  return filtered.map(p => <div key={p.id}>{p.name}</div>);
}

```

3. useCallback — memoize a function reference

```

function Parent() {
  const [count, setCount] = useState(0);

  // ❌ Without useCallback - new function created every render
  // React.memo on Child is useless because handleClick is always
  // "new"
  const handleClick = () => console.log("clicked");

  // ✅ With useCallback - same function reference across renders
  const handleClick = useCallback(() => {
    console.log("clicked");
  }, []); // no deps = never recreated

  return <Child onClick={handleClick} />;
}

const Child = React.memo(({ onClick }) => {
  console.log("Child rendered");
  return <button onClick={onClick}>Click</button>;
});

```

How Memoization Works Internally

First render:

```

useMemo(() => expensiveCalc(a, b), [a, b])
→ runs expensiveCalc(a, b)

```


→ stores result + deps [a, b] on fiber

Second render (a and b unchanged):

- compares old deps [a, b] vs new deps [a, b]
- `Object.is(a, a) = true`, `Object.is(b, b) = true`
- SKIP calculation, return cached result ✓

Third render (a changed):

- `Object.is(oldA, newA) = false`
- RE-RUN `expensiveCalc(newA, b)`
- store new result + new deps

When to use Memoization

✓ Use when:

- Component re-renders often but props rarely change (`React.memo`)
- You have expensive calculations in render (`useMemo`)
- You pass callbacks as props to memoized children (`useCallback`)

✗ Don't use when:

- Component is simple/cheap to render
- Props change on almost every render anyway
- Premature optimization – profile first!

Remember: Memoization itself has a cost (storing, comparing)

Only use it when the computation cost > comparison cost

Lazy Loading

Core idea: Don't load something until you actually need it. Split your bundle into chunks, load on demand.

Problem it solves:

Without lazy loading:

User visits homepage

→ Browser downloads ENTIRE app (all pages, all components)

→ 2MB JS bundle

→ Takes 5 seconds to load

→ User only needed the homepage!

With lazy loading:

User visits homepage

→ Browser downloads only homepage code (200KB)

→ Loads in 0.5 seconds

→ User navigates to /dashboard

→ THEN downloads dashboard code

3 Ways to Lazy Load in React

1. React.lazy + Suspense — lazy load components

```
import React, { lazy, Suspense } from 'react';

// ❌ Normal import - Dashboard JS loaded immediately on app start
import Dashboard from './Dashboard';

// ✅ Lazy import - Dashboard JS loaded only when component is
// rendered
const Dashboard = lazy(() => import('./Dashboard'));
const Settings = lazy(() => import('./Settings'));
const Profile = lazy(() => import('./Profile'));

function App() {
  return (
    <Suspense fallback=<div>Loading...</div>>
      { /* Dashboard code downloads only when this renders */ }
    <Dashboard />
  );
}
```

```

    </Suspense>
  );
}

```

2. Route-based lazy loading (most common)

```

const Home      = lazy(() => import('./pages/Home'));
const Dashboard = lazy(() => import('./pages/Dashboard'));
const Settings  = lazy(() => import('./pages/Settings'));

function App() {
  return (
    <BrowserRouter>
      <Suspense fallback={<Spinner />}>
        <Routes>
          <Route path="/"          element={<Home />} />
          <Route path="/dashboard" element={<Dashboard />} />
          <Route path="/settings"  element={<Settings />} />
        </Routes>
      </Suspense>
    </BrowserRouter>
  );
}

// User visits /           → only Home.js downloaded
// User visits /dashboard → Dashboard.js downloaded NOW
// User visits /settings  → Settings.js downloaded NOW

```

3. Conditional lazy loading — load on interaction

```

const HeavyModal = lazy(() => import('./HeavyModal'));

function App() {
  const [showModal, setShowModal] = useState(false);

```

```

return (
  <>
    <button onClick={() => setShowModal(true)}>Open Modal</button>

    {/* HeavyModal code is NOT downloaded until button is clicked
    */}
    {showModal && (
      <Suspense fallback={<div>Loading modal...</div>}>
        <HeavyModal />
      </Suspense>
    )}
  </>
);
}

```

How Lazy Loading Works Internally

`import('./Dashboard')` ← dynamic import (webpack/vite code splits here)

↓

Creates a separate JS chunk file: `Dashboard.chunk.js`

↓

When React.lazy component is first rendered:

1. React sees it's a lazy component
2. Triggers the `import()` → network request for chunk
3. Suspense catches the "pending" promise → shows fallback
4. Chunk downloads and executes
5. Promise resolves → Suspense swaps fallback for real component

Network tab without lazy loading:

main.js → 2.1MB (everything bundled together)

Network tab with lazy loading:

```
main.js      → 300KB (core app)
home.chunk   → 50KB  (loaded on visit to /)
dashboard.chunk → 120KB (loaded on visit to /dashboard)
settings.chunk → 80KB (loaded on visit to /settings)
```

Side-by-Side Comparison

User visits your app

WITHOUT any optimization:

- Downloads 2MB bundle (all pages)
- Renders everything on every state change
- Slow initial load + janky UI

WITH lazy loading only:

- Downloads 300KB bundle (only what's needed)
- Fast initial load ✓
- But still re-renders unnecessarily ✗

WITH memoization only:

- Downloads 2MB bundle (everything)
- Slow initial load ✗
- But no wasted re-renders ✓

WITH both:

- Downloads only needed code (fast load) ✓
- No wasted re-renders (smooth UI) ✓

Real World Analogy

Memoization = A chef who **remembers** the last dish they made. If same order comes again, they serve the cached dish instead of cooking again.

Lazy Loading = A restaurant that **only stocks ingredients** when a customer orders that dish. Not everything is in the kitchen from day one.

Quick Decision Guide

Component re-renders too often?

→ `React.memo` + `useCallback`

Expensive calculation inside render?

→ `useMemo`

App takes too **long** to load initially?

→ `React.lazy` + route-based splitting

Feature used **rarely** (modal, admin panel)?

→ `React.lazy` + conditional rendering

Images loading slowly?

→ `loading="lazy"` on `<img>` tag

``

Summary

	Memoization	Lazy Loading
Problem	Too many re-renders	Bundle too large
When	Runtime (during renders)	Load time (initial/on-demand)
Tools	<code>React.memo</code> , <code>useMemo</code> , <code>useCallback</code>	<code>React.lazy</code> , <code>dynamic import()</code>
Saves	CPU / render time	Network bandwidth / load time
Works on	Components, values, functions	Components, routes, modules

Use both together for a truly performant React app. 🚀

3/14/2026, 6:16:37 PM



1.

input -

```
(function immediateA(a) {  
  return (function immediateB(b) {  
    console.log(a);  
  })(1);  
})(0);
```

output - ??

2. do u know iife ??

3.

input -

```
for (var i = 0; i < 3; i++) {  
  setTimeout(function log() {  
    console.log(i); // What is logged?  
  }, 1000);  
}
```

output - ??

0

1

2

4.

Input -

```
const object = {  
  who: 'World',  
  greet() {  
    return Hello, ${this.who}!;  
  },  
  farewell: () => {  
    return Goodbye, ${this.who}!; }  
};
```

console.log(object.greet()); // What is logged? //Hello World!


```
console.log(object.farewell()); // What is logged? //Goodbye, World!
```

output - ??

5.

input -

```
try {  
  console.log("starting");  
  setTimeout(() => {  
    console.log("inside setTimeout");  
    throw new Exception("error");  
  }, 10);  
  console.log("just before catch");  
} catch (e) {  
  console.log("in catch");  
}  
console.log("outside try");
```

output - ??

=====

starting

after 10 sec

inside setTimeout

in catch

Q1.. Write a function to convert a string from camel case to snake case.

for e.g

input : thisIsATest

output : This_Is_A_Test

Q2.. You are given a nested array, you have to write a function to flatten it.

For eg.

1.

Input - [1, [2]]

Output - [1,2]

2.

Input - [1, [2], [3 , [4]]]

Output - [1,2,3,4]

You could have arbitrary levels of nesting.

Q3..

Create a countdown timer that will countdown from 30 and update every second.

When the countdown hits 0, display 'Take Off'

There is a Reset button next to the timer.

The Reset button will reset the timer to 30 sec if the takeoff has not already happened.'

After takeoff, reset is not possible.

Q4..

how do we improve performance of a UI page

3/16/2026, 4:16:23 PM



Let me answer all of these one by one.

Output Questions

Q1.

```
(function immediateA(a) {  
  return (function immediateB(b) {  
    console.log(a);  
  })(1);  
})(0);
```

Output: 0

Why?

- `immediateA` is called with `a = 0`
- Inside it, `immediateB` is called with `b = 1`
- `console.log(a)` — `a` is NOT `b`. It logs `a` from outer scope via **closure**

- `a = 0` → logs `0`

```
a = 0 (outer)
b = 1 (inner, never used)
console.log(a) → 0 ✓
```

Q2. Do you know IIFE?

IIFE = Immediately Invoked Function Expression

A function that is **defined and called at the same time**.

```
// Normal function - define now, call later
function greet() { console.log("Hi"); }
greet();

// IIFE - define and call immediately
(function() {
  console.log("I run immediately!");
})();

// With params
(function(name) {
  console.log("Hello", name);
})("Alice"); // → Hello Alice

// Arrow IIFE
(() => console.log("Arrow IIFE"))();
```

Why use IIFE?

- Avoid polluting global scope
- Create a private scope
- Used heavily before ES6 modules existed

```
// Variable stays inside IIFE, doesn't leak globally
(function() {
  var secret = "hidden";
})();
console.log(secret); // ReferenceError 
```

Q3.

```
for (var i = 0; i < 3; i++) {
  setTimeout(function log() {
    console.log(i);
  }, 1000);
}
```

Output: **3, 3, 3** (not 0, 1, 2)

Why?

- **var** is function-scoped — only ONE **i** exists
- setTimeout callbacks run after the loop finishes
- By then **i = 3** for all 3 callbacks
- All 3 log **3**

```
Loop runs:  i=0 → schedule, i=1 → schedule, i=2 → schedule
Loop ends:  i = 3
After 1 sec: all 3 callbacks fire → all see i=3
Output: 3, 3, 3
```

Fix 1 — use **let** (block scoped, new **i** each iteration)

```
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 1000);
}
```

```
}  
// Output: 0, 1, 2 
```

Fix 2 — use IIFE to capture 

```
for (var i = 0; i < 3; i++) {  
  (function(j) {  
    setTimeout(() => console.log(j), 1000);  
  })(i); // pass i as j, captured in closure  
}  
// Output: 0, 1, 2 
```

Q4.

```
const object = {  
  who: 'World',  
  greet() {  
    return `Hello, ${this.who}!`;  
  },  
  farewell: () => {  
    return `Goodbye, ${this.who}!`;  
  }  
};  
console.log(object.greet());  
console.log(object.farewell());
```

Output:

```
Hello, World!  
Goodbye, undefined!
```

Why?

- `greet()` is a **regular function** → `this` = the object → `this.who = 'World'` ✓
- `farewell` is an **arrow function** → `this` = outer lexical scope (window/undefined in strict mode) → `this.who = undefined` ✗

Regular method → `this` = **object** calling it

Arrow method → `this` = wherever the arrow was DEFINED (outer scope)

Q5.

```
try {
  console.log("starting");
  setTimeout(() => {
    console.log("inside setTimeout");
    throw new Error("error");
  }, 10);
  console.log("just before catch");
} catch (e) {
  console.log("in catch");
}
console.log("outside try");
```

Output:

```
starting
just before catch
outside try
inside setTimeout
Uncaught Error: error ← crashes, NOT caught by try/catch
```

Why? This is a very important concept.

- `try/catch` runs **synchronously**
- By the time `setTimeout` fires (after 10ms), the `try/catch` block is **already done and gone**

- The error thrown inside `setTimeout` has **no try/catch to catch it**
- It becomes an **uncaught error**

```
try block runs synchronously:  
  → "starting"  
  → schedules setTimeout (puts it in macrotask queue)  
  → "just before catch"  
try block ends (catch never triggered)  
  → "outside try"  
  
10ms later, event loop picks up setTimeout:  
  → "inside setTimeout"  
  → throw Error → nobody catches it → app crashes
```

Fix — handle error inside `setTimeout`:

```
setTimeout(() => {  
  try {  
    console.log("inside setTimeout");  
    throw new Error("error");  
  } catch(e) {  
    console.log("caught inside:", e.message); //   
  }  
}, 10);
```

Coding Questions

Q1. Camel Case to Snake Case

```
// input:  thisIsATest  
// output: this_Is_A_Test  
  
function camelToSnake(str) {
```

```

    return str
      .replace(/([A-Z])/g, '_$1'); // before every capital letter, add
    —
  }

  console.log(camelToSnake("thisIsATest")); // this_Is_A_Test

```

How the regex works:

```

/([A-Z])/g → find every uppercase letter
'_$1'      → replace it with _ + that letter

thisIsATest
  ↑ ↑ ↑
  I A T → each gets _ before it

→ this_Is_A_Test

```

Q2. Flatten Nested Array

```

// input:  [1, [2], [3, [4]]]
// output: [1, 2, 3, 4]

// Method 1 – Recursion (most important to know)
function flatten(arr) {
  let result = [];
  for (let item of arr) {
    if (Array.isArray(item)) {
      result = result.concat(flatten(item)); // recurse into nested
array
    } else {
      result.push(item);
    }
  }
}

```



```

    return result;
}

// Method 2 – arr.flat(Infinity) built-in
[1, [2], [3, [4]]].flat(Infinity); // [1, 2, 3, 4]

// Method 3 – reduce + recursion (one-liner)
const flatten = arr =>
  arr.reduce((acc, item) =>
    acc.concat(Array.isArray(item) ? flatten(item) : item), []);

// All three:
console.log(flatten([1, [2]]));           // [1, 2]
console.log(flatten([1, [2], [3, [4]]])); // [1, 2, 3, 4]
console.log(flatten([1, [2, [3, [4, [5]]]]])); // [1, 2, 3, 4, 5]

```

How recursion works:

```

flatten([1, [2], [3, [4]]])
  1      → push 1
  [2]    → isArray → flatten([2])
           2      → push 2
  [3, [4]] → isArray → flatten([3, [4]])
                3      → push 3
                [4]    → isArray → flatten([4])
                        4      → push 4

Result: [1, 2, 3, 4] ✓

```

Q3. Countdown Timer

```

import { useState, useEffect, useRef } from "react";

function CountdownTimer() {

```

```

const [time, setTime] = useState(30);
const [launched, setLaunched] = useState(false);
const timerRef = useRef(null);

useEffect(() => {
  // Start countdown
  timerRef.current = setInterval(() => {
    setTime(prev => {
      if (prev <= 1) {
        clearInterval(timerRef.current);
        setLaunched(true);
        return 0;
      }
      return prev - 1;
    });
  }, 1000);

  // Cleanup on unmount
  return () => clearInterval(timerRef.current);
}, []);

function handleReset() {
  if (launched) return; // after takeoff, no reset
  clearInterval(timerRef.current);
  setTime(30);

  // Restart countdown
  timerRef.current = setInterval(() => {
    setTime(prev => {
      if (prev <= 1) {
        clearInterval(timerRef.current);
        setLaunched(true);
        return 0;
      }
      return prev - 1;
    });
  }, 1000);
}

```

```

    });
    }, 1000);
  }

  return (
    <div>
      {launched
        ? <h1>🚀 Take Off!</h1>
        : <h1>{time}s</h1>
      }
      <button onClick={handleReset} disabled={launched}>
        Reset
      </button>
    </div>
  );
}

```

Key points:

useRef stores the interval **ID** (persists across renders, no re-render on change)
 setTime(prev => prev - 1) uses functional **update** (safe inside setInterval)
 clearInterval on cleanup → prevents memory leak
 disabled={launched} → button disabled after takeoff

Q4. How to improve performance of a UI page?

JS / React Level

- ✓ React.memo → prevent unnecessary re-renders
- ✓ useMemo → cache expensive calculations
- ✓ useCallback → stable **function references**
- ✓ Lazy loading → React.lazy + **dynamic** import()

- ✓ Code splitting → route-based chunks
- ✓ Virtualization → render **only** visible list items (react-**window**, react-virtual)
- ✓ Debounce/Throttle → limit API calls **on** input/**scroll**
- ✓ Avoid anonymous → functions **in** JSX **create new** refs
functions **in** JSX

Network Level

- ✓ Minify JS/CSS → smaller bundle size
- ✓ Compress assets → gzip / brotli
- ✓ CDN → serve from nearest server
- ✓ Caching → Cache-Control headers
- ✓ Lazy load images → `<img loading="lazy" />`
- ✓ Use WebP images → smaller than PNG/JPG
- ✓ Prefetch/Preload → `<link rel="prefetch">`

CSS Level

- ✓ Avoid layout thrashing → don't mix read/write DOM
- ✓ Use transform/opacity → GPU **accelerated** (no reflow)
- ✓ Avoid * selector → slow, matches everything
- ✓ CSS containment → contain: layout style

Measuring Performance

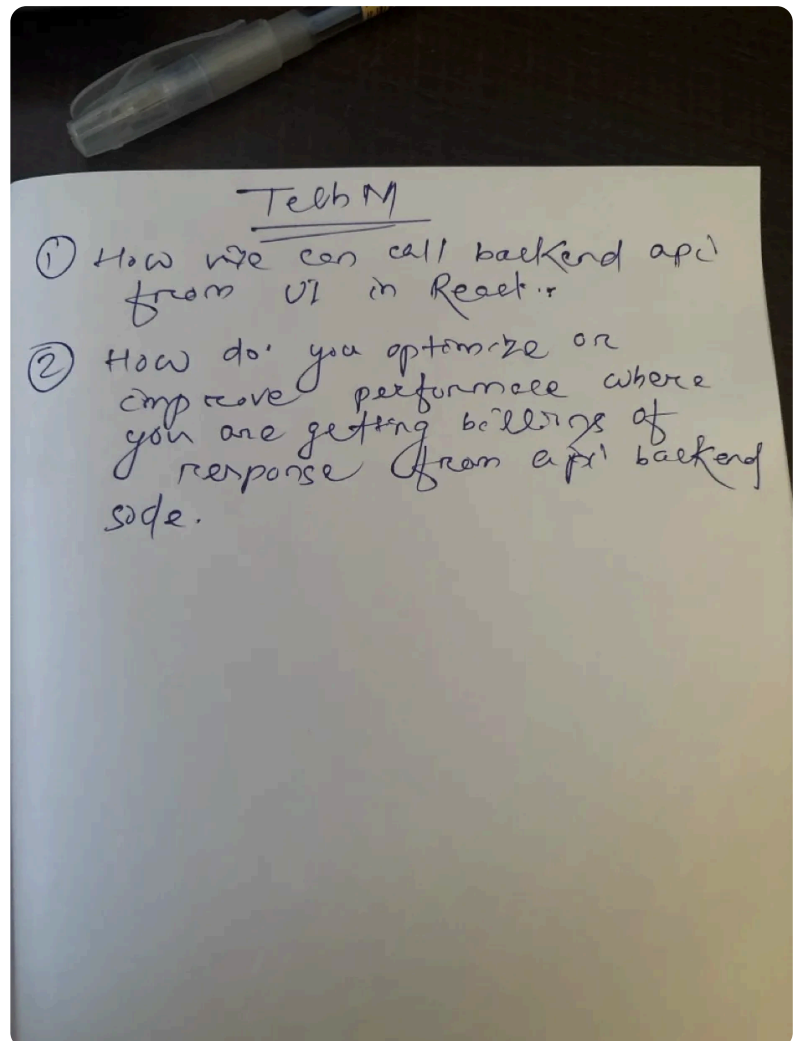
- ✓ Chrome DevTools → Performance tab
- ✓ React DevTools → Profiler tab (see what re-renders)
- ✓ Lighthouse → audit score
- ✓ Web Vitals:
 - LCP (Largest Contentful Paint) < 2.5s

- FID (First Input Delay) < 100ms
- CLS (Cumulative Layout Shift) < 0.1

Most Impactful in React specifically

1. Virtualize long lists → biggest win for lists
2. Route-based code splitting → biggest win for load time
3. Image optimization → biggest win for network
4. React.memo on heavy components → biggest win for renders

3/16/2026, 4:17:19 PM



Answer this

3/20/2026, 4:51:22 PM



TechM Questions

Q1. How we can call Backend API from UI in React?

There are **3 main ways**:

Method 1 — fetch (built-in, no library needed)

```
function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  useEffect(() => {
    setLoading(true);
    fetch("https://api.example.com/users")
      .then(res => {
        if (!res.ok) throw new Error("API failed");
        return res.json();
      })
      .then(data => setUsers(data))
      .catch(err => setError(err.message))
      .finally(() => setLoading(false));
  }, []); // [] = call only once on mount

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <ul>
      {users.map(u => <li key={u.id}>{u.name}</li>)}
    </ul>
  );
}
```

Method 2 — axios (most popular in industry)

```
import axios from 'axios';

function UserList() {
```

```

const [users, setUsers] = useState([]);

useEffect(() => {
  axios.get("https://api.example.com/users")
    .then(res => setUsers(res.data)) // axios auto-parses JSON
    .catch(err => console.error(err));
}, []);
}

// POST request with axios
async function createUser(userData) {
  try {
    const res = await axios.post("/api/users", userData, {
      headers: { "Content-Type": "application/json" }
    });
    console.log("Created:", res.data);
  } catch (err) {
    console.error(err);
  }
}

```

Method 3 — async/await (cleanest syntax)

```

function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(false);

  useEffect(() => {
    // Can't make useEffect itself async
    // So define async function inside and call it
    async function fetchUsers() {
      try {
        setLoading(true);
        const res = await fetch("/api/users");
        const data = await res.json();

```



```

        setUsers(data);
    } catch (err) {
        console.error(err);
    } finally {
        setLoading(false);
    }
}

fetchUsers(); // call it
}, []);
}

```

Method 4 — Custom Hook (best practice, reusable)

```

// useFetch.js - reusable for any API call
function useFetch(url) {
    const [data, setData] = useState(null);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState(null);

    useEffect(() => {
        async function fetchData() {
            try {
                const res = await fetch(url);
                const json = await res.json();
                setData(json);
            } catch (err) {
                setError(err.message);
            } finally {
                setLoading(false);
            }
        }
        fetchData();
    }, [url]);
}

```

```
    return { data, loading, error };
}

// Use anywhere in any component
function Users() {
    const { data, loading, error } = useFetch("/api/users");

    if (loading) return <p>Loading...</p>;
    if (error)    return <p>Error: {error}</p>;
    return <ul>{data.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}

function Posts() {
    const { data, loading } = useFetch("/api/posts"); // reuse same
hook!
}
```

All HTTP methods:

```
// GET - fetch data
axios.get("/api/users");

// POST - create new data
axios.post("/api/users", { name: "Alice" });

// PUT - update entire record
axios.put("/api/users/1", { name: "Alice Updated" });

// PATCH - update partial record
axios.patch("/api/users/1", { name: "Alice" });

// DELETE - delete record
axios.delete("/api/users/1");
```

Q2. How do you optimize/improve performance where you are getting billions of responses from API (backend side)?

This is a great system design question. You tackle it at **multiple levels**:

Problem:

Backend returns 1,000,000 records
→ Browser tries to render all at once
→ Page freezes, crashes, unusable ❌

Solution 1 — Pagination (most common)

Don't fetch all data. Fetch page by page.

```
function UserList() {
  const [users, setUsers] = useState([]);
  const [page, setPage] = useState(1);
  const [hasMore, setHasMore] = useState(true);

  useEffect(() => {
    // Only fetch 20 records at a time
    fetch(`/api/users?page=${page}&limit=20`)
      .then(res => res.json())
      .then(data => {
        setUsers(data.items);
        setHasMore(data.hasNextPage);
      });
  }, [page]);

  return (
    <>
      {users.map(u => <div key={u.id}>{u.name}</div>)}
      <button disabled={page === 1} onClick={() => setPage(p => p -
```

```

1)}}>
    Prev
  </button>
  <button disabled={!hasMore} onClick={() => setPage(p => p +
1)}}>
    Next
  </button>
</>
);
}

```

Solution 2 — Infinite Scroll (load more on scroll)

```

function InfiniteList() {
  const [items, setItems] = useState([]);
  const [page, setPage] = useState(1);
  const loaderRef = useRef(null);

  // Fetch next page
  useEffect(() => {
    fetch(`/api/data?page=${page}&limit=20`)
      .then(r => r.json())
      .then(data => setItems(prev => [...prev, ...data]));
  }, [page]);

  // Watch when loader div enters the screen
  useEffect(() => {
    const observer = new IntersectionObserver(entries => {
      if (entries[0].isIntersecting) {
        setPage(prev => prev + 1); // load next page when reached
        bottom
      }
    });
    if (loaderRef.current) observer.observe(loaderRef.current);
    return () => observer.disconnect();
  }, [loaderRef]);
}

```

```

}, []);

return (
  <>
    {items.map(item => <div key={item.id}>{item.name}</div>)}
    <div ref={loaderRef}>Loading more...</div>
  </>
);
}

```

Solution 3 — Virtualization (most powerful for large lists)

Render **only the items visible on screen**, not all million items in DOM.

```

import { FixedSizeList } from 'react-window';

// Even if you have 1,000,000 items
// Only ~10 visible ones are in the DOM at any time
function VirtualList({ items }) {
  const Row = ({ index, style }) => (
    <div style={style}> { /* style has position - required! */}
      {items[index].name}
    </div>
  );

  return (
    <FixedSizeList
      height={600}           // visible area height
      itemCount={1000000}   // total items
      itemSize={50}         // each row height
      width="100%"
    >
      {Row}
    </FixedSizeList>
  );
}

```

```
);  
}
```

Without virtualization: 1,000,000 DOM nodes → browser dies ❌

With virtualization: ~12 DOM nodes at any time → smooth ✅

Solution 4 — Debounce Search / Filter API calls

```
function SearchUsers() {  
  const [query, setQuery] = useState("");  
  const [results, setResults] = useState([]);  
  
  useEffect(() => {  
    // Wait 500ms after user stops typing before calling API  
    const timer = setTimeout(() => {  
      if (query) {  
        fetch(`/api/users/search?q=${query}`)  
          .then(r => r.json())  
          .then(setResults);  
      }  
    }, 500);  
  
    return () => clearTimeout(timer); // cancel if user types again  
  }, [query]);  
  
  return (  
    <>  
      <input  
        value={query}  
        onChange={e => setQuery(e.target.value)}  
        placeholder="Search..."  
      />  
      {results.map(u => <div key={u.id}>{u.name}</div>)}  
    </>  
  )  
}
```

```
);  
}
```

Solution 5 — Caching with React Query

```
import { useQuery } from '@tanstack/react-query';  
  
function Users() {  
  const { data, isLoading } = useQuery({  
    queryKey: ['users', page],  
    queryFn: () => fetch(`/api/users?page=${page}`).then(r =>  
r.json()),  
    staleTime: 5 * 60 * 1000, // cache for 5 minutes  
    // Same API won't be called again if called within 5 mins  
  });  
}  
  
// Benefits:  
// ✅ Automatic caching  
// ✅ Background refetching  
// ✅ No duplicate requests  
// ✅ Loading/error states built in
```

Solution 6 — Request only what you need (API side)

```
// ❌ Bad - get everything  
GET /api/users  
// Returns: id, name, email, address, phone, photo, bio, settings...  
  
// ✅ Good - get only needed fields  
GET /api/users?fields=id,name,email  
// Returns only: id, name, email  
  
// ✅ Even better - use GraphQL (request exactly what you need)
```

```
query {  
  users {  
    id  
    name    // only these 2 fields, nothing else  
  }  
}
```

Full Strategy Summary:

Billions of records problem → attack at every level:

DATA LEVEL:

- ✓ Pagination → fetch 20 at a time
- ✓ Infinite scroll → load more as user scrolls
- ✓ Search/filter → reduce dataset before fetching
- ✓ Request only needed fields

RENDER LEVEL:

- ✓ Virtualization → only render visible rows (react-window)
- ✓ React.memo → don't re-render unchanged rows

NETWORK LEVEL:

- ✓ Caching (React Query) → don't re-fetch same data
- ✓ Debounce search → reduce API call frequency
- ✓ Compression (gzip) → smaller response size
- ✓ CDN → serve from nearest location

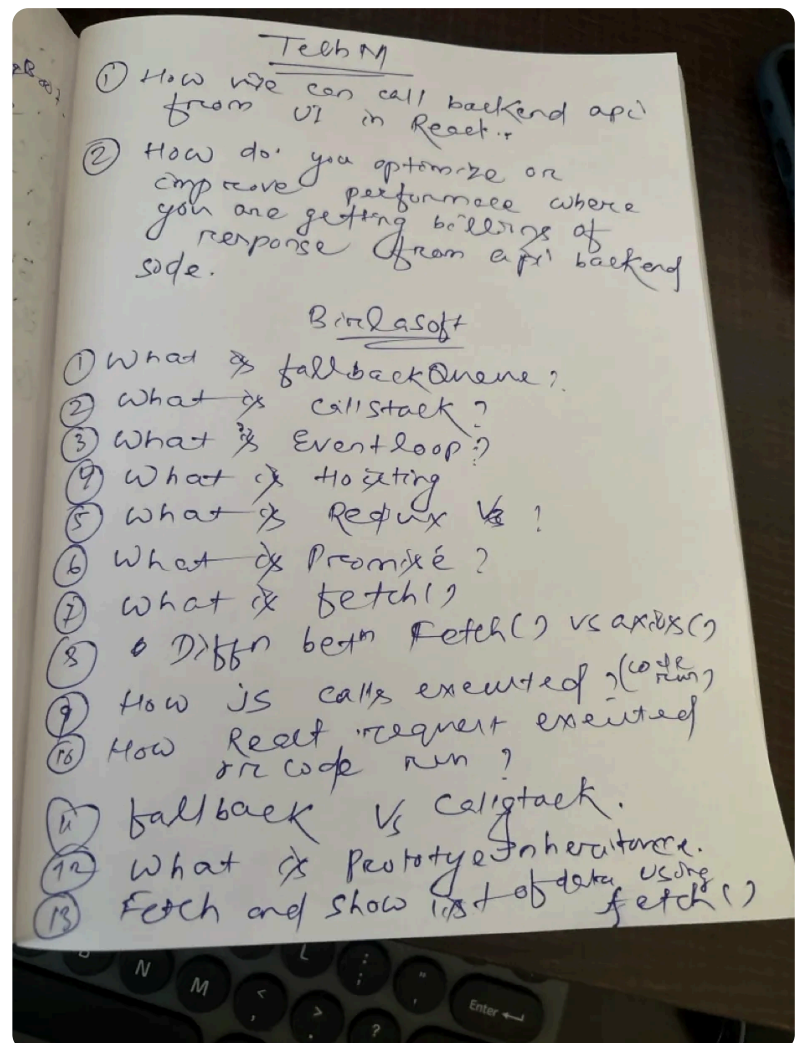
PERCEIVED PERFORMANCE:

- ✓ Skeleton screens → show placeholder while loading
- ✓ Loading spinners → user knows something is happening
- ✓ Optimistic updates → update UI before API confirms

The golden rule: Never fetch what you don't show, never render what the user can't see.



3/20/2026, 4:52:04 PM



Answer birlasoft questions

3/24/2026, 1:20:28 PM



Birlasoft Questions

Q1. What is Callback Queue?

The Callback Queue (also called Task Queue / Macrotask Queue) is where async callbacks wait after completing, before being picked up by the Event Loop and pushed to the Call Stack.

```

setTimeout(() => console.log("A"), 0);
Promise.resolve().then(() => console.log("B"));
console.log("C");

```

// Output: C → B → A

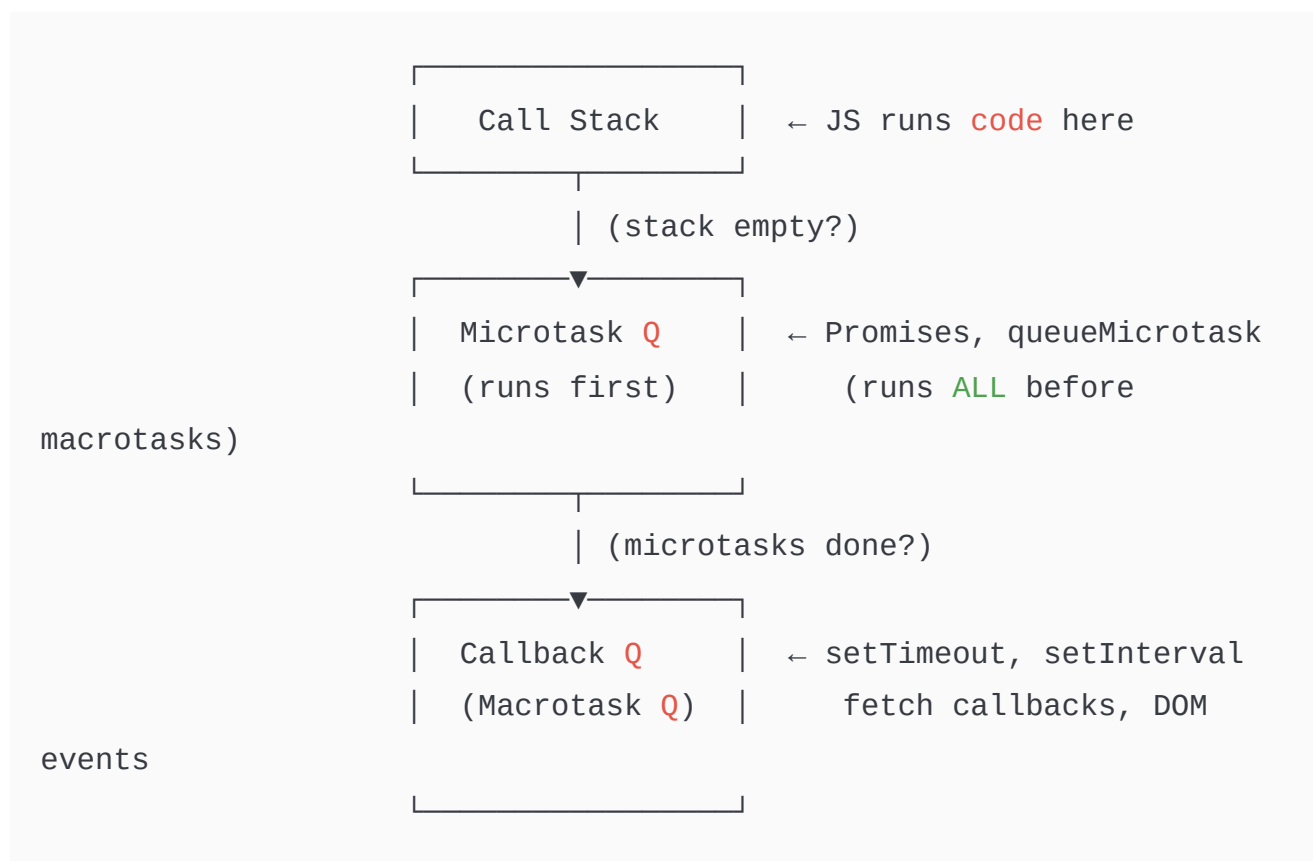
Why?

C → runs immediately (call stack)

B → goes to **Microtask Queue (Promise)** - runs before callback queue

A → goes to **Callback Queue (setTimeout)** - runs last

Full picture:



Q2. What is Call Stack?

A data structure (LIFO — Last In First Out) that tracks which function is currently executing.

When a function is called it's pushed on, when it returns it's popped off.

```

function C() { console.log("C"); }
function B() { C(); }
function A() { B(); }

A();

// Call Stack step by step:
// [A]           ← A pushed
// [A, B]        ← B pushed (A called B)
// [A, B, C]     ← C pushed (B called C)
// [A, B]        ← C popped (C finished)
// [A]           ← B popped (B finished)
// []           ← A popped (A finished)

```

Stack Overflow = stack grows too large (infinite recursion):

```

function infinite() { infinite(); } // calls itself forever
infinite(); // Uncaught RangeError: Maximum call stack size exceeded

```

Q3. What is Event Loop?

The Event Loop is a mechanism that continuously checks:

- Is the Call Stack empty?
- If yes → pick next task from Microtask Queue or Callback Queue and push it to stack

```

console.log("1");           // sync → call stack

setTimeout(() => {
  console.log("4");         // macrotask → callback queue
}, 0);

Promise.resolve().then(() => {
  console.log("3");         // microtask → microtask queue

```

```
});

console.log("2");           // sync → call stack

// Output: 1 → 2 → 3 → 4

// Why?
// 1, 2 run synchronously first
// Then microtasks (Promise → "3")
// Then macrotasks (setTimeout → "4")
```

Event Loop in one sentence: It keeps checking "is the stack empty? then push next task."

Q4. What is Hoisting?

JavaScript moves **declarations** to the top of their scope before execution. Only declarations are hoisted, NOT initializations.

```
// var - hoisted, initialized as undefined
console.log(a); // undefined (not error!)
var a = 5;
console.log(a); // 5

// let/const - hoisted but NOT initialized (Temporal Dead Zone)
console.log(b); // ReferenceError: Cannot access before init
let b = 5;

// Function declaration - FULLY hoisted
greet(); // "Hello" ✅ works before definition
function greet() { console.log("Hello"); }

// Function expression - NOT hoisted
sayBye(); // TypeError ❌
var sayBye = function() { console.log("Bye"); };
```

Q5. What is Redux vs (Context API)?

	Redux	Context API
Size	External library	Built into React
Boilerplate	More (actions, reducers)	Less
DevTools	✅ Time-travel debugging	❌ Basic
Performance	Optimized (selective re-render)	All consumers re-render
Best for	Large complex apps	Small/medium apps
Middleware	✅ thunk, saga (async)	❌ Manual

```
// Redux flow
Action → Reducer → Store → Component

store.dispatch({ type: 'ADD_USER', payload: user }); // action
// reducer processes it → updates store → component re-renders

// Context - simpler, no actions/reducers
const ctx = createContext();
<ctx.Provider value={data}>
  <Child /> // useContext(ctx) to access
</ctx.Provider>
```

Rule of thumb:

```
Small app, simple sharing → Context API
Large app, complex state → Redux Toolkit
```

Q6. What is Promise?

A Promise is an object representing the eventual completion or failure of an async operation. It has 3 states: **pending** → **fulfilled** / **rejected**

```

// Creating a Promise
const promise = new Promise((resolve, reject) => {
  setTimeout(() => {
    const success = true;
    if (success) resolve("Data loaded!"); // fulfilled
    else reject("Something went wrong"); // rejected
  }, 1000);
});

// Consuming a Promise
promise
  .then(data => console.log(data)) // "Data loaded!"
  .catch(err => console.log(err)) // if rejected
  .finally(() => console.log("Done")); // always runs

// Async/await (cleaner way to consume)
async function fetchData() {
  try {
    const data = await promise;
    console.log(data);
  } catch (err) {
    console.error(err);
  }
}

```

Promise combinators:

```

// Run all in parallel, wait for ALL to finish
Promise.all([p1, p2, p3])
  .then(([r1, r2, r3]) => console.log(r1, r2, r3));
// ⚠️ If ANY fails → goes to catch

// Run all, get ALL results (success or fail)
Promise.allSettled([p1, p2, p3])

```

```
.then(results => results.forEach(r => console.log(r.status)));

// First one to resolve wins
Promise.race([p1, p2, p3])
  .then(first => console.log("fastest:", first));
```

Q7. What is fetch()?

fetch() is the built-in browser API to make HTTP requests. It returns a Promise.

```
// Basic GET
fetch("https://api.example.com/users")
  .then(res => res.json())          // parse JSON (also returns
  Promise)
  .then(data => console.log(data))
  .catch(err => console.error(err));

// POST request
fetch("/api/users", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name: "Alice", age: 25 })
})
  .then(res => res.json())
  .then(data => console.log("Created:", data));

// With async/await
async function getUsers() {
  const res = await fetch("/api/users");
  const data = await res.json();
  return data;
}
```

Q8. Difference between fetch() vs axios()?

	fetch	axios
Built-in	✅ Yes (no install)	❌ npm install axios
JSON parsing	Manual (<code>res.json()</code>)	Automatic
Error handling	Only network errors	HTTP errors too (404, 500)
Request timeout	❌ No built-in	✅ Yes
Interceptors	❌ No	✅ Yes
Progress tracking	❌ No	✅ Yes
Older browsers	❌ Needs polyfill	✅ Works

```

// fetch - must manually parse JSON, 404 is NOT an error
const res = await fetch("/api/users");
if (!res.ok) throw new Error("HTTP error: " + res.status); // manual
check!
const data = await res.json(); // manual parse

// axios - auto parses, 404 throws automatically
const { data } = await axios.get("/api/users"); // done! ✅

// axios interceptors (powerful feature fetch doesn't have)
axios.interceptors.request.use(config => {
  config.headers.Authorization = `Bearer ${getToken()}`;
  return config; // auto-add token to every request
});

axios.interceptors.response.use(
  res => res,
  err => {
    if (err.response.status === 401) logout(); // auto logout on 401
    return Promise.reject(err);
  }
);

```

Q9. How JS calls are executed at code runtime?

Source code

↓

JS Engine (V8 in Chrome/Node)

↓

Parser → AST (Abstract Syntax Tree)

↓

Interpreter → Bytecode (runs immediately)

↓

JIT Compiler → Optimized Machine Code (hot paths)

Execution flow:

1. Global Execution Context created

- Memory phase: all vars hoisted, functions stored
- Execution phase: code runs line by line

2. Each function call → new Execution Context

- Has its own: variables, arguments, this
- Pushed onto Call Stack

3. Function returns → Execution Context destroyed

- Popped from Call Stack

```
var x = 10;
```

```
function multiply(a, b) {  
  return a * b;  
}
```

```
var result = multiply(x, 2);
```

```
// Memory phase:
```

```
// x = undefined, multiply = fn, result = undefined

// Execution phase:
// x = 10
// multiply called → new EC pushed
//   a=10, b=2 → returns 20 → EC popped
// result = 20
```

Q10. How React request is executed / code run?

You write JSX

↓

Babel compiles JSX → React.createElement() calls

↓

React Elements created (plain JS objects)

↓

React Fiber builds Virtual DOM tree

↓

Reconciliation: diff **old** tree vs **new** tree

↓

Commit Phase: **update** **real** DOM (**only** changes)

↓

Browser repaints screen

↓

useEffect() runs (after paint)

```
// 1. You write
function App() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count+1)}>{count}</button>;
}

// 2. Babel compiles to
React.createElement("button",
```

```

    { onClick: () => setCount(count+1) },
    count
  )

// 3. React Element (plain object)
{ type: "button", props: { onClick: fn, children: 0 } }

// 4. Fiber processes it, creates DOM node
// 5. ReactDOM inserts <button>0</button> into real DOM
// 6. User clicks → setCount → re-render → only text "0" changes to "1"

```

Q11. Callback vs Call Stack

	Callback	Call Stack
What it is	A function passed to another function to run later	Data structure tracking current execution
When used	Async operations (setTimeout, events, fetch)	Every function call, always
Where it waits	Callback Queue	Active execution frame

```

// Call Stack - immediate, synchronous
function greet() { console.log("Hi"); } // pushed to stack, runs NOW
greet();

// Callback - deferred, async
setTimeout(function cb() { // cb goes to Callback Queue
  console.log("Later"); // runs AFTER stack is clear
}, 1000);

// How they connect:
// Event Loop sees stack is empty
// Takes cb from Callback Queue

```

```
// Pushes cb onto Call Stack
// cb runs
```

Q12. What is Prototype Inheritance?

Every JS object has a hidden `__proto__` link to another object (its prototype). When you access a property, JS looks up this chain until found or reaches null.

```
// Constructor function
function Animal(name) {
  this.name = name;
}
Animal.prototype.speak = function() {
  return `${this.name} makes a sound`;
};

const dog = new Animal("Rex");
dog.speak(); // "Rex makes a sound"
// dog has no 'speak' → looks up __proto__ → finds it on
Animal.prototype ✅

// Prototype chain:
// dog → Animal.prototype → Object.prototype → null

// ES6 Class (same thing, cleaner syntax)
class Animal {
  constructor(name) { this.name = name; }
  speak() { return `${this.name} makes a sound`; }
}

class Dog extends Animal {
  bark() { return "Woof!"; }
}

const d = new Dog("Rex");
```

d.bark(); // found on Dog.prototype ✓

d.speak(); // not on Dog → goes up → found on Animal.prototype ✓

Q13. Fetch and show list of data using fetch()

```
function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then(res => {
        if (!res.ok) throw new Error("Failed to fetch");
        return res.json();
      })
      .then(data => {
        setUsers(data);
        setLoading(false);
      })
      .catch(err => {
        setError(err.message);
        setLoading(false);
      });
  }, []);

  if (loading) return <p>Loading...</p>;
  if (error) return <p style={{color:"red"}}>{error}</p>;

  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>
          <strong>{user.name}</strong> – {user.email}
        </li>
      ))}
    </ul>
  );
}
```

```
        </li>
    }}}
</ul>
);
}

// Output on screen:
// • Leanne Graham – sincere@april.biz
// • Ervin Howell – shanna@melissa.tv
// • ...
```

Always handle 3 states when fetching:

```
loading → show spinner/skeleton
error   → show error message
success → show data
```

3/24/2026, 1:21:38 PM



Is this promise

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then(res => {
    if (!res.ok) throw new Error("Failed to fetch");
    return res.json();
  })
  .then(data => {
    setUsers(data);
    setLoading(false);
  })
  .catch(err => {
    setError(err.message);
    setLoading(false);
  });
```

3/24/2026, 1:30:10 PM



Yes! This is exactly a Promise chain. Let me break it down completely.

Yes, fetch() returns a Promise

```
const promise = fetch("https://jsonplaceholder.typicode.com/users");
console.log(promise); // Promise { <pending> }
```

Breaking Down Every Line

```
fetch("https://jsonplaceholder.typicode.com/users")
```


`fetch()` makes an HTTP request
returns a Promise (pending state)
waiting for server to respond...

```
.then(res => {  
  if (!res.ok) throw new Error("Failed to fetch");  
  return res.json();  
})
```

When Promise resolves (server responded)

→ `res = Response object {status: 200, ok: true, body: ...}`

`res.ok` checks if status is 200-299

if NOT ok (404, 500 etc) → manually throw error → goes to `.catch`

`res.json()` → ALSO returns a Promise!

(reading the body stream is async too)

→ so we return it to chain the next `.then`

```
.then(data => {  
  setUsers(data);  
  setLoading(false);  
})
```

When `res.json()` Promise resolves

→ `data = actual` parsed JS array/object

→ now we use the `data`

```
.catch(err => {  
  setError(err.message);  
})
```

```
    setLoading(false);  
  })
```

If ANY **.then** above throws an **error**
→ jumps directly here
→ **handles** the **error**

Visual Flow

```
fetch()                                → Promise<Response> (pending)  
  ↓ server responds  
.then(res => res.json())               → Promise<Data> (pending)  
  ↓ body parsed  
.then(data => setUsers)               → done ✅  
  
if error anywhere ↓  
.catch(err => setError)              → handles error ✅
```

Two Promises Inside This Code

```
// Promise 1 - fetch() waiting for server response  
fetch(url)                                // → Promise<Response>  
  
// Promise 2 - res.json() waiting to parse body  
.then(res => res.json())                 // → Promise<Data>  
  
// That's why we need 2 .then() calls  
// First .then → handles network response  
// Second .then → handles parsed data
```

Same Code Written with async/await

```

// .then() chain and async/await are EXACTLY the same thing
// async/await is just cleaner syntax for Promises

// With .then() – Promise chain
fetch(url)
  .then(res => res.json())
  .then(data => setUsers(data))
  .catch(err => setError(err));

// With async/await – same thing, different look
async function loadUsers() {
  try {
    const res = await fetch(url);           // wait for Promise 1
    const data = await res.json();         // wait for Promise 2
    setUsers(data);
  } catch (err) {
    setError(err.message);
  }
}

```

Both do **identical things**. async/await is just syntactic sugar over Promises.

Promise States in This Code

State 1 → PENDING

fetch() called, waiting for server
 → loading = true, show spinner

State 2a → FULFILLED

Server responded, data parsed
 → setUsers(data), setLoading(false)
 → show list 

State 2b → REJECTED

Network error / res.ok failed
→ goes **to** `.catch`
→ `setError(message), setLoading(false)`
→ **show** error **✗**

Simple Rule to Remember

`fetch()` → always returns a Promise
`.then()` → runs **when** Promise SUCCEEDS (fulfilled)
`.catch()` → runs **when** Promise FAILS (rejected)
`.finally()` → runs ALWAYS (success **or** fail)

Each `.then()` can **return** a **new** Promise
→ *that's how you chain them*
→ **next** `.then` waits **for** it **to** resolve

So yes — `fetch().then().then().catch()` is a **Promise chain** — one of the most common patterns in all of JavaScript. 🚀

3/24/2026, 1:30:32 PM



Callback Queue vs Call Stack

Simple One Line Difference

Call Stack → **where** code RUNS right now (present)
Callback Queue → **where** **async** code WAITS **to** run (future)

Call Stack

LIFO — Last In First Out (like a stack of plates)

```
function C() { console.log("C"); }  
function B() { C(); }  
function A() { B(); }  
  
A();
```

Step 1:	[A]	← A pushed
Step 2:	[A, B]	← B pushed (A called B)
Step 3:	[A, B, C]	← C pushed (B called C)
Step 4:	[A, B]	← C popped (C finished)
Step 5:	[A]	← B popped (B finished)
Step 6:	[]	← A popped (A finished)
	EMPTY	

Rules:

- ✓ Synchronous **only**
- ✓ **One** thing **at** a **time**
- ✓ **Function called** → pushed
- ✓ **Function returns** → popped
- ✗ Cannot wait/pause

Callback Queue

Where async callbacks sit and **wait** until Call Stack is empty.

```
console.log("1");

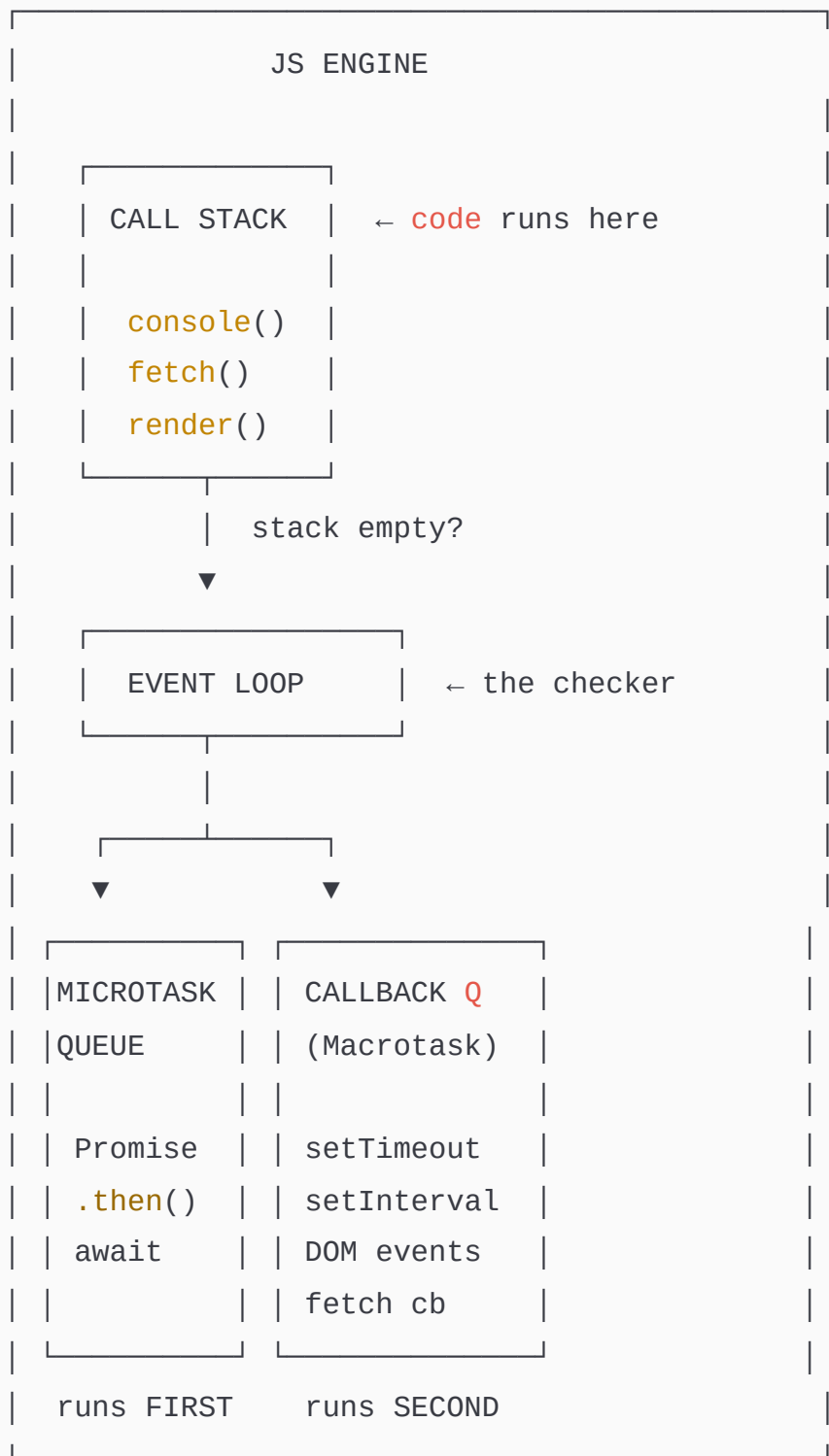
setTimeout(() => {
  console.log("3"); // goes to Callback Queue
}, 1000);

console.log("2");
```

Call Stack runs:	Callback Queue:
console.log("1")	→ (empty)
setTimeout()	→ schedules timer...
console.log("2")	→ (empty)
stack is EMPTY!	→ [setTimeout cb] ← waiting here
	→ Event Loop picks it up
	→ pushes to Call Stack
console.log("3")	→ (empty)

Output: 1 → 2 → 3

Full Picture — Everything Together



Priority Order

- 1st → Call **Stack** (sync code, runs immediately)
- 2nd → Microtask **Queue** (Promises - runs before callbacks)

3rd → Callback Queue (setTimeout, setInterval, events)

```
console.log("1");           // sync           → Call Stack

setTimeout(() => {
  console.log("4");         // macrotask → Callback Queue
}, 0);

Promise.resolve()
  .then(() => console.log("3")); // microtask → Microtask Queue

console.log("2");           // sync           → Call Stack

// Output: 1 → 2 → 3 → 4
```

Why?

```
"1" → runs immediately (sync)
"2" → runs immediately (sync)
Stack is now empty!
Event Loop checks Microtask Queue first → "3" runs
Event Loop checks Callback Queue next → "4" runs
```

Real Example — fetch()

```
console.log("A");

fetch("/api/data")           // starts async request
  .then(res => res.json())
  .then(data => {
    console.log("C", data); // goes to Microtask Queue
  });
```



```
console.log("B");

// Output:  A → B → C

// A, B run synchronously
// Stack empties
// fetch completes → .then callbacks in Microtask Queue
// Event Loop picks them up → C runs
```

Analogy

- Call Stack** = Chef cooking RIGHT NOW
(only one dish at a time)
- Callback Queue** = Orders waiting on the counter
(waiting for chef to be free)
- Microtask Q** = Urgent orders (VIP)
(served before regular orders)
- Event Loop** = Manager watching the chef
"Chef free? Pick next order!"

Key Differences Table

	Call Stack	Callback Queue	Microtask Queue
What goes in	Every function call	setTimeout, setInterval, events	Promise.then, await
When runs	Immediately	After stack empty + microtasks done	After stack empty
Priority	1st (highest)	3rd (lowest)	2nd
Type	Synchronous	Async (Macro)	Async (Micro)
Example	<code>greet()</code>	<code>setTimeout(cb)</code>	<code>fetch().then()</code>

Common Interview Question

```
function main() {  
  console.log("start");  
  
  setTimeout(() => console.log("timeout"), 0);  
  
  Promise.resolve().then(() => console.log("promise"));  
  
  console.log("end");  
}  
  
main();
```

What is the output?

```
start      ← sync, Call Stack  
end        ← sync, Call Stack  
promise    ← Microtask Queue (before setTimeout!)  
timeout    ← Callback Queue (last)
```

If you answer this correctly in an interview

→ interviewer knows you understand JS deeply 🚀

3/24/2026, 1:31:11 PM



- 2.defaultProps in React ?
- 3.props vs defaultProps
- 10.rules to use Hooks ? (UI)
- 13. what is reconciliation ?
- 14. what is react fiber ?
- 15. what is fragments ?
- 16. is react 2 way data binding or ternary or what ?
- 17. what is SPA ?
- 18. why it is 1 way data binding ?

3/25/2026, 6:29:33 AM



Answers

Q2. defaultProps in React?

Default values for props when parent doesn't pass them.

```
function Greeting({ name, age }) {  
  return <h1>Hello {name}, age: {age}</h1>;  
}  
  
// Setting defaultProps  
Greeting.defaultProps = {  
  name: "Guest",  
  age: 18  
};  
  
// Usage  
<Greeting /> // Hello Guest, age: 18
```

```
<Greeting name="Alice" />    // Hello Alice, age: 18
<Greeting name="Bob" age={25} /> // Hello Bob, age: 25
```

Modern way — default params (preferred now):

```
function Greeting({ name = "Guest", age = 18 }) {
  return <h1>Hello {name}, age: {age}</h1>;
}
// Same result, no need for .defaultProps
```

Q3. props vs defaultProps

	props	defaultProps
Passed by	Parent component	Developer (fallback)
When used	Always	Only when prop is NOT passed
Mutable	❌ Read only	❌ Read only
Purpose	Send data down	Prevent undefined errors

```
function Button({ text, color }) {
  return <button style={{ background: color }}>{text}</button>;
}

Button.defaultProps = {
  text: "Click Me",    // fallback if text not passed
  color: "blue"        // fallback if color not passed
};

// Parent passes nothing    → uses defaultProps
<Button />                  // "Click Me", blue

// Parent passes some props → uses passed, fallback for rest
<Button text="Submit" />    // "Submit", blue
```

```
// Parent passes all props → defaultProps ignored  
<Button text="Go" color="red" /> // "Go", red
```

Q10. Rules to use Hooks

React has **2 strict rules** for hooks:

Rule 1 — Only call hooks at TOP LEVEL

```
// ❌ WRONG - hook inside condition  
function App() {  
  if (isLoggedIn) {  
    const [user, setUser] = useState(null); // BREAKS!  
  }  
}  
  
// ❌ WRONG - hook inside loop  
function App() {  
  for (let i = 0; i < 3; i++) {  
    const [val, setVal] = useState(0); // BREAKS!  
  }  
}  
  
// ❌ WRONG - hook inside nested function  
function App() {  
  function handleClick() {  
    const [x, setX] = useState(0); // BREAKS!  
  }  
}  
  
// ✅ CORRECT - hooks at top level always  
function App() {  
  const [user, setUser] = useState(null);
```

```

const [val, setVal] = useState(0);
// then use conditions below
if (isLoggedIn) { /* use user here */ }
}

```

Rule 2 — Only call hooks inside React Functions

```

// ❌ WRONG - hook in plain JS function
function normalFunction() {
  const [x, setX] = useState(0); // BREAKS!
}

// ❌ WRONG - hook outside component
const [x, setX] = useState(0); // BREAKS!

// ✅ CORRECT - inside functional component
function MyComponent() {
  const [x, setX] = useState(0); // ✅
}

// ✅ CORRECT - inside custom hook
function useMyHook() {
  const [x, setX] = useState(0); // ✅
}

```

Why these rules exist?

Hooks are stored as a linked list on the fiber node:

Render 1: useState → useEffect → useState
 Hook1 Hook2 Hook3

Render 2: useState → useEffect → useState
 Hook1 Hook2 Hook3

(must be same **order every** render!)

If inside **condition** → hook might be missing **in** next render
→ linked list **order** breaks → React gets confused → crash

Q13. What is Reconciliation?

The process React uses to **figure out what changed** in the Virtual DOM and update only those parts in the real DOM.

State changes

↓

React creates **NEW** Virtual DOM tree

↓

Compares **NEW** tree vs **OLD** tree ← this **is** Reconciliation

↓

Finds the differences (diffing)

↓

Updates **ONLY** changed parts **in real** DOM

How diffing works:

```
// Rule 1: Different element type → rebuild entire subtree
// Old:           New:
<div>             → <span>
  <Child />       <Child /> // Child unmounts & remounts!
</div>           </span>

// Rule 2: Same type → update only changed attributes
// Old:           New:
<div className="old" /> → <div className="new" />
// React just updates className, doesn't touch the DOM node

// Rule 3: Lists → uses key to match elements
```

```
// Old: [A, B, C]    New: [A, X, C]
// Without key: React re-renders all 3
// With key:      React knows only B changed → X, updates only that
```

```
// Real example
function App() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h1>Title</h1>      {/* never changes → never re-renders */}
      <p>{count}</p>      {/* changes → only this updates in DOM */}
      <button onClick={() => setCount(count + 1)}></button>
    </div>
  );
}
// Without reconciliation: entire page re-renders on every click
// With reconciliation:    only <p> text node updates ✅
```

Q14. What is React Fiber?

Already covered in depth earlier — here's the summary:

React Fiber is the **reimplemented reconciliation engine** (React 16+) that makes rendering **interruptible and prioritized**.

Before **Fiber** (React **15**):

Rendering = synchronous, all **or** nothing

Big render → blocks everything → UI freezes ❌

After **Fiber** (React **16+**):

Rendering = split into small chunks

Can pause → handle urgent work → resume ✅

Fiber gives each component a "fiber node":

```
{
  type: "div",
  child: Fiber,      // first child
  sibling: Fiber,     // next sibling
  return: Fiber,     // parent
  effectTag: UPDATE, // what needs doing
  lanes: priority,   // how urgent
  alternate: Fiber   // old version (for diffing)
}
```

Two phases:

Render Phase → figure out changes (async, pauseable)

Commit Phase → apply to DOM (sync, cannot pause)

Q15. What is Fragments?

Lets you **group multiple elements without adding extra DOM node.**

```
// Problem - React needs single root element
// Wrapping in div adds unnecessary DOM node
return (
  <div>                {/* extra div in DOM ❌ */}
    <h1>Title</h1>
    <p>Text</p>
  </div>
);
```

```
// Solution - Fragment (no extra DOM node)
return (
  <React.Fragment>
    <h1>Title</h1>
    <p>Text</p>
  </React.Fragment>
);
```

```
);

// Shorthand (most used)
return (
  <>
    <h1>Title</h1>
    <p>Text</p>
  </>
);
```

When fragments really matter:

```
// Table example - extra div would break HTML structure!
function TableRows() {
  return (
    <>                                {/* Fragment - no extra <div> inside
<tr> */}
    <td>Name</td>
    <td>Age</td>
  </>
);
}

// If we used div instead:
// <tr><div><td>Name</td><td>Age</td></div></tr> ← invalid HTML ❌
// <tr><td>Name</td><td>Age</td></tr>           ← correct ✅
```

Fragment with key (only time you need full syntax):

```
// Shorthand <> doesn't support key
// Use full Fragment when you need key in a list
{items.map(item => (
  <React.Fragment key={item.id}>
    <dt>{item.term}</dt>
```

```
    <dd>{item.desc}</dd>
  </React.Fragment>
)}}}
```

Q16. Is React 2-way data binding or 1-way or what?

React is 1-way (unidirectional) data binding.

1-way (React):

Parent
↓ props
Child
↓ props
GrandChild

2-way (Angular):

Model $\longleftrightarrow$ View
(auto sync both ways)

Data flows **ONE** direction **only**: Parent → Child

```
// In React, data flows DOWN only via props
function Parent() {
  const [name, setName] = useState("Alice");
  return <Child name={name} />; // data goes DOWN
}

function Child({ name }) {
  // name is READ ONLY here
  // Child CANNOT change parent's name directly
  // Must call a function passed from parent
  return <p>{name}</p>;
}
```

But what about input fields?

```
// This LOOKS like 2-way but it's not
function App() {
  const [value, setValue] = useState("");

  return (
    <input
      value={value}           // state → input   (down)
      onChange={e =>         // input → state   (up via event)
        setValue(e.target.value)}
    />
  );
}

// It's actually TWO one-way flows:
// 1. state → input value   (React controls display)
// 2. user types → event → setState → re-render (explicit update)
// You are manually wiring it – React doesn't do it automatically
// Angular 2-way binding does this automatically with [(ngModel)]
```

Q17. What is SPA (Single Page Application)?

A web app that **loads ONE HTML page** and dynamically updates content without full page reloads.

Traditional (MPA):

User clicks link

↓

Browser requests new page

↓

Server sends full HTML

↓

Full page reload ❌

SPA (React):

User clicks link

↓

JS intercepts click

↓

React renders new component

↓

Only content swaps ✅

↓
White flash, slow

↓
Instant, smooth

index.html (only **ONE** html file ever loaded)
<div id="root"></div> ← **React** controls everything inside here

URL changes: /home → React renders <Home />
 /about → React renders <About />
 /profile → React renders <Profile />

No page reload ever! **Browser** just changes **URL**,
React Router intercepts and swaps components.

Pros vs Cons:

✓ Pros:

- Fast **navigation** (no reload)
- App-like experience
- Less server **load** (only API calls)
- Smooth transitions

✗ Cons:

- Initial load **slower** (downloads entire JS bundle)
- SEO **harder** (content rendered by JS, not HTML)
- Needs JS enabled in browser

Fix cons with:

- Code splitting → load only needed JS
- SSR** (Next.js) → render on server **for** SEO

Q18. Why is React 1-way data binding?

By design choice — for predictability and easier debugging.

The core reason:

2-way binding problem:

View changes Model

Model changes View

View changes Model again...

→ Infinite loops possible

→ Hard to track where data changed

→ Unpredictable in large apps

1-way binding solution:

Data flows ONE direction only

State changes → UI updates

UI cannot change state directly

→ Predictable ✓

→ Easy to debug ✓

→ Easy to trace data ✓

React's explicit update pattern:

// In React you always know:

// 1. WHERE state lives (useState)

// 2. HOW it changes (setState functions)

// 3. WHO changed it (the function that called setState)

```
function App() {  
  const [count, setCount] = useState(0); // single source of truth  
  
  // Only THIS function can change count  
  function increment() { setCount(count + 1); }  
  
  return <button onClick={increment}>{count}</button>;  
}
```

```
// If something is wrong, you check:  
// → what called setCount? → easy to find ✅  
  
// In 2-way binding:  
// count could be changed from anywhere automatically  
// → hard to find who changed it ❌
```

Flux / Redux pattern (same principle):

```
Action → Reducer → Store → View  
      ↑_____↓  
      (new action from View)
```

Data always flows in ONE circle

Never jumps around randomly

→ This is why Redux was invented alongside React

Summary:

1-way data binding = predictable, traceable, debuggable

2-way data binding = convenient but unpredictable at scale

React chose correctness over convenience

That's why it's 1-way 🚀

3/25/2026, 6:30:40 AM



how to make UI faster in react
how do u make async function without using async keyword
how do u call external module called a chatsystem using script in UI
react and javascript
what are the way we can write useEffect , what is that called () => {} in
useEffect
why microfrontent u have used ?
how do u handle lazy loading in react ?
for this integration of chatsystem , how do u use lazy things ?

4/7/2026, 12:37:25 PM



All Questions Answered

1. How to Make UI Faster in React

Level 1 — Prevent Unnecessary Re-renders

```
// React.memo – skip re-render if props didn't change
const Card = React.memo(({ title, desc }) => {
  return <div><h2>{title}</h2><p>{desc}</p></div>;
});

// useCallback – stable function reference
const handleClick = useCallback(() => {
  doSomething(id);
}, [id]); // only recreated when id changes

// useMemo – cache expensive calculation
const filtered = useMemo(() => {
```



```
    return bigList.filter(item => item.active);  
  }, [bigList]);
```

Level 2 — Code Splitting & Lazy Loading

```
// Load component only when needed  
const Dashboard = lazy(() => import('./Dashboard'));  
const Profile    = lazy(() => import('./Profile'));  
  
function App() {  
  return (  
    <Suspense fallback={<Spinner />}>  
      <Routes>  
        <Route path="/dashboard" element={<Dashboard />} />  
        <Route path="/profile"    element={<Profile />} />  
      </Routes>  
    </Suspense>  
  );  
}
```

// Initial bundle: 2MB → 200KB (only loads what's needed)

Level 3 — Virtualize Long Lists

```
import { FixedSizeList } from 'react-window';  
  
// 100,000 items but only ~10 in DOM at any time  
function BigList({ items }) {  
  return (  
    <FixedSizeList  
      height={600}  
      itemCount={items.length}  
      itemSize={50}  
    >  
      {({ index, style }) => (  

```

```

        <div style={style}>{items[index].name}</div>
      )}
    </FixedSizeList>
  );
}

```

Level 4 — Other Tricks

```

// 1. Debounce search input
const debouncedSearch = useCallback(
  debounce((q) => fetchResults(q), 500), []
);

// 2. Lazy load images


// 3. useTransition – mark non-urgent updates
const [isPending, startTransition] = useTransition();
startTransition(() => {
  setSearchResults(filter(query)); // won't block typing
});

// 4. Avoid anonymous functions in JSX
// ❌ creates new function every render
<button onClick={() => handleClick(id)}>Click</button>

// ✅ stable reference
const handle = useCallback(() => handleClick(id), [id]);
<button onClick={handle}>Click</button>

```

2. Async Function WITHOUT async keyword

Three ways:

Way 1 — Promise directly

```
// async/await version
async function fetchData() {
  const res = await fetch('/api/data');
  const data = await res.json();
  return data;
}

// Same thing WITHOUT async keyword – using Promise
function fetchData() {
  return fetch('/api/data')           // fetch returns Promise
    .then(res => res.json())           // .then chains Promise
    .then(data => {
      return data;                    // return value
    });
}

// Both called the same way
fetchData().then(data => console.log(data));
```

Way 2 — new Promise()

```
function delay(ms) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      resolve("Done after " + ms + "ms");
    }, ms);
  });
}

delay(2000).then(msg => console.log(msg)); // "Done after 2000ms"
```

Way 3 — Generator function (advanced)

```
function* fetchGen() {
  const res = yield fetch('/api/data'); // yield pauses like await
  const data = yield res.json();
  return data;
}
// (generators need a runner to work like async/await)
```

Summary:

`async function` = syntactic sugar over Promises
Under the hood, both `do` EXACTLY the same thing

`async/await` → cleaner to read
`.then().catch()` → same result, no `async` keyword needed

3. How to Call External Module (Chat System) Using Script in React/JS

Method 1 — Script tag in index.html (simplest)

```
<!-- public/index.html -->
<head>
  <!-- Load external chat SDK -->
  <script src="https://cdn.chatsystem.com/sdk.js"></script>
</head>
```

```
// Now use it anywhere in React
function App() {
  useEffect(() => {
    // window.ChatSystem loaded from script tag
    window.ChatSystem.init({
      apiKey: "your-key",
      container: "#chat-widget"
```

```

});

return () => window.ChatSystem.destroy(); // cleanup
}, []);

return <div id="chat-widget"></div>;
}

```

Method 2 — Dynamically inject script (better control)

```

function useChatSystem() {
  useEffect(() => {
    // Check if already loaded
    if (window.ChatSystem) return;

    // Create script element
    const script = document.createElement('script');
    script.src = 'https://cdn.chatsystem.com/sdk.js';
    script.async = true;

    // When script loads, initialize
    script.onload = () => {
      window.ChatSystem.init({ apiKey: 'your-key' });
    };

    script.onerror = () => {
      console.error('Failed to load chat system');
    };

    // Inject into DOM
    document.body.appendChild(script);

    // Cleanup
    return () => {
      document.body.removeChild(script);
    };
  });
}

```

```

        window.ChatSystem?.destroy();
    };
}, []); // only once on mount
}

// Use in component
function App() {
    useChatSystem(); // custom hook handles everything
    return <div>My App</div>;
}

```

Method 3 — npm package (if available)

```

import ChatSystem from 'chatsystem-sdk';

function App() {
    useEffect(() => {
        ChatSystem.init({ apiKey: 'your-key' });
        return () => ChatSystem.destroy();
    }, []);
}

```

4. Ways to Write useEffect + What is `() => {}` Called

All ways to write useEffect:

```

// Way 1 – No dependency array
// Runs after EVERY render
useEffect(() => {
    console.log("runs every render");
});

// Way 2 – Empty dependency array []
// Runs ONCE on mount only (componentDidMount)

```

```

useEffect(() => {
  fetchData();
}, []);

// Way 3 – With dependencies
// Runs when dependency changes
useEffect(() => {
  fetchUser(userId);
}, [userId]); // re-runs when userId changes

// Way 4 – With cleanup (return function)
// Cleanup runs before next effect OR on unmount
useEffect(() => {
  const timer = setInterval(() => tick(), 1000);
  return () => clearInterval(timer); // cleanup
}, []);

// Way 5 – Multiple effects (separate concerns)
useEffect(() => { fetchUser(); }, [userId]);
useEffect(() => { fetchPosts(); }, [userId]);
useEffect(() => { trackPage(); }, [page]);

```

What is `() => {}` called?

```

useEffect(() => {
  //      ^^^^^^^^^
  //      This is an ARROW FUNCTION
  //      Also called: Anonymous Arrow Function
  //      Because it has no name

  console.log("effect");
}, []);

```

```
// Different function types:

// 1. Arrow Function (anonymous)
() => { }
() => value           // implicit return

// 2. Named Arrow Function
const myFn = () => { }

// 3. Regular Anonymous Function
function() { }

// 4. Named Function
function myFunc() { }

// In useEffect we pass an arrow function as ARGUMENT
// useEffect takes a function as its first parameter
useEffect(FUNCTION, DEPS);

// So () => {} is:
// → An arrow function
// → Passed as argument to useEffect
// → Called a "callback function" in this context
```

5. Why Micro-Frontend?

Micro-frontend = applying microservices idea to the frontend.

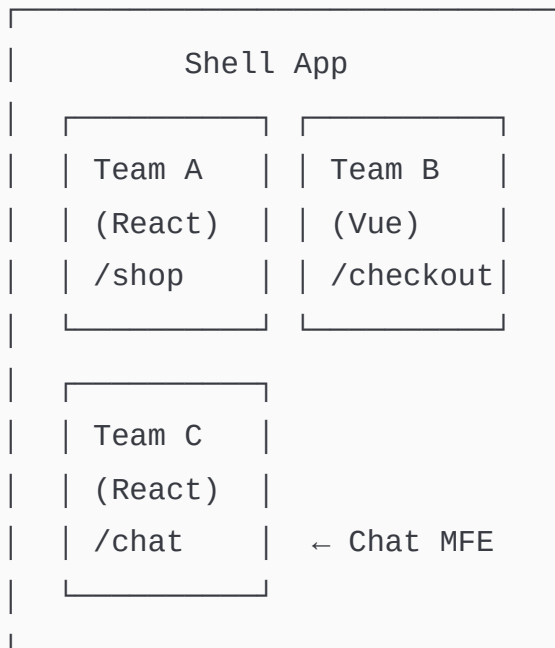
Monolith Frontend (problem):

- One giant React app
- Team A and Team B both edit same codebase
 - merge conflicts
 - one bug breaks everything
 - deploy entire app for small change

→ slow, risky ❌

Micro-Frontend (solution):

App split **into** independent pieces



Why we use it:

- ✅ Independent deployment – deploy chat **without** touching shop
- ✅ Independent teams – **no merge** conflicts
- ✅ Technology freedom – **each** MFE can use different framework
- ✅ Isolated failures – chat crashes, shop still works
- ✅ Smaller bundles – load **only** what page needs
- ✅ Easy **to** scale teams

When to use:

- ✅ **Large** enterprise apps
- ✅ Multiple teams
- ✅ Different **release** cycles
- ❌ Small apps – overkill

6. How to Handle Lazy Loading in React

```

// 1. Component lazy loading
import { lazy, Suspense } from 'react';

const HeavyComponent = lazy(() => import('./HeavyComponent'));

function App() {
  return (
    <Suspense fallback={<div>Loading...</div>}>
      <HeavyComponent />
    </Suspense>
  );
}

// 2. Route-based (most common)
const Home = lazy(() => import('./pages/Home'));
const Dashboard = lazy(() => import('./pages/Dashboard'));
const ChatPage = lazy(() => import('./pages/ChatPage'));

function App() {
  return (
    <Suspense fallback={<PageSpinner />}>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/chat" element={<ChatPage />} />
      </Routes>
    </Suspense>
  );
}

// 3. On user interaction
const ChatWidget = lazy(() => import('./ChatWidget'));

function App() {
  const [showChat, setShowChat] = useState(false);

```

```

return (
  <>
    <button onClick={() => setShowChat(true)}>Open Chat</button>

    {showChat && ( // ChatWidget downloaded only on
click
    <Suspense fallback={<div>Loading chat...</div>}>
      <ChatWidget />
    </Suspense>
    )}
  </>
);
}

```

Custom lazy loading with loading states:

```

function useLazyLoad(importFn) {
  const [Component, setComponent] = useState(null);
  const [loading, setLoading] = useState(false);

  const load = useCallback(async () => {
    setLoading(true);
    const module = await importFn();
    setComponent(() => module.default);
    setLoading(false);
  }, []);

  return { Component, loading, load };
}

// Usage
function App() {
  const { Component, loading, load } = useLazyLoad(
    () => import('./ChatWidget')
  );
}

```

```

);

return (
  <>
    <button onClick={load}>Open Chat</button>
    {loading && <Spinner />}
    {Component && <Component />}
  </>
);
}

```

7. How to Use Lazy Loading for Chat System Integration

```

// Combining external script + lazy loading for chat

// Step 1 – Lazy load the ChatWidget component
const ChatWidget = lazy(() => import('./ChatWidget'));

// Step 2 – Inside ChatWidget, load external script lazily
// ChatWidget.jsx
function ChatWidget() {
  const [ready, setReady] = useState(false);

  useEffect(() => {
    // Only load SDK when this component mounts
    // (not on app start)
    if (window.ChatSystem) {
      setReady(true);
      return;
    }

    const script = document.createElement('script');
    script.src    = 'https://cdn.chatsystem.com/sdk.js';
    script.async  = true;

```

```

script.onload = () => {
  window.ChatSystem.init({ apiKey: 'key' });
  setReady(true);
};
document.body.appendChild(script);

return () => window.ChatSystem?.destroy();
}, []);

if (!ready) return <div>Loading chat...</div>;
return <div id="chat-container">Chat Ready 

```

// Step 3 – Use in App with full lazy chain

```

function App() {
  const [openChat, setOpenChat] = useState(false);

  return (
    <>
      <button onClick={() => setOpenChat(true)}>
         Chat
      </button>

      {openChat && (
        <Suspense fallback=<div>Loading...</div>>
          <ChatWidget />
          {/*
            Nothing loads until user clicks button:
            1. User clicks → React loads ChatWidget.js chunk
            2. ChatWidget mounts → loads chat SDK script
            3. SDK ready → shows chat UI
          */}
        </Suspense>
      )}
    </>
  )
}

```

```
);  
}
```

Full lazy chain:

App loads (fast, **no** chat code)
↓
User clicks "Chat" button
↓
React.lazy downloads ChatWidget.chunk.js
↓
ChatWidget mounts
↓
useEffect runs → injects <script> tag
↓
Browser downloads chat SDK
↓
SDK initializes → chat appears
↓
User closes chat → cleanup runs
↓
SDK destroyed (**no** memory leak)

Result:

- ✓ Zero cost **on initial** page load
- ✓ Chat SDK **only** loads **when** actually needed
- ✓ **No** memory leaks (cleanup)
- ✓ Graceful loading states **at each** step

4/7/2026, 12:38:19 PM



what is sync and async functions in javascript
how do you write in css align horizontal and vertical

4/8/2026, 3:27:37 PM



1. Sync vs Async in JavaScript

Synchronous (Sync)

One thing at a time. Next line waits for current line to finish.

```
console.log("1");  
console.log("2");  
console.log("3");
```

```
// Output: 1 → 2 → 3  
// Each line WAITS for previous to finish
```

```
// Real problem with sync  
function fetchData() {  
  // imagine this takes 5 seconds  
  const data = readFileSync('bigfile.txt'); // BLOCKS everything  
  console.log(data);  
}
```

```
fetchData();  
console.log("I have to wait 5 sec!"); // stuck waiting ❌
```

```
// During these 5 seconds:  
// ❌ UI is frozen
```

```
// ❌ User can't click anything
// ❌ Nothing else runs
```

Asynchronous (Async)

Start a task, don't wait, continue. Come back when done.

```
console.log("1");

setTimeout(() => {
  console.log("3"); // runs later
}, 2000);

console.log("2"); // doesn't wait for setTimeout

// Output: 1 → 2 → (2 seconds later) → 3
// "2" doesn't wait for setTimeout to finish
```

```
// Same file read, async way
readFile('bigfile.txt', (data) => {
  console.log(data); // runs when ready
});
console.log("I run immediately!"); // doesn't wait ✅

// ✅ UI stays responsive
// ✅ Other code runs
// ✅ Callback called when file is ready
```

Visual Difference

SYNCHRONOUS:

Task1 done

Task2  done
Task3  done

Time: 12 units (sequential, blocking)

ASYNCHRONOUS:

Task1  done
Task2  done (parallel)
Task3  done (parallel)

Time: 4 units (concurrent, non-blocking)

4 Ways to Write Async in JavaScript

Way 1 — Callback (oldest way)

```
function fetchUser(id, callback) {  
  setTimeout(() => {  
    const user = { id, name: "Alice" }; // simulated API  
    callback(user); // call when done  
  }, 1000);  
}  
  
fetchUser(1, (user) => {  
  console.log(user); // { id: 1, name: "Alice" }  
});  
  
console.log("This runs before callback!");
```

Callback Hell problem:

```
// ❌ Deeply nested, hard to read
fetchUser(1, (user) => {
  fetchPosts(user.id, (posts) => {
    fetchComments(posts[0].id, (comments) => {
      fetchLikes(comments[0].id, (likes) => {
        console.log(likes); // pyramid of doom!
      });
    });
  });
});
```

Way 2 — Promise (ES6)

```
function fetchUser(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id) resolve({ id, name: "Alice" }); // success
      else reject("No ID provided"); // failure
    }, 1000);
  });
}

// Consuming Promise
fetchUser(1)
  .then(user => console.log(user)) // success
  .catch(err => console.log(err)) // failure
  .finally(() => console.log("Done")); // always

// Chaining (no callback hell)
fetchUser(1)
  .then(user => fetchPosts(user.id))
  .then(posts => fetchComments(posts[0].id))
  .then(comments => console.log(comments))
  .catch(err => console.log(err)); // one catch for all ✅
```

Way 3 — async/await (ES8, most used today)

```
// async keyword makes function return a Promise
async function loadUserData() {
  try {
    // await pauses ONLY this function, not the whole app
    const user      = await fetchUser(1);
    const posts     = await fetchPosts(user.id);
    const comments = await fetchComments(posts[0].id);

    console.log(comments); // clean, readable ✅
  } catch (err) {
    console.log("Error:", err);
  }
}

loadUserData();
console.log("This still runs immediately!"); // doesn't wait
```

Parallel with async/await:

```
async function loadAll() {
  // ❌ Sequential - total 3 seconds
  const users  = await fetchUsers(); // 1 sec
  const posts  = await fetchPosts(); // 1 sec
  const photos = await fetchPhotos(); // 1 sec

  // ✅ Parallel - total 1 second
  const [users, posts, photos] = await Promise.all([
    fetchUsers(), // all 3 start at same time
    fetchPosts(),
    fetchPhotos()
  ]);
}
```

Way 4 — Promise without async keyword

```
// Same as async/await but using .then()
// (already covered in previous answer)
function loadData() {
  return fetch('/api/users')
    .then(res => res.json())
    .then(data => data);
}
```

Sync vs Async Comparison Table

	Sync	Async
Execution	One by one	Concurrent
Blocking	✅ Blocks	❌ Non-blocking
UI during task	Freezes	Stays responsive
Use case	Simple calculations	API calls, timers, file read
Examples	Math, array methods	fetch, setTimeout, fs.readFile

In React — Async is Everywhere

```
function App() {
  const [data, setData] = useState(null);

  // useEffect callback cannot be async directly
  useEffect(() => {

    // ✅ define async inside, call immediately
    async function fetchData() {
      const res = await fetch('/api/data');
      const json = await res.json();
      setData(json);
    }
  });
}
```

```

    }

    fetchData(); // call it

  }, []);

  return <div>{data?.name}</div>;
}

```

Simple Rule to Remember

Is the task instant? → Sync (math, string ops)
 Does the task **take** time? → Async (network, timer, disk)

Sync = cook one dish, serve, **then** cook **next**
 Async = start all dishes simultaneously, serve **when each is** ready

2. CSS — Align Horizontal and Vertical (Center)

Method 1 — Flexbox (most used)

```

.container {
  display: flex;
  justify-content: center; /* horizontal center */
  align-items: center;     /* vertical center */

  height: 100vh; /* needs height to center vertically */
}

```

```

<div class="container">
  <div class="box">I am centered!</div>

```

```
</div>
```

justify-content → main axis (horizontal by default)
align-items → cross axis (vertical by default)

All alignment options:

```
/* Horizontal (justify-content) */
justify-content: flex-start; /* left */
justify-content: center; /* center */
justify-content: flex-end; /* right */
justify-content: space-between; /* space between items */
justify-content: space-around; /* space around items */

/* Vertical (align-items) */
align-items: flex-start; /* top */
align-items: center; /* middle */
align-items: flex-end; /* bottom */
align-items: stretch; /* fill height (default) */
```

Method 2 — Grid (cleanest one-liner)

```
.container {
  display: grid;
  place-items: center; /* centers BOTH horizontal and vertical */

  height: 100vh;
}
```

`place-items: center` = shorthand for `align-items + justify-items` together.

Method 3 — Position + Transform

```
.container {  
  position: relative;  
  height: 100vh;  
}  
  
.box {  
  position: absolute;  
  top: 50%;           /* move down 50% */  
  left: 50%;          /* move right 50% */  
  transform: translate(-50%, -50%); /* pull back by own size */  
}
```

Without transform: top-left corner at center

With transform: true center of element at center 

Method 4 — Margin auto (horizontal only)

```
/* Only works for horizontal */  
.box {  
  width: 200px;  
  margin: 0 auto; /* auto left and right margin = centered */  
}
```

Method 5 — Text align (inline content only)

```
.container {  
  text-align: center; /* centers text/inline elements horizontally */  
  
  /* For vertical text centering in fixed height box */  
  height: 100px;
```

```
line-height: 100px; /* same as height = vertically centered */
}
```

Quick Reference — Which to Use When

Centering a `div` in page?

→ Flexbox or Grid ✓

Centering text in a `button`?

→ Flexbox or `text-align` ✓

Centering a popup/modal?

→ `Position` + `transform` ✓

Centering a block element horizontally?

→ `margin: 0 auto` ✓

Centering multiple items in a row?

→ Flexbox with `justify-content` ✓

Centering in a grid layout?

→ Grid `place-items: center` ✓

Most Common Pattern in React

```
// Center anything on the full page
const styles = {
  container: {
    display: 'flex',
    justifyContent: 'center', /* note: camelCase in React */
    alignItems: 'center',
    height: '100vh'
  }
};
```



```
function CenteredPage() {  
  return (  
    <div style={styles.container}>  
      <div>I am perfectly centered!</div>  
    </div>  
  );  
}
```

Remember:

justify-content = horizontal (main axis)

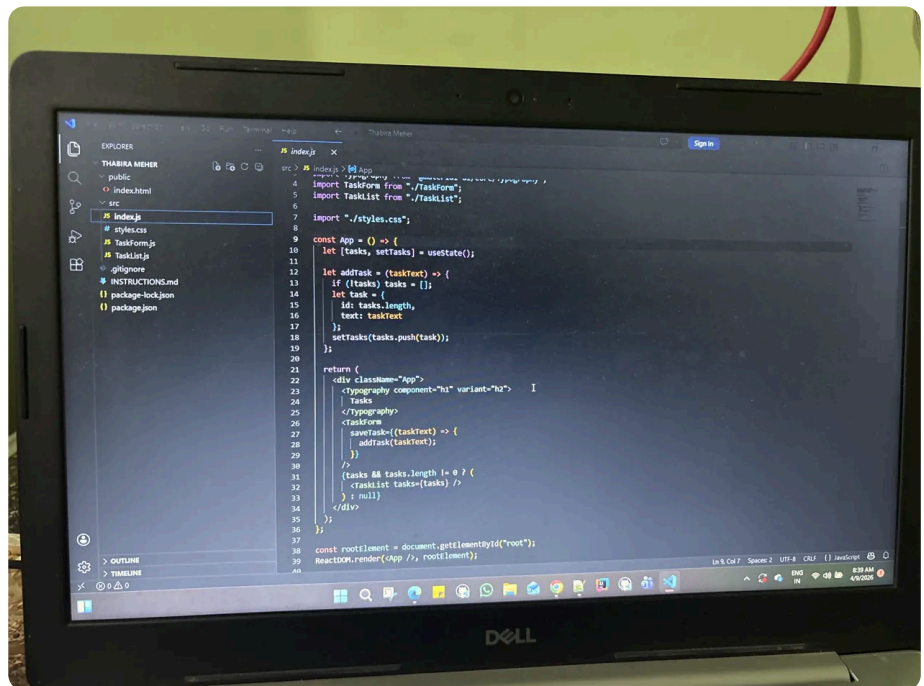
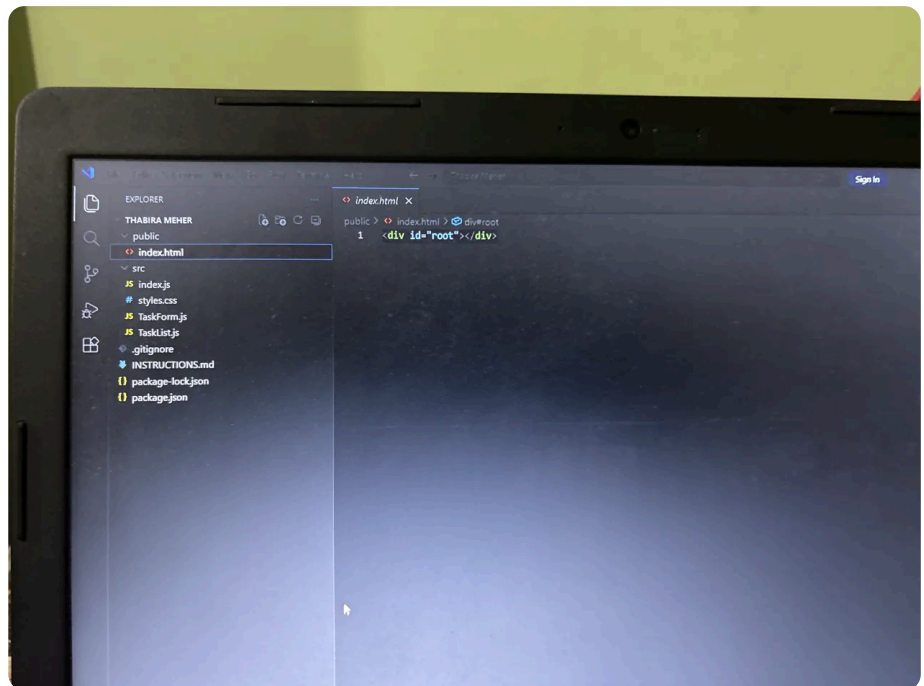
align-items = vertical (cross axis)

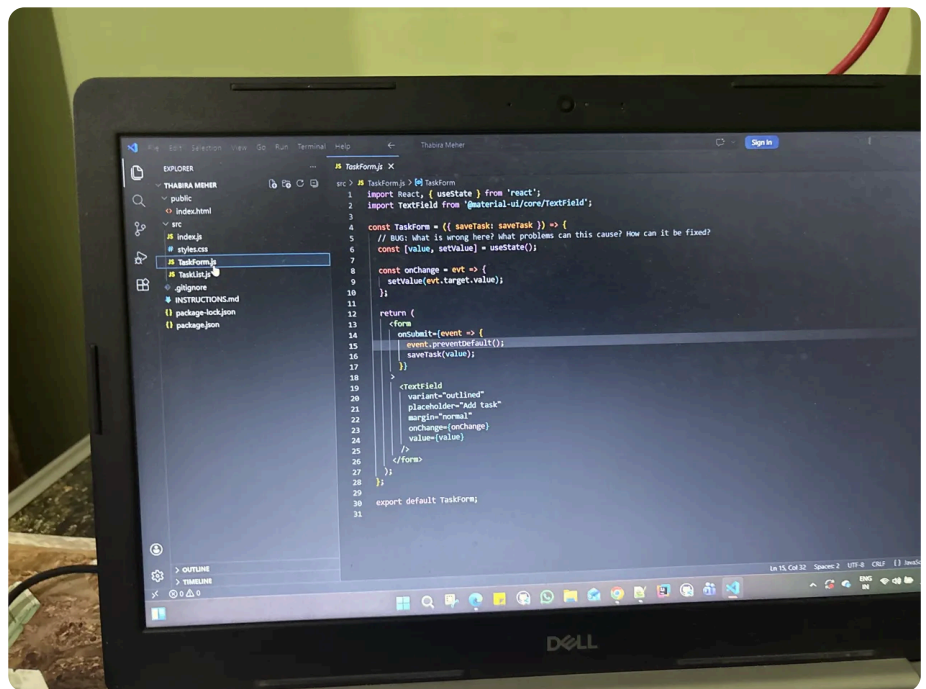
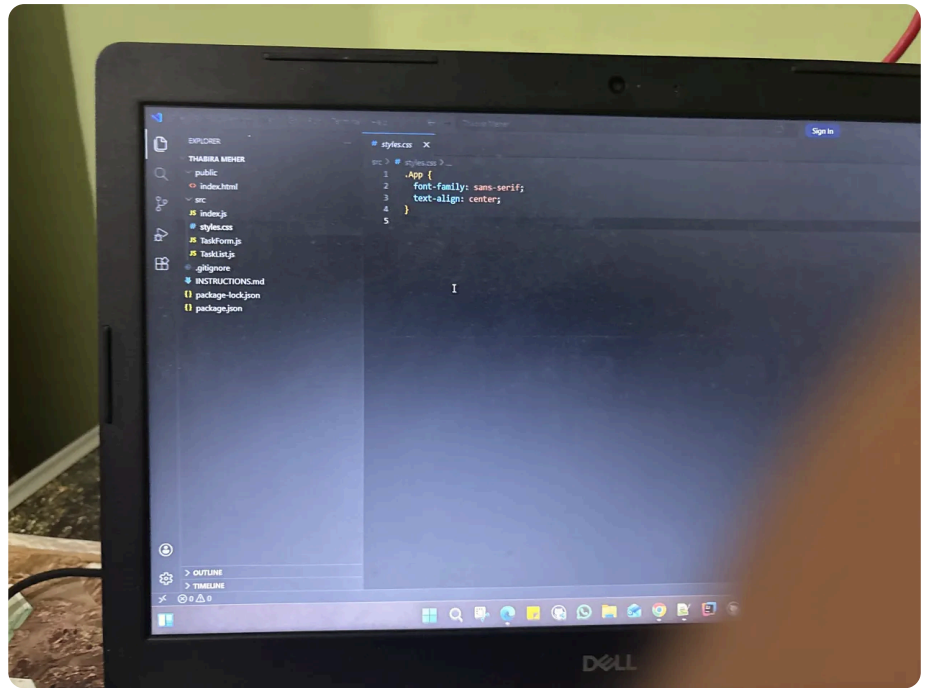
These flip when you use flex-direction: column!

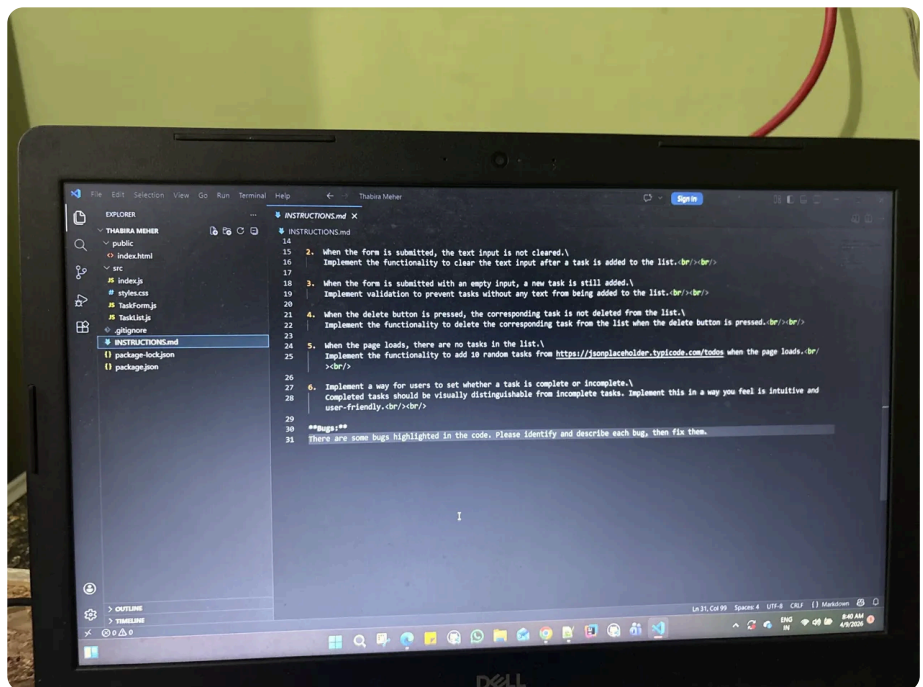
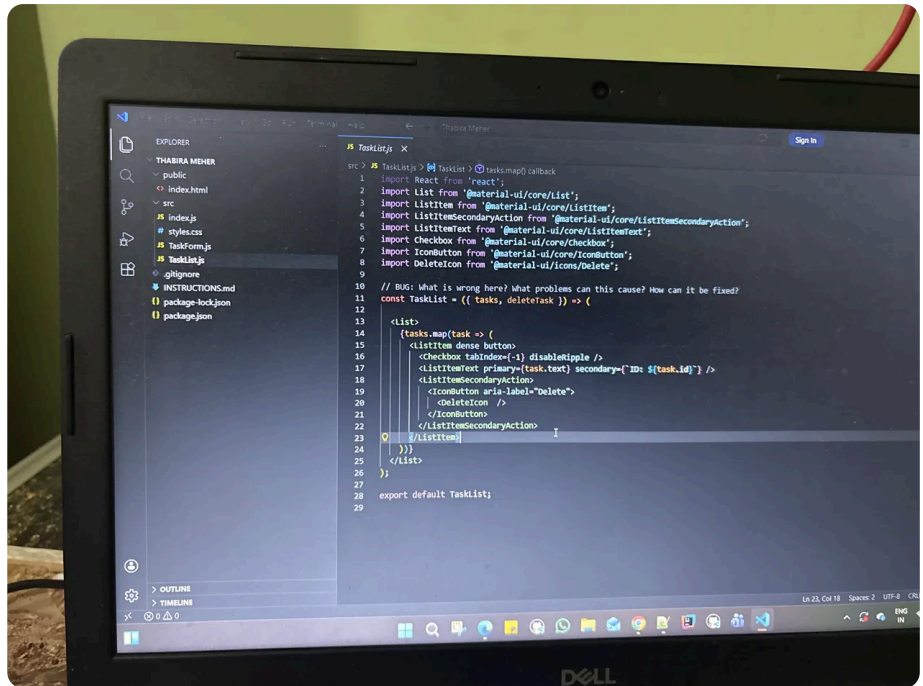
column → justify-content becomes vertical

→ align-items becomes horizontal

4/8/2026, 3:28:22 PM







How do we fix this above project

4/9/2026, 8:40:43 AM



I can see the full project. Let me identify all bugs and implement all the required features.

Bugs Found

Bug 1 — index.js: `setTasks(tasks.push(task))`

```
// ❌ WRONG - tasks.push() mutates array and returns LENGTH (number)
// setTasks gets a number, not an array!
setTasks(tasks.push(task));

// ✅ FIXED - spread into new array (immutable update)
setTasks([...tasks, task]);
```

Bug 2 — index.js: `useState()` no initial value

```
// ❌ WRONG - undefined causes crash on tasks.length check
let [tasks, setTasks] = useState();

// ✅ FIXED - initialize as empty array
const [tasks, setTasks] = useState([]);
```

Bug 3 — TaskForm.js: no submit button

```
// ❌ form has no submit button – can never submit!
// ✅ Add a submit button inside the form
```

Bug 4 — TaskList.js: no key prop on list items

```
// ❌ WRONG - no key on mapped elements
{tasks.map(task => (
  <ListItem> // missing key!

// ✅ FIXED
{tasks.map(task => (
  <ListItem key={task.id}>
```

Bug 5 — TaskList.js: deleteTask not wired up

```
// ❌ delete button does nothing - no onClick
<IconButton aria-label="Delete">

// ✅ FIXED
<IconButton aria-label="Delete" onClick={() => deleteTask(task.id)}>
```

Full Fixed Code — All Files

index.js (App)

```
import React, { useState, useEffect } from 'react';
import Typography from '@material-ui/core/Typography';
import TaskForm from './TaskForm';
import TaskList from './TaskList';
import './styles.css';

const App = () => {
  // Bug fix 2: initialize with []
  const [tasks, setTasks] = useState([]);

  // Feature 5: load 10 random tasks on page load
  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/todos?_limit=10')
      .then(res => res.json())
      .then(data => {
        const loaded = data.map(todo => ({
          id: todo.id,
          text: todo.title,
          completed: todo.completed // Feature 6
        }));
        setTasks(loaded);
      })
  })
}
```

```

        .catch(err => console.error('Failed to load tasks', err));
    }, []); // runs once on mount

const addTask = (taskText) => {
    // Feature 3: validation – no empty tasks
    if (!taskText || taskText.trim() === '') return;

    if (!tasks) tasks = [];

    const task = {
        id: tasks.length + Date.now(), // unique id
        text: taskText,
        completed: false // Feature 6
    };

    // Bug fix 1: spread into new array
    setTasks([...tasks, task]);
};

// Feature 4: delete task by id
const deleteTask = (id) => {
    setTasks(tasks.filter(task => task.id !== id));
};

// Feature 6: toggle complete/incomplete
const toggleTask = (id) => {
    setTasks(tasks.map(task =>
        task.id === id
        ? { ...task, completed: !task.completed }
        : task
    ));
};

return (
    <div className="App">

```

```

    <Typography component="h1" variant="h2">
      Tasks
    </Typography>

    <TaskForm
      saveTask={({taskText}) => {
        addTask(taskText);
      }}
    />

    {tasks && tasks.length !== 0 ? (
      <TaskList
        tasks={tasks}
        deleteTask={deleteTask}
        toggleTask={toggleTask}
      />
    ) : null}
  </div>
);
};

const rootElement = document.getElementById('root');
ReactDOM.render(<App />, rootElement);

```

TaskForm.js

```

import React, { useState } from 'react';
import TextField from '@material-ui/core/TextField';
import Button from '@material-ui/core/Button';

const TaskForm = ({ saveTask }) => {
  const [value, setValue] = useState(''); // Bug fix 2: init with ''

  const onChange = evt => {
    setValue(evt.target.value);
  }

```



```

};

return (
  <form
    onSubmit={{(event) => {
      event.preventDefault();

      // Feature 3: validation
      if (!value || value.trim() === '') return;

      saveTask(value);

      // Feature 2: clear input after submit
      setValue('');
    }}}
  >
    <TextField
      variant="outlined"
      placeholder="Add task"
      margin="normal"
      onChange={onChange}
      value={value}
    />
    {/* Bug fix 3: add submit button */}
    <Button
      type="submit"
      variant="contained"
      color="primary"
      style={{ marginTop: '16px', marginLeft: '8px' }}
    >
      Add Task
    </Button>
  </form>
);
};

```

```
export default TaskForm;
```

TaskList.js

```
import React from 'react';
import List from '@material-ui/core/List';
import ListItem from '@material-ui/core/ListItem';
import ListItemSecondaryAction from '@material-
ui/core/ListItemSecondaryAction';
import ListItemText from '@material-ui/core/ListItemText';
import Checkbox from '@material-ui/core/Checkbox';
import IconButton from '@material-ui/core/IconButton';
import DeleteIcon from '@material-ui/icons/Delete';

const TaskList = ({ tasks, deleteTask, toggleTask }) => (
  <List>
    {tasks.map(task => (
      // Bug fix 4: key prop added
      <ListItem key={task.id} dense button>

        {/* Feature 6: toggle complete */}
        <Checkbox
          tabIndex={-1}
          disableRipple
          checked={task.completed}
          onChange={() => toggleTask(task.id)}
        />

        {/* Feature 6: strike through completed tasks */}
        <ListItemText
          primary={
            <span style={{
              textDecoration: task.completed ? 'line-through' :
'none',
```

```

        color: task.completed ? 'gray' : 'black'
      }}>
      {task.text}
    </span>
  }
  secondary={`ID: ${task.id}`}
/>

<ListItemSecondaryAction>
  {/* Bug fix 5: wire up deleteTask */}
  <IconButton
    aria-label="Delete"
    onClick={() => deleteTask(task.id)}
  >
    <DeleteIcon />
  </IconButton>
</ListItemSecondaryAction>

</ListItem>
)}}
</List>
);

export default TaskList;

```

styles.css

```

.App {
  font-family: sans-serif;
  text-align: center;
  max-width: 600px;
  margin: 0 auto;
  padding: 20px;
}

```

```
/* Feature 6: completed task styling */  
.task-complete {  
  text-decoration: line-through;  
  color: gray;  
}
```

Summary of ALL fixes

- Bug 1  `tasks.push()` → `[...tasks, task]`
(push mutates + returns length, not array)
- Bug 2  `useState()` → `useState([])`
(undefined breaks `.length` and `.map`)
- Bug 3  Added `<Button type="submit">` to `TaskForm`
(form had no way to submit!)
- Bug 4  Added `key={task.id}` to `<ListItem>`
(React needs key for list reconciliation)
- Bug 5  Added `onClick={() => deleteTask(task.id)}`
(delete button was doing nothing)
- Feature 2  `setValue('')` after form submit
(input now clears after adding task)
- Feature 3  `value.trim() === ''` check
(empty tasks not added)
- Feature 4  `deleteTask` filters by id
(correct task removed)
- Feature 5  `useEffect` fetches from jsonplaceholder
(10 tasks load on page mount)

Feature 6  completed field + toggleTask + strikethrough
(visually shows complete vs incomplete)

4/9/2026, 8:41:18 AM



HTML basic questions and answers

what is redux, how we use redux and explain in details ? give small example

what is react fiber ? explain in details ? give small example

write code for fetch api, todo app, read data from an api and populate in a table using pagination with limit and offset and size

4/15/2026, 6:33:54 AM



HTML Basic Questions & Answers

Q. What is HTML?

HTML (HyperText Markup Language) is the skeleton of a webpage. It defines the structure and content.

```
<!DOCTYPE html>
<html>
  <head>
    <title>My Page</title>
  </head>
  <body>
    <h1>Hello World</h1>
    <p>This is a paragraph</p>
  </body>
</html>
```

Q. Block vs Inline elements?

```
<!-- Block - takes full width, new line -->
<div> <p> <h1> <ul> <table> <form>
```

```
<!-- Inline - only takes content width, stays in line -->
<span> <a> <img> <strong> <em> <input>
```

Q. What is semantic HTML?

```
<!-- Non-semantic (bad) -->
<div id="header"> <div class="nav"> <div id="footer">

<!-- Semantic (good) - describes meaning -->
<header> <nav> <main> <article> <section> <footer> <aside>
```

Q. Forms in HTML?

```
<form action="/submit" method="POST">
  <input type="text"      placeholder="Name" />
  <input type="email"     placeholder="Email" />
  <input type="password"  placeholder="Password" />
  <input type="checkbox" /> Remember me
  <input type="radio"     name="gender" value="male" /> Male
  <select>
    <option>Option 1</option>
    <option>Option 2</option>
  </select>
  <textarea rows="4">Write here...</textarea>
  <button type="submit">Submit</button>
</form>
```

Q. What are data attributes?

```
<!-- Store custom data on elements -->
<div data-user-id="123" data-role="admin">User</div>

<script>
```

```
const div = document.querySelector('div');
console.log(div.dataset.userId); // "123"
console.log(div.dataset.role);   // "admin"
</script>
```

Q. iframe, meta, link tags?

```
<head>
  <!-- Meta tags - data about the page -->
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
  <meta name="description" content="My website description" />

  <!-- Link - external resources -->
  <link rel="stylesheet" href="styles.css" />
  <link rel="icon"      href="favicon.ico" />
</head>

<!-- iframe - embed another page -->
<iframe src="https://google.com" width="600" height="400"></iframe>
```

Q. What is the difference between id and class?

```
<!-- id - unique, only ONE element per page -->
<div id="header">Only one header</div>

<!-- class - reusable, MULTIPLE elements -->
<div class="card">Card 1</div>
<div class="card">Card 2</div>
<div class="card">Card 3</div>
```

Q. HTML5 new features?


```
<!-- New semantic tags -->
<header> <footer> <nav> <article> <section> <aside>

<!-- New input types -->
<input type="date" />
<input type="color" />
<input type="range" />
<input type="search" />

<!-- Audio/Video -->
<video src="video.mp4" controls></video>
<audio src="audio.mp3" controls></audio>

<!-- Canvas -->
<canvas id="myCanvas"></canvas>

<!-- Local Storage API -->
localStorage.setItem('key', 'value');
```

Redux — Full Deep Dive

What is Redux?

Redux is a **predictable state management library**. All your app's state lives in **one central store**. State can only change through **actions** processed by **pure reducer functions**.

Problem Redux solves:

Without Redux:

```
ComponentA
  ↓ props
ComponentB
  ↓ props
```

ComponentC
↓ props
ComponentD ← needs the data
(prop drilling through 4 levels!)

With Redux:

Store (central state)
↑↓
Any component directly connects ✓
No prop drilling!

3 Core Principles

1. Single Source of Truth
→ All state in ONE store object
2. State is Read-Only
→ Can only change by dispatching actions
→ Never mutate state directly
3. Changes via Pure Functions (Reducers)
→ Same input always gives same output
→ No side effects

5 Core Concepts

1. STORE → holds all state (single JS object)
2. ACTION → plain object describing WHAT happened
3. REDUCER → function deciding HOW state changes
4. DISPATCH → function to SEND an action to store
5. SELECTOR → function to READ state from store

How Redux Works — Flow

User clicks button



dispatch(action) → { type: 'INCREMENT', payload: 1 }



Reducer receives → (currentState, action) => newState



Store updates → new state stored



Component re-renders → useSelector gets new state



UI updates 

Simple Counter Example — Redux Toolkit (Modern Way)

```
npm install @reduxjs/toolkit react-redux
```

Step 1 — Create a Slice (action + reducer together)

```
// store/counterSlice.js
import { createSlice } from '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 }, // initial state

  reducers: {
    // Each function here = one action
    increment: (state) => {
      state.value += 1; // RTK uses Immer - looks like mutation but
      // isn't
    },
    decrement: (state) => {
      state.value -= 1;
    }
  }
});
```

```

    },
    incrementByAmount: (state, action) => {
      state.value += action.payload; // payload = data sent with
action
    },
    reset: (state) => {
      state.value = 0;
    }
  }
});

// Export actions (to dispatch)
export const { increment, decrement, incrementByAmount, reset } =
  counterSlice.actions;

// Export reducer (to add to store)
export default counterSlice.reducer;

```

Step 2 — Create Store

```

// store/store.js
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from './counterSlice';

const store = configureStore({
  reducer: {
    counter: counterReducer,
    // add more reducers here
    // user: userReducer,
    // posts: postsReducer,
  }
});

export default store;

```

Step 3 — Provide Store to App

```
// index.js
import { Provider } from 'react-redux';
import store from './store/store';

ReactDOM.render(
  <Provider store={store}> { /* wrap entire app */}
    <App />
  </Provider>,
  document.getElementById('root')
);
```

Step 4 — Use in Components

```
// Counter.jsx
import { useSelector, useDispatch } from 'react-redux';
import { increment, decrement, incrementByAmount, reset }
  from './store/counterSlice';

function Counter() {
  // useSelector → READ from store
  const count = useSelector(state => state.counter.value);

  // useDispatch → SEND actions to store
  const dispatch = useDispatch();

  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={() => dispatch(increment())}>+</button>
      <button onClick={() => dispatch(decrement())}>-</button>
      <button onClick={() =>
dispatch(incrementByAmount(5))}>+5</button>
      <button onClick={() => dispatch(reset())}>Reset</button>
    </div>
  );
}
```

```
    </div>
  );
}
```

Async Redux — API Calls with createAsyncThunk

```
// store/userSlice.js
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';

// createAsyncThunk handles pending/fulfilled/rejected automatically
export const fetchUsers = createAsyncThunk(
  'users/fetchAll', // action name
  async () => {
    const res = await
fetch('https://jsonplaceholder.typicode.com/users');
    const data = await res.json();
    return data; // this becomes action.payload
  }
);

const userSlice = createSlice({
  name: 'users',
  initialState: {
    list: [],
    loading: false,
    error: null
  },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchUsers.pending, (state) => {
        state.loading = true;
        state.error = null;
      })
      .addCase(fetchUsers.fulfilled, (state, action) => {
```

```

        state.loading = false;
        state.list     = action.payload; // data from API
    })
    .addCase(fetchUsers.rejected, (state, action) => {
        state.loading = false;
        state.error    = action.error.message;
    });
}
});

export default userSlice.reducer;

// Component
function UserList() {
    const { list, loading, error } = useSelector(state =>
state.users);
    const dispatch = useDispatch();

    useEffect(() => {
        dispatch(fetchUsers()); // trigger async thunk
    }, []);

    if (loading) return <p>Loading...</p>;
    if (error)   return <p>Error: {error}</p>;

    return (
        <ul>
            {list.map(u => <li key={u.id}>{u.name}</li>)}
        </ul>
    );
}

```

Redux vs Context API

Context API:

- ✓ Built-in, no library
- ✓ Simple sharing (theme, language, auth)
- ✗ No DevTools
- ✗ All consumers re-render on any change
- ✗ No middleware for async

Redux:

- ✓ DevTools (time-travel debugging)
- ✓ Selective re-renders (performance)
- ✓ Middleware (thunk, saga for async)
- ✓ Predictable, traceable state changes
- ✗ More setup/boilerplate
- ✗ External library

Use Context → small apps, simple state

Use Redux → large apps, complex state, multiple teams

React Fiber — Full Deep Dive

What is React Fiber?

React Fiber is the **complete rewrite of React's reconciliation engine** introduced in React 16. It makes rendering **interruptible, prioritized, and incremental**.

Why Was Fiber Needed?

Before Fiber (React 15) – Stack Reconciler:

- User clicks button → React starts rendering
- Renders 500 components synchronously
- Takes 300ms
- During these 300ms:

- ❌ User typing `is` ignored
- ❌ Animations freeze
- ❌ Clicks don't *register*
- ❌ UI `is` completely unresponsive

Like a chef who refuses to stop cooking until ALL 10 courses are done, even if the restaurant is on fire.

What Fiber Changed

After Fiber (React 16+):

React starts rendering

- Does small chunk of work (5ms)
- PAUSES → checks: "Anything urgent?"
- User `is` typing! (high priority)
- Handles typing first (feels instant)
- RESUMES rendering
- Does another chunk...
- Repeat until done

Like a chef who checks orders every few minutes and handles urgent requests in between.

Fiber Node — The Core Data Structure

Every component gets a Fiber node:

```
// Internal Fiber node structure
{
  // Identity
  type: 'div',           // or MyComponent function
  key: null,
```

```

// Tree links (linked list, not tree!)
child:   FiberNode,    // first child
sibling: FiberNode,    // next sibling
return:  FiberNode,    // parent (called 'return')

// State
memoizedState: Hook,    // hooks linked list lives here
memoizedProps: {},       // last rendered props
pendingProps:  {},       // new props to process

// Work
effectTag: 'UPDATE',     // INSERT | UPDATE | DELETE
lanes:      0b0010,      // priority (bitmask)

// Double buffering
alternate: FiberNode,  // pointer to old fiber
}

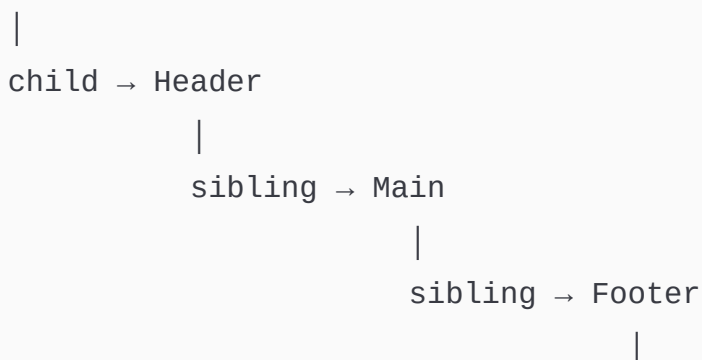
```

Fiber Tree is a Linked List

Why linked list instead of tree?

- You can pause in the middle of a linked list
- Save your position (which fiber you were on)
- Come back and continue later
- IMPOSSIBLE with recursive tree traversal

App



return (back to App)

Traversal: App → Header → Main → Footer → back to App

Can pause after any step and resume ✓

Two Phases of Rendering

PHASE 1: RENDER PHASE (async, interruptible)

React traverses fiber tree

Calls your component functions

Runs hooks

Finds what changed (diffing)

Tags fibers: INSERT / UPDATE / DELETE

→ No DOM changes yet

→ Can be paused, resumed, or aborted

PHASE 2: COMMIT PHASE (sync, cannot interrupt)

Sub-phase 1: beforeMutation

→ getSnapshotBeforeUpdate

Sub-phase 2: mutation

→ Actually changes real DOM

→ appendChild, removeChild, textContent

Sub-phase 3: layout

→ useEffect runs

→ componentDidMount/Update

↓

Browser paints screen

↓

useEffect runs (passive effects)

Double Buffering — current vs work-in-progress

React maintains TWO fiber trees:

current tree	work- in -progress tree
(on screen now)	(being built)
└── alternate ───┘	
(each fiber points to its mirror)	

Steps:

1. React builds work-**in**-progress tree
2. Render phase completes
3. **Commit**: swap → work-**in**-progress becomes **current**
4. Old **current** becomes next work-**in**-progress buffer
5. Memory reused – **no** garbage collection ✅

Like a **double**-buffered display:

Show frame A while rendering frame B

Swap **when** B **is** ready – **no** flicker

Priority Lanes (React 18)

```
// Lanes are bitmasks – multiple priorities merged with bitwise OR
const SyncLane      = 0b0000001; // user input, highest
const InputLane     = 0b0000010; // typing, clicking
const TransitionLane = 0b0000100; // startTransition
const IdleLane      = 0b1000000; // lowest, offscreen

// useTransition – you control priority
const [isPending, startTransition] = useTransition();

function handleSearch(e) {
  setInput(e.target.value); // SyncLane → instant

  startTransition(() => {
```

```
    setResults(filter(e.target.value)); // TransitionLane → can
interrupt
  });
}

// Result: typing always feels instant
// even while filtering 100,000 items
```

Code Examples — Fetch API, Todo App, Paginated Table

1. Basic Fetch API

```
function FetchExample() {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  async function fetchData() {
    try {
      setLoading(true);
      const res = await
fetch('https://jsonplaceholder.typicode.com/posts/1');
      if (!res.ok) throw new Error('HTTP Error: ' + res.status);
      const json = await res.json();
      setData(json);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }

  useEffect(() => { fetchData(); }, []);
```

```

if (loading) return <p>Loading...</p>;
if (error)    return <p style={{color:'red'}}>{error}</p>;

return (
  <div>
    <h2>{data?.title}</h2>
    <p>{data?.body}</p>
  </div>
);
}

```

2. Todo App (Complete)

```

function TodoApp() {
  const [todos, setTodos] = useState([]);
  const [input, setInput] = useState('');
  const [filter, setFilter] = useState('all'); // all | active |
  completed

  // Add todo
  const addTodo = () => {
    if (!input.trim()) return;
    setTodos([...todos, {
      id: Date.now(),
      text: input.trim(),
      completed: false
    }]);
    setInput('');
  };

  // Delete todo
  const deleteTodo = (id) => {
    setTodos(todos.filter(t => t.id !== id));
  };
}

```

```

// Toggle complete
const toggleTodo = (id) => {
  setTodos(todos.map(t =>
    t.id === id ? { ...t, completed: !t.completed } : t
  ));
};

// Filter todos
const filtered = todos.filter(t => {
  if (filter === 'active')    return !t.completed;
  if (filter === 'completed') return t.completed;
  return true; // all
});

return (
  <div style={{ maxWidth: 500, margin: '0 auto', padding: 20 }}>
    <h1>Todo App</h1>

    {/* Input */}
    <div style={{ display: 'flex', gap: 8 }}>
      <input
        value={input}
        onChange={e => setInput(e.target.value)}
        onKeyDown={e => e.key === 'Enter' && addTodo()}
        placeholder="Add a task..."
        style={{ flex: 1, padding: 8 }}
      />
      <button onClick={addTodo}>Add</button>
    </div>

    {/* Filter buttons */}
    <div style={{ margin: '10px 0', display: 'flex', gap: 8 }}>
      {[ 'all', 'active', 'completed' ].map(f => (
        <button

```

```

      key={f}
      onClick={() => setFilter(f)}
      style={{
        fontWeight: filter === f ? 'bold' : 'normal',
        background: filter === f ? '#007bff' : '#eee',
        color:      filter === f ? 'white'  : 'black',
        border: 'none', padding: '4px 12px', borderRadius: 4
      }}
    >
      {f}
    </button>
  )}}
</div>

```

```

{/* Todo list */}
{filtered.length === 0
  ? <p>No tasks here!</p>
  : filtered.map(todo => (
    <div
      key={todo.id}
      style={{
        display: 'flex',
        alignItems: 'center',
        gap: 8,
        padding: '8px 0',
        borderBottom: '1px solid #eee'
      }}
    >
      <input
        type="checkbox"
        checked={todo.completed}
        onChange={() => toggleTodo(todo.id)}
      />
      <span style={{
        flex: 1,

```



```

        textDecoration: todo.completed ? 'line-through' :
'none',
        color:          todo.completed ? 'gray' : 'black'
    }}>
    {todo.text}
</span>
<button
    onClick={() => deleteTodo(todo.id)}
    style={{
        background: 'red', color: 'white',
        border: 'none', borderRadius: 4, padding: '2px 8px'
    }}
>
    ×
</button>
</div>
))
}

{/* Footer */}
<p style={{ color: 'gray', fontSize: 12 }}>
    {todos.filter(t => !t.completed).length} tasks remaining
</p>
</div>
);
}

```

3. Paginated Table with limit, offset, size

```

function PaginatedTable() {
    const [data, setData]      = useState([]);
    const [loading, setLoading] = useState(false);
    const [total, setTotal]     = useState(0);

    // Pagination state

```

```

const [page, setPage]      = useState(1);    // current page
const [limit, setLimit]    = useState(10);   // rows per page

// Calculate offset from page and limit
const offset = (page - 1) * limit;

// Total pages
const totalPages = Math.ceil(total / limit);

useEffect(() => {
  async function fetchData() {
    setLoading(true);
    try {
      // Send limit and offset to API
      const res = await fetch(
        `https://jsonplaceholder.typicode.com/posts?
_limit=${limit}&_start=${offset}`
      );

      // Get total count from header
      const totalCount = res.headers.get('x-total-count');
      setTotal(Number(totalCount) || 100);

      const json = await res.json();
      setData(json);
    } catch (err) {
      console.error(err);
    } finally {
      setLoading(false);
    }
  }

  fetchData();
}, [page, limit]); // re-fetch when page or limit changes

```

```

return (
  <div style={{ padding: 20, fontFamily: 'sans-serif' }}>
    <h2>Posts Table</h2>

    {/* Page size selector */}
    <div style={{ marginBottom: 12, display: 'flex', alignItems:
'center', gap: 8 }}>
      <label>Rows per page:</label>
      <select
        value={limit}
        onChange={e => {
          setLimit(Number(e.target.value));
          setPage(1); // reset to page 1 when size changes
        }}
      >
        {[5, 10, 20, 50].map(s => (
          <option key={s} value={s}>{s}</option>
        ))}
      </select>
      <span style={{ marginLeft: 'auto', color: 'gray' }}>
        Showing {offset + 1}-{Math.min(offset + limit, total)} of
{total}
      </span>
    </div>

    {/* Table */}
    {loading ? (
      <p>Loading...</p>
    ) : (
      <table style={{
        width: '100%',
        borderCollapse: 'collapse',
        fontSize: 14
      }}>
        <thead>

```

```

        <tr style={{ background: '#007bff', color: 'white' }}>
          <th style={th}>ID</th>
          <th style={th}>Title</th>
          <th style={th}>Body</th>
        </tr>
      </thead>
      <tbody>
        {data.map((post, i) => (
          <tr
            key={post.id}
            style={{ background: i % 2 === 0 ? '#f9f9f9' :
'white' }}
          >
            <td style={td}>{post.id}</td>
            <td style={td}>{post.title}</td>
            <td style={td}>{post.body.substring(0, 60)}...</td>
          </tr>
        ))}
      </tbody>
    </table>
  )}

  {/* Pagination controls */}
  <div style={{
    marginTop: 16,
    display: 'flex',
    alignItems: 'center',
    gap: 8,
    justifyContent: 'center'
  }}>
    {/* First page */}
    <button
      onClick={() => setPage(1)}
      disabled={page === 1}
      style={btnStyle}
    >

```

```

>
  «
</button>

{/* Previous page */}
<button
  onClick={() => setPage(p => p - 1)}
  disabled={page === 1}
  style={btnStyle}
>
  < Prev
</button>

{/* Page numbers */}
{Array.from({ length: totalPages }, (_, i) => i + 1)
  .filter(p =>
    p === 1 || p === totalPages ||
    (p >= page - 2 && p <= page + 2)
  )
  .map((p, idx, arr) => (
    <>
      {idx > 0 && arr[idx - 1] !== p - 1 && (
        <span key={`dots-${p}`}>...</span>
      )}
      <button
        key={p}
        onClick={() => setPage(p)}
        style={{
          ...btnStyle,
          background: p === page ? '#007bff' : '#eee',
          color:      p === page ? 'white'   : 'black',
          fontWeight: p === page ? 'bold'    : 'normal'
        }}
      >
        {p}
    </>
  )
  )}

```



```
// Styles
const th = {
  padding: '10px 14px',
  textAlign: 'left',
  border: '1px solid #ddd'
};
const td = {
  padding: '8px 14px',
  border: '1px solid #ddd',
  verticalAlign: 'top'
};
const btnStyle = {
  padding: '6px 12px',
  cursor: 'pointer',
  border: '1px solid #ddd',
  borderRadius: 4,
  background: '#eee'
};
```

How pagination with limit/offset works:

Total records: 100

Limit (page size): 10

Page 1: offset=0, limit=10 → records 1-10

Page 2: offset=10, limit=10 → records 11-20

Page 3: offset=20, limit=10 → records 21-30

...

Page 10: offset=90, limit=10 → records 91-100

Formula:

offset = (currentPage - 1) * limit

API call:

GET /posts?_limit=10&_start=0 → page 1

GET /posts?_limit=10&_start=10 → page 2

GET /posts?_limit=10&_start=20 → page 3

4/15/2026, 6:35:26 AM



what is pure component ?
and what is non pure component ?
what is useReducer ?
what is useContext ?
diff between useState and useContext ?
what is useRef ? why we use it ?
what is virtual DOM ?
write code to create a list of item and show below items
const items=[
 {id:1, text:'item1'},
 {id:2, text:'item2'}
]
after that create a button and on click of that button a new item will be
added to that items list

4/16/2026, 7:18:53 AM



All Answers

1. Pure Component

A component that **only re-renders when its props or state actually change**. Same input = same output, no side effects.

```
// Class - PureComponent (auto shallow comparison)
class MyComponent extends React.PureComponent {
  render() {
    return <h1>{this.props.name}</h1>;
  }
}
```

```
// Functional - React.memo (same thing for functions)
const MyComponent = React.memo(({ name }) => {
  console.log("rendered!");
  return <h1>{name}</h1>;
});

// Test:
<MyComponent name="Alice" /> // renders → "rendered!"
<MyComponent name="Alice" /> // SKIPS render → same props ✓
<MyComponent name="Bob" />   // renders → props changed
```

Key point:

Pure Component = `React.PureComponent` / `React.memo`
 Does shallow comparison of props before re-rendering
 Same props → skip render → performance boost ✓

2. Non-Pure Component (Impure)

A component that **re-renders every time parent re-renders**, regardless of whether props changed.

```
// Class - regular Component (no comparison, always re-renders)
class MyComponent extends React.Component {
  render() {
    console.log("rendered!");
    return <h1>{this.props.name}</h1>;
  }
}

// Functional - regular function (always re-renders with parent)
const MyComponent = ({ name }) => {
  console.log("rendered!");
  return <h1>{name}</h1>;
}
```

```
};
```

```
// Parent re-renders 10 times → Child re-renders 10 times  
// Even if name prop never changed! ❌
```

Pure vs Non-Pure Comparison

Parent re-renders (count changes):

Non-Pure Child:

```
render 1 ✅  
render 2 ❌ (wasted)  
render 3 ❌ (wasted)  
render 4 ❌ (wasted)  
render 5 ❌ (wasted)
```

Pure Child (React.memo):

```
render 1 ✅  
skip ✅  
skip ✅  
skip ✅  
skip ✅
```

Pure = **only** renders **when** IT needs **to**

Non-Pure = renders **whenever** parent does

3. useReducer

Like `useState` but for **complex state logic**. Instead of setting state directly, you dispatch actions to a reducer function.

```
// Syntax  
const [state, dispatch] = useReducer(reducer, initialState);  
  
// reducer = function(currentState, action) → newState  
// dispatch = function to send actions
```

Simple Counter example:

```

// Step 1: Define reducer
function counterReducer(state, action) {
  switch (action.type) {
    case 'INCREMENT':
      return { count: state.count + 1 };
    case 'DECREMENT':
      return { count: state.count - 1 };
    case 'RESET':
      return { count: 0 };
    case 'ADD':
      return { count: state.count + action.payload };
    default:
      return state;
  }
}

// Step 2: Use in component
function Counter() {
  const [state, dispatch] = useReducer(
    counterReducer,
    { count: 0 } // initial state
  );

  return (
    <div>
      <h1>Count: {state.count}</h1>
      <button onClick={() => dispatch({ type: 'INCREMENT' })}>+
</button>
      <button onClick={() => dispatch({ type: 'DECREMENT' })}>-
</button>
      <button onClick={() => dispatch({ type: 'RESET'
    })}>Reset</button>
      <button onClick={() => dispatch({ type: 'ADD', payload: 10
    })}>+10</button>
    </div>
  );
}

```

```
);  
}
```

Real world example — Todo with useReducer:

```
function todoReducer(state, action) {  
  switch (action.type) {  
    case 'ADD':  
      return [...state, {  
        id: Date.now(),  
        text: action.payload,  
        done: false  
      }];  
    case 'DELETE':  
      return state.filter(t => t.id !== action.payload);  
    case 'TOGGLE':  
      return state.map(t =>  
        t.id === action.payload  
          ? { ...t, done: !t.done }  
          : t  
      );  
    default:  
      return state;  
  }  
}  
  
function TodoApp() {  
  const [todos, dispatch] = useReducer(todoReducer, []);  
  const [input, setInput] = useState('');  
  
  return (  
    <div>  
      <input value={input} onChange={e => setInput(e.target.value)}  
    />  
  
    <button onClick={() => {
```

```

    dispatch({ type: 'ADD', payload: input });
    setInput('');
  }}>Add</button>

  {todos.map(todo => (
    <div key={todo.id}>
      <span style={{
        textDecoration: todo.done ? 'line-through' : 'none'
      }}>
        {todo.text}
      </span>
      <button onClick={() =>
        dispatch({ type: 'TOGGLE', payload: todo.id })
      }>✓</button>
      <button onClick={() =>
        dispatch({ type: 'DELETE', payload: todo.id })
      }>X</button>
    </div>
  )))
</div>
);
}

```

useState vs useReducer — When to use which

useState:

- ✓ Simple values (string, number, boolean)
- ✓ Independent state pieces
- ✓ 1-2 ways state can change

useReducer:

- ✓ Complex objects with multiple fields
- ✓ Multiple related state updates
- ✓ State depends on previous state

- ✓ Many ways state can change
- ✓ Logic `is` testable separately

4. useContext

Lets you **read shared data from anywhere in the component tree** without passing props down through every level.

```
// Step 1: Create context
const ThemeContext = React.createContext();
const UserContext = React.createContext();

// Step 2: Provide value at top level
function App() {
  const [theme, setTheme] = useState('light');
  const user = { name: 'Alice', role: 'admin' };

  return (
    <UserContext.Provider value={user}>
      <ThemeContext.Provider value={{ theme, setTheme }}>
        <Page />
      </ThemeContext.Provider>
    </UserContext.Provider>
  );
}

// Step 3: Consume ANYWHERE in tree (no prop drilling!)
function DeepChild() {
  const user = useContext(UserContext);
  const { theme, setTheme } = useContext(ThemeContext);

  return (
    <div style={{
      background: theme === 'dark' ? '#333' : '#fff',

```

```

        color:      theme === 'dark' ? '#fff' : '#333'
    }}>
    <p>Hello {user.name}</p>
    <button onClick={() =>
        setTheme(t => t === 'dark' ? 'light' : 'dark')
    }>
        Toggle Theme
    </button>
</div>
);
}
// Page → Section → Card → DeepChild
// None of the middle components need to pass props! ✅

```

5. useState vs useContext

	useState	useContext
Scope	Local to ONE component	Shared across MANY components
Where state lives	Inside the component	In a Provider above
Who can read it	Only that component	Any component inside Provider
Who can update it	Only that component	Anyone with access to setter
Use case	Form input, toggle, counter	Theme, language, auth, user

```

// useState - LOCAL, only this component uses it
function Counter() {
    const [count, setCount] = useState(0); // only Counter uses count
    return <button onClick={() => setCount(c => c + 1)}>{count}
</button>;
}

// useContext - GLOBAL, any component can read it
const CountContext = createContext();

```



```

function App() {
  const [count, setCount] = useState(0);
  return (
    <CountContext.Provider value={{ count, setCount }}>
      <DisplayCount />  {/* reads count */}
      <IncrementBtn />  {/* updates count */}
      {/* both access same count without props! */}
    </CountContext.Provider>
  );
}

function DisplayCount() {
  const { count } = useContext(CountContext);
  return <h1>{count}</h1>;
}

function IncrementBtn() {
  const { setCount } = useContext(CountContext);
  return <button onClick={() => setCount(c => c + 1)}>+</button>;
}

```

6. useRef

useRef gives you a **mutable ref object** that persists across renders. Used for 2 main things:

Use 1 — Access DOM elements directly

```

function InputFocus() {
  const inputRef = useRef(null);

  function handleClick() {
    inputRef.current.focus(); // directly access DOM node
    inputRef.current.style.border = '2px solid blue';
  }
}

```

```

    console.log(inputRef.current.value);
  }

  return (
    <>
      <input ref={inputRef} placeholder="Type here" />
      <button onClick={handleClick}>Focus Input</button>
    </>
  );
}

```

Use 2 — Store value without causing re-render

```

function Timer() {
  const [seconds, setSeconds] = useState(0);
  const intervalRef = useRef(null); // stores interval ID, no re-render

  function start() {
    intervalRef.current = setInterval(() => {
      setSeconds(s => s + 1);
    }, 1000);
  }

  function stop() {
    clearInterval(intervalRef.current); // access stored ID
  }

  return (
    <div>
      <p>Seconds: {seconds}</p>
      <button onClick={start}>Start</button>
      <button onClick={stop}>Stop</button>
    </div>
  );
}

```

```
);  
}
```

Key difference — useRef vs useState

```
// useState → stores value + causes re-render on change  
const [count, setCount] = useState(0);  
setCount(5); // → component RE-RENDERS  
  
// useRef → stores value + NO re-render on change  
const countRef = useRef(0);  
countRef.current = 5; // → NO re-render ✅  
  
// When to use useRef:  
// ✅ Access DOM nodes (focus, scroll, measure)  
// ✅ Store timer/interval IDs  
// ✅ Store previous value  
// ✅ Any value you want to persist but NOT display
```

7. Virtual DOM

A lightweight **JavaScript copy** of the real DOM kept in memory. React uses it to find minimal changes before updating the real DOM.

Real DOM **update** (expensive):

- Change **one** text node
- Browser recalculates layout
- Repaints entire section
- Slow ❌

Virtual DOM **update** (cheap):

- Change **one** text node **in** JS object
- Compare **old** vs **new** virtual DOM
- Find **only** what changed (diffing)

- Update **ONLY** that **one real** DOM node
- Fast 

```
// What you write
const el = <h1 className="title">Hello {name}</h1>;

// Virtual DOM object created
{
  type: 'h1',
  props: {
    className: 'title',
    children: 'Hello Alice'
  }
}

// State changes → name = "Bob"
// New Virtual DOM:
{
  type: 'h1',
  props: {
    className: 'title',
    children: 'Hello Bob' // ← only this changed
  }
}

// React diffs old vs new
// Finds: only children text changed
// Updates: ONLY the text node in real DOM
// Does NOT touch: h1 tag, className, anything else 
```

8. List of Items + Add Button

```
import React, { useState } from 'react';
```

```

function ItemList() {
  const [items, setItems] = useState([
    { id: 1, text: 'item1' },
    { id: 2, text: 'item2' }
  ]);

  function addItem() {
    const newItem = {
      id: items.length + 1,
      text: `item${items.length + 1}`
    };
    setItems([...items, newItem]); // spread old + add new
  }

  return (
    <div style={{ padding: 20, fontFamily: 'sans-serif' }}>
      <h2>Item List</h2>

      {/* List of items */}
      <ul style={{ marginBottom: 16 }}>
        {items.map(item => (
          <li
            key={item.id}
            style={{
              padding: '8px 0',
              borderBottom: '1px solid #eee',
              listStyle: 'none'
            }}
          >
            ◆ {item.text} (id: {item.id})
          </li>
        ))}
      </ul>

      {/* Add button */}
    </div>
  );
}

```

```

    <button
      onClick={addItem}
      style={{
        padding: '8px 20px',
        background: '#007bff',
        color: 'white',
        border: 'none',
        borderRadius: 4,
        cursor: 'pointer',
        fontSize: 16
      }}
    >
      + Add Item
    </button>

    <p style={{ color: 'gray', marginTop: 8 }}>
      Total items: {items.length}
    </p>
  </div>
);
}

export default ItemList;

```

With custom text input:

```

function ItemList() {
  const [items, setItems] = useState([
    { id: 1, text: 'item1' },
    { id: 2, text: 'item2' }
  ]);
  const [input, setInput] = useState('');

  function addItem() {
    if (!input.trim()) return; // no empty items
  }
}

```

```

    setItems([...items, {
      id: Date.now(),           // unique id
      text: input.trim()
    }]);
    setInput('');               // clear input
  }

  function deleteItem(id) {
    setItems(items.filter(item => item.id !== id));
  }

  return (
    <div style={{ padding: 20, maxWidth: 400 }}>
      <h2>Item List</h2>

      {/* Items */}
      <ul style={{ padding: 0 }}>
        {items.map(item => (
          <li
            key={item.id}
            style={{
              display: 'flex',
              justifyContent: 'space-between',
              alignItems: 'center',
              padding: '8px 0',
              borderBottom: '1px solid #eee',
              listStyle: 'none'
            }}
          >
            <span>◆ {item.text}</span>
            <button
              onClick={() => deleteItem(item.id)}
              style={{
                background: 'red', color: 'white',
                border: 'none', borderRadius: 4,

```

```

        padding: '2px 8px', cursor: 'pointer'
      }}
    >
      ×
    </button>
  </li>
))}
</ul>

{/* Input + Add button */}
<div style={{ display: 'flex', gap: 8, marginTop: 16 }}>
  <input
    value={input}
    onChange={e => setInput(e.target.value)}
    onKeyDown={e => e.key === 'Enter' && addItem()}
    placeholder="Enter item name..."
    style={{ flex: 1, padding: '8px', borderRadius: 4,
      border: '1px solid #ddd' }}
  />
  <button
    onClick={addItem}
    style={{
      padding: '8px 16px', background: '#28a745',
      color: 'white', border: 'none',
      borderRadius: 4, cursor: 'pointer'
    }}
  >
    + Add
  </button>
</div>

<p style={{ color: 'gray', marginTop: 8, fontSize: 13 }}>
  {items.length} items total
</p>
</div>

```



```
);  
}
```

How addItem works:

Initial state:

```
items = [{id:1, text:'item1'}, {id:2, text:'item2'}]
```

User clicks Add:

```
newItem = { id: 3, text: 'item3' }
```

```
setItems([...items, newItem])
```

```
→ [...old items, new item]
```

```
→ [{id:1}, {id:2}, {id:3}]
```

React re-renders with new list 

Key prop on each item helps React

update only the new item in DOM 

4/16/2026, 7:19:51 AM



What is error boundary ? explain in details (why we used ? where we used and how we use ?)

What is Redux ? explain in details (why we used ? where we used and how we use?)

what is React Fiber ? explain in details (why we used ? where we used and how we use?)

4/19/2026, 4:37:14 PM



1. Error Boundary — Complete Guide

What is it?

A React component that **catches JavaScript errors** anywhere in its child component tree, logs them, and displays a **fallback UI** instead of crashing the whole app.

Without **Error** Boundary:

One component throws **error**

→ Entire app crashes

→ User sees blank white screen ❌

With **Error** Boundary:

One component throws **error**

→ **Error** Boundary catches it

→ Shows fallback UI ("Something went wrong")

→ Rest **of** app still works ✅

Why We Use It

Real world scenario:

Your app has:

Header

Sidebar

ProductList ← this component crashes (API returned bad data)

Footer

Without Error Boundary:

ProductList crashes → ENTIRE page goes blank

User loses everything ❌

With Error Boundary:

ProductList crashes → shows "Failed to load products"

Header, Sidebar, Footer still work perfectly ✅

User can still navigate the app

Where We Use It

- ✅ Around route-level components
- ✅ Around third-party components
- ✅ Around widgets that fetch data
- ✅ Around anything that might fail
- ✅ Multiple boundaries = more granular recovery



/	\
Sidebar	ProductList

How We Use It

Error Boundaries MUST be **class components** (no hook equivalent yet).

```
// ErrorBoundary.jsx
import React from 'react';

class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      hasError: false,
      error: null,
      errorInfo: null
    };
  }

  // Runs when error is thrown
  // Returns new state
  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }

  // Runs after error is caught
  // Good for logging to error service
  componentDidCatch(error, errorInfo) {
    console.error('Error caught:', error);
    console.error('Component stack:', errorInfo.componentStack);

    // Log to service like Sentry
    // Sentry.captureException(error);
    this.setState({ errorInfo });
  }
}
```

```

}

render() {
  if (this.state.hasError) {
    // Fallback UI
    return (
      <div style={{
        padding: 20,
        background: '#fff3f3',
        border: '1px solid red',
        borderRadius: 8,
        textAlign: 'center'
      }}>
        <h2>⚠️ Something went wrong</h2>
        <p style={{ color: 'red' }}>
          {this.state.error?.message}
        </p>
        <button
          onClick={() => this.setState({
            hasError: false,
            error: null
          })}
          style={{
            padding: '8px 16px',
            background: '#007bff',
            color: 'white',
            border: 'none',
            borderRadius: 4,
            cursor: 'pointer'
          }}
        >
          Try Again
        </button>
      </div>
    );
  }
}

```

```
    }

    return this.props.children;
  }
}

export default ErrorBoundary;
```

Usage — Wrap any component

```
// App.jsx

// 1. Route level – protect entire pages
function App() {
  return (
    <BrowserRouter>
      <ErrorBoundary>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/shop" element={<Shop />} />
          <Route path="/profile" element={<Profile />} />
        </Routes>
      </ErrorBoundary>
    </BrowserRouter>
  );
}

// 2. Component level – protect specific sections
function Dashboard() {
  return (
    <div>
      <Header /> {/* unprotected, rarely fails */}

      <ErrorBoundary>
        <UserProfile /> {/* if this fails, only this section shows
```

```

error */}
    </ErrorBoundary>

    <ErrorBoundary>
        <ProductList />  {/* if this fails, only this section shows
error */}
    </ErrorBoundary>

    <ErrorBoundary>
        <RecommendationWidget />  {/* third party - might crash */}
    </ErrorBoundary>

    <Footer />
</div>
);
}

// 3. Custom fallback per boundary
function App() {
    return (
        <ErrorBoundary fallback={<h1>Page crashed!</h1>}>
            <ErrorBoundary fallback={<p>Chart failed to load</p>}>
                <AnalyticsChart />
            </ErrorBoundary>
            <ErrorBoundary fallback={<p>Table failed to load</p>}>
                <DataTable />
            </ErrorBoundary>
        </ErrorBoundary>
    );
}

```

Reusable Error Boundary with custom fallback

```

class ErrorBoundary extends React.Component {
    state = { hasError: false, error: null };

```

```

static getDerivedStateFromError(error) {
  return { hasError: true, error };
}

componentDidCatch(error, info) {
  console.error(error, info);
}

render() {
  if (this.state.hasError) {
    // Use custom fallback if provided
    if (this.props.fallback) {
      return this.props.fallback;
    }
    // Default fallback
    return <h2>Something went wrong.</h2>;
  }
  return this.props.children;
}
}

// Different fallbacks for different sections
<ErrorBoundary fallback={<h2>Chart unavailable</h2>}>
  <Chart />
</ErrorBoundary>

<ErrorBoundary fallback={<h2>Map unavailable</h2>}>
  <Map />
</ErrorBoundary>

```

What Error Boundary CANNOT catch

```

// ❌ Async errors (setTimeout, fetch)
useEffect(() => {

```



```
    setTimeout(() => {
      throw new Error("async error"); // NOT caught!
    }, 1000);
  });

// ❌ Event handlers
<button onClick={() => {
  throw new Error("click error"); // NOT caught!
}}>

// ❌ The Error Boundary itself

// ✅ Errors in render()
// ✅ Errors in lifecycle methods
// ✅ Errors in child component tree

// For async/event errors → use try/catch
async function fetchData() {
  try {
    await fetch('/api');
  } catch(err) {
    setError(err.message); // handle manually
  }
}
```

Two lifecycle methods

```
getDerivedStateFromError(error)
  → called during render phase
  → returns new state { hasError: true }
  → used to show fallback UI

componentDidCatch(error, info)
  → called during commit phase
```

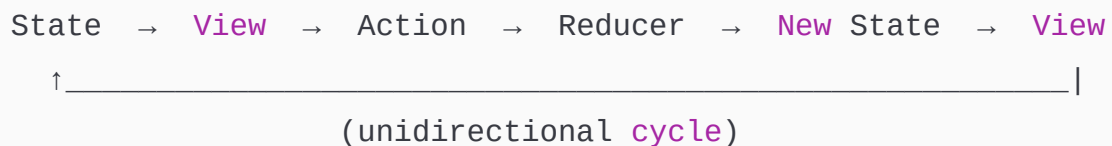
- used for side effects (logging)
- `info.componentStack` shows which component crashed

2. Redux — Complete Guide

What is Redux?

A **predictable state management library**. All app state lives in ONE central store. State changes only through dispatched actions processed by pure reducers.

Core idea:



Why We Use Redux

Problem 1 – Prop Drilling:

```
App
├── Page
│   └── Section
│       └── Component
│           └── DeepChild ← needs user data
```

Solution: Store → DeepChild reads directly ✅

Problem 2 – Shared State:

Cart (top right) and ProductList both need cart data

Solution: Both read from same store ✅

Problem 3 – Complex state logic:

Many actions change state in different ways

Solution: Reducers organize all logic ✅

Problem 4 — Debugging:

Hard to track what changed state

Solution: Redux DevTools → time-travel debugging ✓

Where We Use Redux

- ✓ Large scale applications
- ✓ Multiple teams working together
- ✓ Complex state with many interactions
- ✓ State shared between many components
- ✓ Need for debugging / state history

- ✗ Small apps → use useState
- ✗ Simple sharing → use Context
- ✗ Server state → use React Query

How Redux Works — 5 Core Concepts

1. STORE → single JS object holding ALL state
2. ACTION → plain object { type, payload } describing what happened
3. REDUCER → pure function (state, action) => newState
4. DISPATCH → sends action to store
5. SELECTOR → reads specific data from store

How We Use It — Step by Step

```
npm install @reduxjs/toolkit react-redux
```

Step 1 — Create Slice (action + reducer combined)

```

// store/cartSlice.js
import { createSlice } from '@reduxjs/toolkit';

const cartSlice = createSlice({
  name: 'cart',
  initialState: {
    items: [],
    totalPrice: 0
  },

  reducers: {
    addItem: (state, action) => {
      // RTK uses Immer - can "mutate" directly
      state.items.push(action.payload);
      state.totalPrice += action.payload.price;
    },

    removeItem: (state, action) => {
      state.items = state.items.filter(
        item => item.id !== action.payload
      );
    },

    clearCart: (state) => {
      state.items = [];
      state.totalPrice = 0;
    }
  }
});

export const { addItem, removeItem, clearCart } = cartSlice.actions;
export default cartSlice.reducer;

```

Step 2 — Create Store

```
// store/store.js
import { configureStore } from '@reduxjs/toolkit';
import cartReducer from './cartSlice';
import userReducer from './userSlice';
import productsReducer from './productsSlice';

const store = configureStore({
  reducer: {
    cart: cartReducer,
    user: userReducer,
    products: productsReducer
  }
});

export default store;
```

Step 3 — Provide to App

```
// index.js
import { Provider } from 'react-redux';
import store from './store/store';

ReactDOM.render(
  <Provider store={store}>
    <App />
  </Provider>,
  document.getElementById('root')
);
```

Step 4 — Use in Components

```
// ProductCard.jsx
import { useDispatch, useSelector } from 'react-redux';
import { addItem } from './store/cartSlice';
```

```

function ProductCard({ product }) {
  const dispatch = useDispatch();

  // READ from store
  const cartItems = useSelector(state => state.cart.items);
  const isInCart = cartItems.some(item => item.id === product.id);

  return (
    <div>
      <h3>{product.name}</h3>
      <p>₹{product.price}</p>
      <button
        onClick={() => dispatch(addItem(product))} // WRITE to store
        disabled={isInCart}
      >
        {isInCart ? 'In Cart' : 'Add to Cart'}
      </button>
    </div>
  );
}

```

```

// CartIcon.jsx (completely separate component, same store)
function CartIcon() {
  const items = useSelector(state => state.cart.items);
  const totalPrice = useSelector(state => state.cart.totalPrice);
  const dispatch = useDispatch();

  return (
    <div>
      🛒 {items.length} items – ₹{totalPrice}
      <button onClick={() => dispatch(clearCart())}>Clear</button>
    </div>
  );
}

```

Async Redux — API Calls

```
// store/userSlice.js
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';

// Async action
export const fetchUsers = createAsyncThunk(
  'users/fetchAll',
  async (_, { rejectWithValue }) => {
    try {
      const res = await fetch('/api/users');
      if (!res.ok) throw new Error('Failed');
      return await res.json();
    } catch (err) {
      return rejectWithValue(err.message);
    }
  }
);

const userSlice = createSlice({
  name: 'users',
  initialState: { list: [], loading: false, error: null },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchUsers.pending, (state) => {
        state.loading = true;
        state.error = null;
      })
      .addCase(fetchUsers.fulfilled, (state, action) => {
        state.loading = false;
        state.list = action.payload;
      })
      .addCase(fetchUsers.rejected, (state, action) => {
        state.loading = false;
```

```

        state.error    = action.payload;
    });
}
});

export default userSlice.reducer;

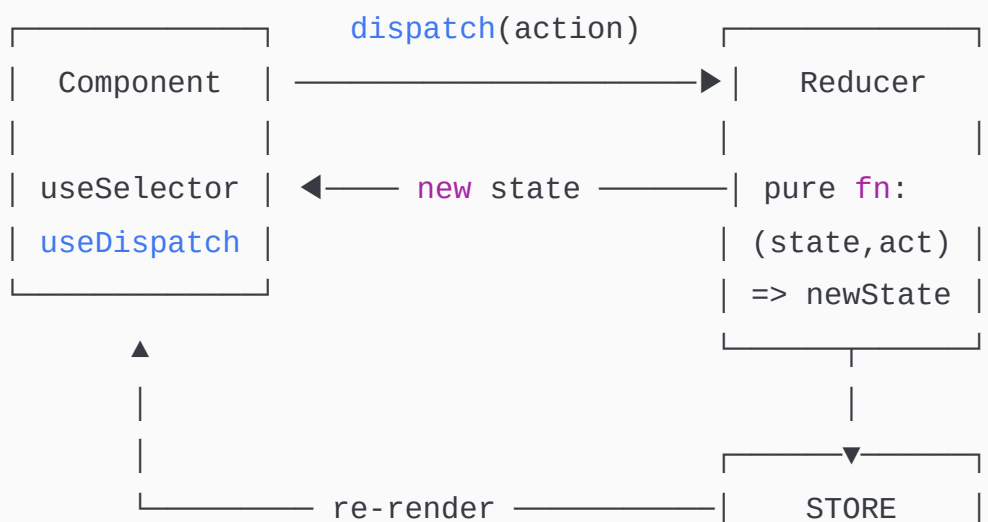
// Component
function UserList() {
    const { list, loading, error } = useSelector(s => s.users);
    const dispatch = useDispatch();

    useEffect(() => {
        dispatch(fetchUsers()); // trigger async thunk
    }, []);

    if (loading) return <p>Loading...</p>;
    if (error)    return <p>Error: {error}</p>;
    return <ul>{list.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}

```

Redux Flow Diagram




```
| { state } |  
└──────────┘
```

3. React Fiber — Complete Guide

What is React Fiber?

The **complete rewrite of React's reconciliation engine** (React 16). It transforms React's rendering from synchronous/blocking to asynchronous/interruptible.

Why React Fiber Was Needed

Before Fiber — Stack Reconciler (React 15):

Problem: Synchronous rendering

User types **in** search box

- React starts re-rendering large tree
- Takes **400ms** (large component tree)
- During **400ms**:

- ❌ Keystrokes ignored
- ❌ Animations freeze
- ❌ Clicks don't *work*
- ❌ Browser completely frozen

Root cause:

Old reconciler used JS **call** stack (recursive)

Once started → cannot **stop until** done

Like going down a staircase — must reach bottom

Cannot pause halfway ❌

What Fiber Changed

After Fiber:

Rendering **is** split **into** units **of** work

Each unit = **one** fiber node

React does work → checks **time** →

"Is there urgent work? (user typing, animation?)"

YES → pause, handle urgent work, resume

NO → continue rendering

Key insight:

Total work **time**: **similar**

User experience: completely different

UI stays responsive throughout 

Where Fiber Is Used

Fiber is the ENGINE running under every React app.

You don't "use" it directly – it works automatically.

But Fiber ENABLES these features you use directly:

1. Concurrent Mode (React 18)
2. `useTransition`
3. `useDeferredValue`
4. `Suspense`
5. Streaming SSR
6. React Server Components

How Fiber Works — Deep Internals

Fiber Node — one per component:

```
// Every component gets this internal object
{
  // What it is
  type:    MyComponent,    // function or 'div'
  key:     null,

  // Tree structure (linked list)
  child:   FiberNode,      // first child
  sibling:  FiberNode,      // next sibling
  return:  FiberNode,      // parent

  // State & Props
  pendingProps: {},        // new props
  memoizedProps: {},       // last rendered props
  memoizedState: Hook,     // hooks linked list

  // Work to do
  effectTag: 'UPDATE',     // INSERT | UPDATE | DELETE

  // Priority
  lanes:    0b0010,        // bitmask priority

  // Double buffering
  alternate: FiberNode     // old version for diffing
}
```

Why linked list instead of tree?

Tree traversal (old):	Linked list (Fiber):
recursive	iterative
uses call stack	uses pointers
cannot pause	CAN pause 
App	App.child → Header



Header.sibling → Main
 Main.sibling → Footer
 Footer.return → App

Must go top to bottom
 all at once

Can stop at ANY point
 save pointer, resume later

Two Phases

PHASE 1: RENDER (async, interruptible)

- Traverse fiber tree
- Call component functions
- Run hooks
- Diff old vs new
- Tag: INSERT / UPDATE / DELETE

⚡ CAN be paused, resumed, aborted

❌ No DOM changes yet



PHASE 2: COMMIT (sync, un interruptible)

Sub-phase 1: beforeMutation

→ getSnapshotBeforeUpdate

Sub-phase 2: mutation

→ appendChild / removeChild

→ textContent updates

→ Real DOM now updated ✅

Sub-phase 3: layout

→ useLayoutEffect

→ componentDidMount/Update

```
|  
| ⚡ Browser paints screen  
| ⚡ useEffect runs  
|
```

Double Buffering

React keeps TWO trees:

```
current tree          work-in-progress  
(showing on screen)  (being built)  
    |                  |  
    └─── alternate ───┘  
    (mirror pointers)
```

Steps:

1. Build work-in-progress (render phase)
2. Commit: WIP becomes current
3. Old current → next WIP buffer
4. Memory reused → no garbage collection

Like a double-buffered screen:

Display frame A while rendering frame B

Swap instantly → no flicker ✅

Priority System — Lanes

```
// React 18 priority lanes (bitmask)  
SyncLane      = 0b0000001 // user input, highest  
InputLane     = 0b0000010 // click, type  
TransitionLane = 0b0001000 // startTransition  
IdleLane      = 0b1000000 // lowest, offscreen  
  
// useTransition – you control priority
```

```

const [isPending, startTransition] = useTransition();

function handleSearch(e) {
  // HIGH priority → runs immediately, never interrupted
  setInputValue(e.target.value);

  // LOW priority → can be paused for urgent work
  startTransition(() => {
    setSearchResults(filterData(e.target.value));
  });
}

// User types rapidly:
// Each keystroke → setInputValue runs immediately ✅
// filterData gets interrupted and restarted ✅
// Input feels instant even with 100,000 items ✅

```

Fiber + Hooks Connection

```

// Hooks are stored as linked list on fiber.memoizedState

function Counter() {
  const [count, setCount] = useState(0); // Hook 1
  const [name, setName] = useState(''); // Hook 2
  useEffect(() => {}, [count]); // Hook 3
}

// fiber.memoizedState:
Hook1          Hook2          Hook3
{ state: 0 } → { state: '' } → { effect: fn } → null

// Why hooks can't be in conditions:
// Every render MUST have same number of hooks

```

```
// in same order – it's a linked list!
```

```
// Missing one hook = entire list shifts = CRASH ❌
```

Summary Table

Concept	Before Fiber	After Fiber
Rendering	Synchronous	Async, chunked
Interruptible	❌ No	✅ Yes
Priority	❌ None	✅ Lanes system
UI during render	Freezes	Stays responsive
Concurrent features	❌ None	✅ useTransition, Suspense
Data structure	Recursive tree	Linked list

Simple way to remember:

Error Boundary → safety net for component crashes

Redux → single store for all app state

React Fiber → smart engine for rendering

Error Boundary: "If something breaks, show this instead"

Redux: "All state lives here, changes only through actions"

Fiber: "Render smartly – prioritize, pause, resume" 🚀

4/19/2026, 4:38:41 PM



Search/Filter Bar: Add a search input to your Todo app that filters the list in real-time.

Debouncing: Implement a search bar that only triggers an API call 500ms after the user stops typing.

The "Counter" with a Twist: Create a counter that starts/stops/resets and preserves the value even if the page refreshes (using localStorage).

Infinite Scroll: Load more items as the user scrolls to the bottom of the page.

4/19/2026, 9:31:14 PM



Here are all 4 implementations:

1. Todo App with Search/Filter Bar

Viewed file ▾

- 📄 Check frontend design skill
- 🕒 Done

Todo list

Add a new task...

Add

🔍 Search tasks...

All

Active

Done

☐ Buy groceries



☐ Read a book



☒ Write unit tests

×

☐ Fix navigation bug

×

☒ Review pull request

×

3 tasks remaining · 5 total

2. Debounced Search Bar (500ms API Call)

Debounced search

API call fires 500ms after you stop typing. Watch the log below.

🔍 Search users... (try 'John', 'Alex')

● Ready — start typing

event log

[06:13:58] App ready. Waiting for input...

The debounce logic in plain code:

```
// The core pattern
let timer = null;

function onType(value) {
  clearTimeout(timer); // cancel previous timer on every keystroke

  timer = setTimeout(() => {
    callAPI(value); // only fires 500ms after LAST keystroke
  }, 500);
}
```

```
// React version
function SearchBar() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);

  useEffect(() => {
    if (!query.trim()) return;

    const timer = setTimeout(async () => {
      const res = await fetch(`/api/search?q=${query}`);
      const data = await res.json();
      setResults(data);
    }, 500);

    return () => clearTimeout(timer); // cleanup = cancel on next
    keystroke
  }, [query]);

  return <input value={query} onChange={e =>
    setQuery(e.target.value)} />;
}
```

3. Counter with localStorage (Start/Stop/Reset)

Persistent counter

0

stopped

Start

Reset

Saved to `localStorage` — value persists on refresh

counter_value 0

Last saved —

Simulate page refresh → Refresh

React code for this:

```
function PersistentCounter() {
  // Initialize from localStorage
  const [count, setCount] = useState(() => {
    return parseInt(localStorage.getItem('counter') || '0', 10);
  });
  const [running, setRunning] = useState(false);
  const intervalRef = useRef(null);

  // Save to localStorage whenever count changes
  useEffect(() => {
    localStorage.setItem('counter', count);
  }, [count]);

  function startStop() {
    if (running) {
      clearInterval(intervalRef.current);
      setRunning(false);
    } else {
      intervalRef.current = setInterval(() => {
        setCount(c => c + 1); // functional update = always latest
        value
      }, 1000);
      setRunning(true);
    }
  }
}
```

```

    }
  }

  function reset() {
    clearInterval(intervalRef.current);
    setRunning(false);
    setCount(0);
  }

  // Cleanup on unmount
  useEffect(() => () => clearInterval(intervalRef.current), []);

  return (
    <div>
      <h1 style={{ color: running ? 'green' : 'black' }}>{count}</h1>
      <button onClick={startStop}>{running ? 'Stop' : 'Start'}</button>
      <button onClick={reset}>Reset</button>
      <p>Saved to localStorage – survives refresh 

```

4. Infinite Scroll

Infinite scroll

Scroll down — more items load automatically as you reach the bottom.

10

loaded

1

pages fetched

2

API calls

DH

Dan Hiro

#1

	dan.hiro1@yahoo.com	
GO	Grace Omar grace.omar2@outlook.com	#2
JG	Jake Grace jake.grace3@dev.io	#3
MN	Maya Nora maya.nora4@work.co	#4
AF	Alice Frank alice.frank5@gmail.com	#5
DM	Dan Maya dan.maya6@yahoo.com	#6

React code for infinite scroll:

```
function InfiniteScroll() {
  const [items, setItems] = useState([]);
  const [page, setPage] = useState(1);
  const [loading, setLoading] = useState(false);
  const [hasMore, setHasMore] = useState(true);
  const loaderRef = useRef(null);

  // Fetch page of data
  useEffect(() => {
    if (!hasMore) return;
    async function load() {
      setLoading(true);
      const res = await fetch(
        `https://jsonplaceholder.typicode.com/posts?_page=${page}&_limit=10`
      );
      const data = await res.json();
      setItems(prev => [...prev, ...data]); // APPEND, not replace
    }
  }, [page, hasMore]);
}
```

```

        setHasMore(data.length === 10);          // if less than limit,
no more
        setLoading(false);
    }
    load();
}, [page]);

// IntersectionObserver watches the bottom sentinel div
useEffect(() => {
    const observer = new IntersectionObserver(
        (entries) => {
            if (entries[0].isIntersecting && !loading && hasMore) {
                setPage(p => p + 1); // triggers useEffect above
            }
        },
        { threshold: 0.1 }
    );

    if (loaderRef.current) observer.observe(loaderRef.current);
    return () => observer.disconnect();
}, [loading, hasMore]);

return (
    <div style={{ height: '500px', overflowY: 'auto' }}>
        {items.map(item => (
            <div key={item.id} style={{ padding: 16, borderBottom: '1px
solid #eee' }}>
                <h4>{item.title}</h4>
                <p>{item.body}</p>
            </div>
        ))}

        {/* Sentinel div – when this enters viewport, load more */}
        <div ref={loaderRef}>
            {loading && <p>Loading more...</p>}

```

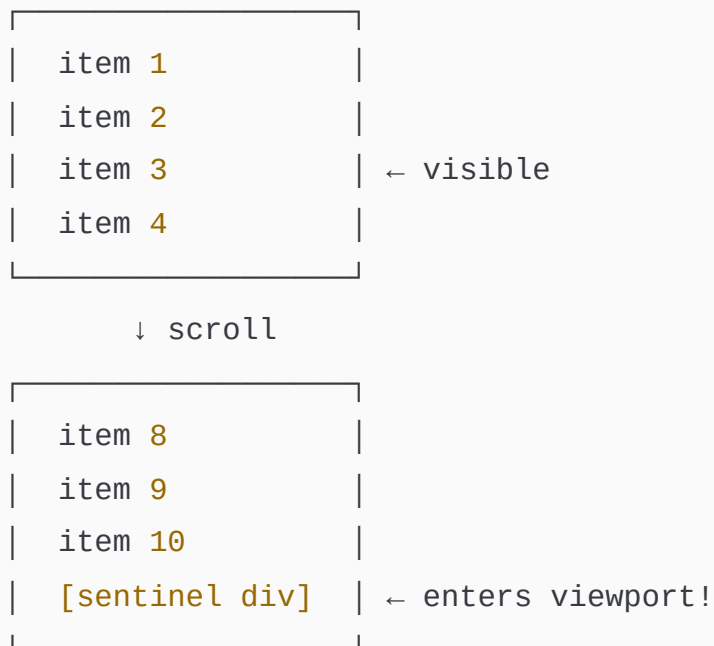
```

    {!hasMore && <p>All items loaded </p>}
  </div>
</div>
);
}

```

How IntersectionObserver works:

scroll-box viewport



↓

Observer fires callback

↓

setPage(p => p + 1)

↓

useEffect fetches next page

↓

items appended to list 

4/19/2026, 9:33:32 PM



you missed to add code for this
Todo App with Search/Filter Bar

4/19/2026, 9:35:19 PM



Here's the full React code for the Todo App with Search/Filter Bar:

```
import React, { useState } from 'react';

function TodoApp() {
  const [todos, setTodos] = useState([
    { id: 1, text: 'Buy groceries',      completed: false },
    { id: 2, text: 'Read a book',        completed: false },
    { id: 3, text: 'Write unit tests',    completed: true  },
    { id: 4, text: 'Fix navigation bug',  completed: false },
    { id: 5, text: 'Review pull request', completed: true  },
  ]);

  const [input, setInput] = useState('');
  const [search, setSearch] = useState('');
  const [filter, setFilter] = useState('all'); // all | active |
  completed

  // — Add Todo —————
  function addTodo() {
    if (!input.trim()) return; // no empty todos
    setTodos([
      ...todos,
      { id: Date.now(), text: input.trim(), completed: false }
    ]);
    setInput(''); // clear input after add
  }
}
```



```

}

// — Delete Todo —————
function deleteTodo(id) {
  setTodos(todos.filter(t => t.id !== id));
}

// — Toggle Complete —————
function toggleTodo(id) {
  setTodos(todos.map(t =>
    t.id === id ? { ...t, completed: !t.completed } : t
  ));
}

// — Filter + Search (both work together) —————
const visibleTodos = todos
  .filter(t => {
    // Step 1: apply tab filter
    if (filter === 'active') return !t.completed;
    if (filter === 'completed') return t.completed;
    return true; // 'all'
  })
  .filter(t =>
    // Step 2: apply search query (case insensitive)
    t.text.toLowerCase().includes(search.toLowerCase())
  );

const activeCount = todos.filter(t => !t.completed).length;

return (
  <div style={{
    maxWidth: 500,
    margin: '40px auto',
    padding: '0 16px',
    fontFamily: 'sans-serif'
  }}

```

```

}}>
<h1 style={{ fontSize: 28, marginBottom: 24 }}>Todo App</h1>

{/* — Add Input — */}
<div style={{ display: 'flex', gap: 8, marginBottom: 12 }}>
  <input
    value={input}
    onChange={e => setInput(e.target.value)}
    onKeyDown={e => e.key === 'Enter' && addTodo()}
    placeholder="Add a new task..."
    style={{
      flex: 1,
      padding: '10px 14px',
      fontSize: 15,
      borderRadius: 8,
      border: '1px solid #ddd',
      outline: 'none'
    }}
  />
  <button
    onClick={addTodo}
    style={{
      padding: '10px 20px',
      background: '#111',
      color: '#fff',
      border: 'none',
      borderRadius: 8,
      fontSize: 15,
      cursor: 'pointer'
    }}
  >
    Add
  </button>
</div>

```

```

{/* — Search Bar — */}
<div style={{ position: 'relative', marginBottom: 12 }}>
  <span style={{
    position: 'absolute',
    left: 12,
    top: '50%',
    transform: 'translateY(-50%)',
    color: '#999',
    fontSize: 16
  }}>
    🔍
  </span>
  <input
    value={search}
    onChange={e => setSearch(e.target.value)}
    placeholder="Search tasks..."
    style={{
      width: '100%',
      padding: '10px 14px 10px 36px',
      fontSize: 15,
      borderRadius: 8,
      border: '1px solid #ddd',
      outline: 'none',
      boxSizing: 'border-box'
    }}
  />
{/* Clear search button */}
{search && (
  <button
    onClick={() => setSearch('')}
    style={{
      position: 'absolute',
      right: 10,
      top: '50%',
      transform: 'translateY(-50%)',

```

```

        background: 'none',
        border: 'none',
        cursor: 'pointer',
        color: '#999',
        fontSize: 18,
        lineHeight: 1
      }}
    >
      x
    </button>
  )}
</div>

{/* — Filter Tabs — */}
<div style={{ display: 'flex', gap: 6, marginBottom: 16 }}>
  {[ 'all', 'active', 'completed' ].map(f => (
    <button
      key={f}
      onClick={() => setFilter(f)}
      style={{
        padding: '5px 16px',
        borderRadius: 99,
        border: '1px solid #ddd',
        background: filter === f ? '#111' : 'transparent',
        color: filter === f ? '#fff' : '#555',
        cursor: 'pointer',
        fontSize: 13,
        fontWeight: filter === f ? 500 : 400,
        textTransform: 'capitalize'
      }}
    >
      {f}
    </button>
  ))}
</div>

```

```

    { /* — Todo List — */ }
    <div style={{ display: 'flex', flexDirection: 'column', gap: 6
  }}>

      {visibleTodos.length === 0 ? (
        <p style={{
          textAlign: 'center',
          color: '#999',
          padding: '24px 0',
          fontSize: 14
        }}>
          {search
            ? `No tasks match "${search}"`
            : 'No tasks here!'
          }
        </p>
      ) : (
        visibleTodos.map(todo => (
          <div
            key={todo.id}
            style={{
              display: 'flex',
              alignItems: 'center',
              gap: 10,
              padding: '12px 14px',
              background: '#fff',
              border: '1px solid #eee',
              borderRadius: 8,
              // Fade completed items slightly
              opacity: todo.completed ? 0.7 : 1
            }}
          >
            { /* Checkbox */ }
            <input
              type="checkbox"

```

```

        checked={todo.completed}
        onChange={() => toggleTodo(todo.id)}
        style={{ width: 16, height: 16, cursor: 'pointer',
flexShrink: 0 }}
      />

      {/* Text – highlight search match */}
      <span style={{
        flex: 1,
        fontSize: 15,
        textDecoration: todo.completed ? 'line-through' :
'none',
        color:          todo.completed ? '#aaa'          :
'#111'
      }}>
        {highlightMatch(todo.text, search)}
      </span>

      {/* Delete button */}
      <button
        onClick={() => deleteTodo(todo.id)}
        style={{
          background: 'none',
          border: 'none',
          cursor: 'pointer',
          color: '#ccc',
          fontSize: 18,
          lineHeight: 1,
          padding: '0 2px',
          flexShrink: 0
        }}
        onMouseEnter={e => e.target.style.color = '#e24b4a'}
        onMouseLeave={e => e.target.style.color = '#ccc'}
      >
        ×

```

```

        </button>
      </div>
    ))
  )}
</div>

{/* — Footer Stats — */}
<div style={{
  marginTop: 16,
  display: 'flex',
  justifyContent: 'space-between',
  fontSize: 13,
  color: '#999'
}}>
  <span>{activeCount} task{activeCount !== 1 ? 's' : ''}
remaining</span>
  <span>
    {visibleTodos.length} of {todos.length} shown
    {search && ` · searching "${search}"`}
  </span>
</div>
</div>
);
}

// — Highlight matching text in search results —
function highlightMatch(text, query) {
  if (!query.trim()) return text; // no search → return plain text

  const index = text.toLowerCase().indexOf(query.toLowerCase());
  if (index === -1) return text; // no match → return plain text

  // Split text into 3 parts: before, match, after
  const before = text.slice(0, index);
  const match = text.slice(index, index + query.length);

```

```

const after = text.slice(index + query.length);

return (
  <>
    {before}
    <mark style={{
      background: '#fef3c7',
      color: '#92400e',
      borderRadius: 2,
      padding: '0 1px'
    }}>
      {match}
    </mark>
    {after}
  </>
);
}

export default TodoApp;

```

How it all connects:

State:

todos → full list
 input → new task input
 search → search bar value
 filter → all | active | completed

visibleTodos = todos

.filter(tab filter) ← step 1: all / active / completed
 .filter(search query) ← step 2: text match

Both filters work TOGETHER in real time

No API call needed – pure JS array filter

Search "read":

```
todos = [Buy groceries, Read a book, Write tests...]
```

↑ match

```
visibleTodos = [Read a book]
```

```
highlight "Read" in yellow ✓
```

Filter "active" + search "bug":

Step 1: filter to uncompleted only

Step 2: filter those by "bug"

Result: [Fix navigation bug] ✓

Key things to note:

1. `highlightMatch()` returns JSX (`<mark>` tag)
 - wraps matched portion in yellow highlight
 - only works in `.map()` since JSX is returned
2. `visibleTodos` uses METHOD CHAINING
 - `.filter().filter()`
 - first filter = tab (all/active/completed)
 - second filter = search text
 - order matters – filter first, then search
3. Clear button (×) on search input
 - only shows when search has text
 - `{search && <button>}`
 - `onClick` → `setSearch('')`
4. Empty state message is SMART
 - if searching → "No tasks match 'xyz'"
 - if not searching → "No tasks here!"

4/19/2026, 9:35:54 PM



maybank / MBBLabs

2. what is useReducer ? why we use useReducer ?
3. react lifecycle ?
4. what is redux ? what reducer will do in redux ?
5. Destructuring in react ?
6. how to manage session in react ? like which user is currently using how can we track in react ?
7. what is pure component ?
8. what is destructuring in react ? and how to do that destructuring ?
9. what is hooks ?
10. diff between class component and functional component ?
11. in react why useEffect do run infinite rendering ? and how to avoid that ?
12. diff between axios and fetch in react ?
13. diff between useContext and redux ?

4/24/2026, 7:48:52 AM



MBBLabs / Maybank Interview Questions

Q2. What is useReducer? Why we use it?

`useReducer` is a hook for managing **complex state logic**. Instead of updating state directly, you dispatch actions to a reducer function.

```
const [state, dispatch] = useReducer(reducer, initialState);
```

Simple example:

```
// Reducer function – pure function, no side effects
function reducer(state, action) {
  switch (action.type) {
    case 'INCREMENT':
      return { count: state.count + 1 };
    case 'DECREMENT':
      return { count: state.count - 1 };
    case 'RESET':
      return { count: 0 };
    case 'SET':
      return { count: action.payload };
    default:
      return state; // always return state for unknown actions
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, { count: 0 });

  return (
    <div>
      <h1>{state.count}</h1>
      <button onClick={() => dispatch({ type: 'INCREMENT' })}>+
</button>
      <button onClick={() => dispatch({ type: 'DECREMENT' })}>-
</button>
      <button onClick={() => dispatch({ type: 'RESET'
    })}>Reset</button>
      <button onClick={() => dispatch({ type: 'SET', payload: 10
    })}>Set 10</button>
    </div>
  );
}
```

```
    </div>
  );
}
```

Why use useReducer instead of useState?

useState:

```
const [name, setName] = useState('');
const [email, setEmail] = useState('');
const [age, setAge] = useState(0);
const [loading, setLoading] = useState(false);
const [error, setError] = useState(null);
// 5 separate state pieces → hard to manage ❌
```

useReducer:

```
const [state, dispatch] = useReducer(reducer, {
  name: '', email: '', age: 0,
  loading: false, error: null
}); // one state object ✅
// all logic in one reducer function ✅
// easy to test ✅
```

Use useReducer when:

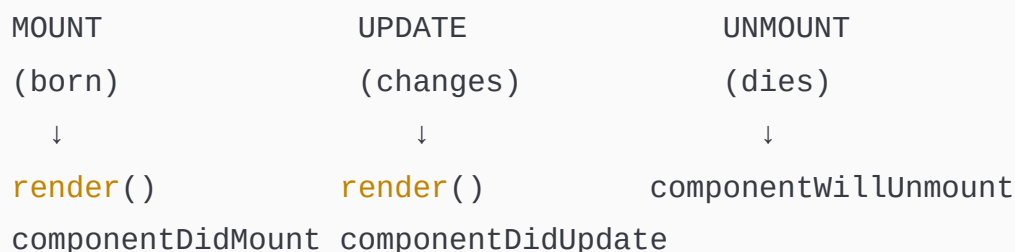
- ✅ State has multiple sub-values
- ✅ Next state depends on previous state
- ✅ Complex update logic
- ✅ Multiple actions change the same state
- ✅ State logic you want to test separately

Use useState when:

- ✅ Simple values (string, number, boolean)
- ✅ Independent pieces of state
- ✅ 1-2 ways state can change

Q3. React Lifecycle

Three phases: **Mount** → **Update** → **Unmount**



Class component:

```
class MyComponent extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
    // Step 1: initialize state
  }

  componentDidMount() {
    // Step 2: runs AFTER first render
    // Good for: API calls, subscriptions, timers
    fetch('/api/data').then(res => res.json())
      .then(data => this.setState({ data }));
  }

  componentDidUpdate(prevProps, prevState) {
    // Step 3: runs after every update
    // Good for: react to prop/state changes
    if (prevProps.userId !== this.props.userId) {
      this.fetchUser(this.props.userId); // only re-fetch if userId
      changed
    }
  }
}
```

```

componentWillUnmount() {
  // Step 4: runs before component removed
  // Good for: cleanup timers, subscriptions
  clearInterval(this.timer);
  this.subscription.unsubscribe();
}

render() {
  return <div>{this.state.count}</div>;
}
}

```

Functional component (hooks equivalent):

```

function MyComponent({ userId }) {
  const [data, setData] = useState(null);

  // componentDidMount → empty deps []
  useEffect(() => {
    fetchInitialData();
  }, []);

  // componentDidUpdate → specific deps
  useEffect(() => {
    fetchUser(userId); // re-runs when userId changes
  }, [userId]);

  // componentWillUnmount → return cleanup fn
  useEffect(() => {
    const timer = setInterval(() => tick(), 1000);
    return () => clearInterval(timer); // cleanup ✅
  }, []);

  // componentDidMount + componentDidUpdate + Unmount → all in one
  useEffect(() => {

```

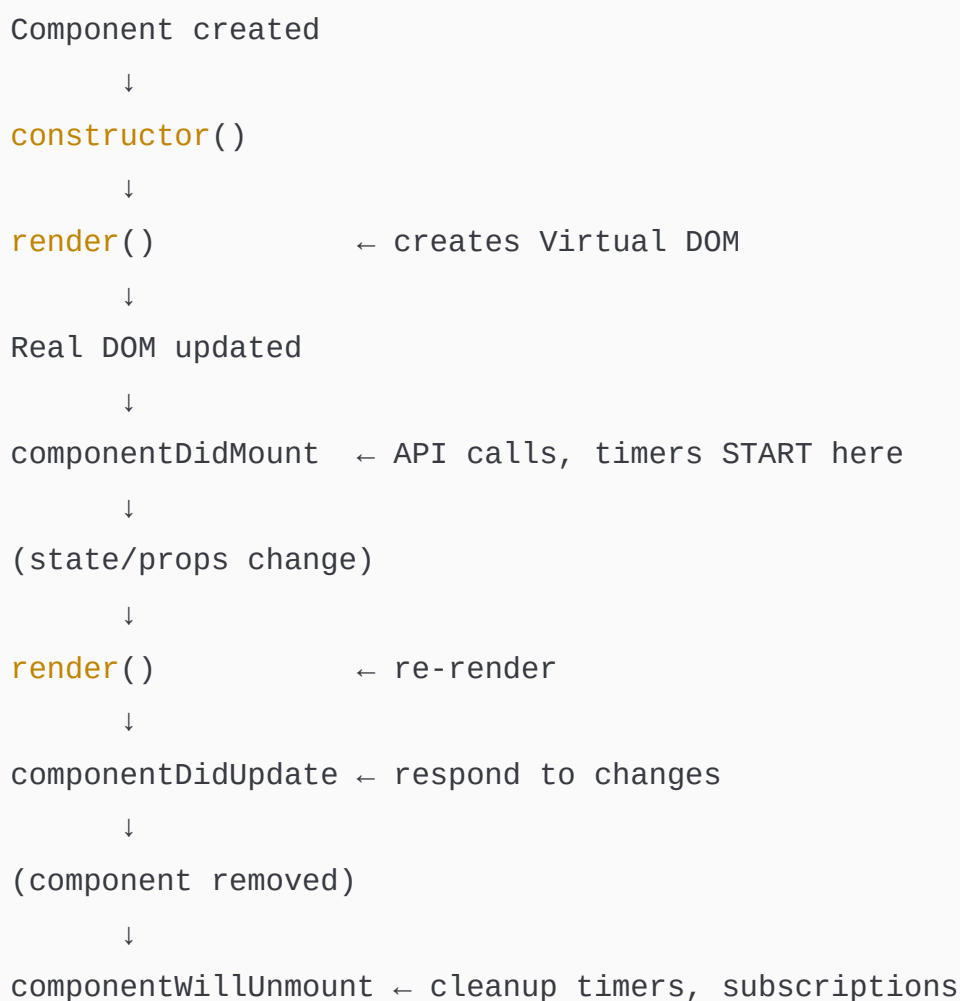
```

    const sub = subscribe(userId);
    return () => sub.unsubscribe(); // cleanup on unmount
  }, [userId]);

  return <div>{data?.name}</div>;
}

```

Lifecycle diagram:



Q4. What is Redux? What does Reducer do?

Redux = predictable global state management. One store, one source of truth.

Flow:

Component → dispatch(action) → Reducer → Store → Component

↑ _____ |

(re-render with new state)

What Reducer does:

```
// Reducer = pure function that takes current state + action  
// and returns NEW state
```

```
// Rules of Reducer:
```

```
// 1. Pure function – same input always = same output
```

```
// 2. No side effects – no API calls, no mutation
```

```
// 3. Never mutate state directly – return NEW object
```

```
// 4. Always return state for unknown actions
```

```
function cartReducer(state = { items: [], total: 0 }, action) {  
  switch (action.type) {  
  
    case 'ADD_ITEM':  
      return {  
        ...state, // copy old state  
        items: [...state.items, action.payload], // add new item  
        total: state.total + action.payload.price  
      };  
  
    case 'REMOVE_ITEM':  
      return {  
        ...state,  
        items: state.items.filter(item => item.id !==  
action.payload),  
      };  
  
    case 'CLEAR_CART':
```



```

    return { items: [], total: 0 };

    default:
    return state; // MUST return state for unknown actions
  }
}

```

Full Redux setup (Redux Toolkit):

```

// 1. Create slice
const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [], total: 0 },
  reducers: {
    addItem: (state, action) => {
state.items.push(action.payload); },
    removeItem: (state, action) => {
      state.items = state.items.filter(i => i.id !==
action.payload);
    },
  }
});

// 2. Create store
const store = configureStore({ reducer: { cart: cartSlice.reducer }
});

// 3. Use in component
function Cart() {
  const items = useSelector(state => state.cart.items);
  const dispatch = useDispatch();

  return (
    <div>
      {items.map(item => (

```

```

    <div key={item.id}>
      {item.name}
      <button onClick={() =>
        dispatch(cartSlice.actions.removeItem(item.id))
      }>Remove</button>
    </div>
  )})
</div>
);
}

```

Q7 & Q12. Destructuring in React

Destructuring = pulling values out of objects/arrays into variables.

Array destructuring (useState):

```

// Without destructuring
const stateArray = useState(0);
const count      = stateArray[0];
const setCount   = stateArray[1];

// With destructuring ✅
const [count, setCount] = useState(0);
const [name, setName]   = useState('');
const [open, setOpen]   = useState(false);

// Rename while destructuring
const [value, setValue] = useState('');

```

Object destructuring (props):

```

// Without destructuring
function Profile(props) {

```

```

    return <h1>{props.name} – {props.age}</h1>;
  }

  // With destructuring ✅
  function Profile({ name, age }) {
    return <h1>{name} – {age}</h1>;
  }

  // With default values
  function Profile({ name = 'Guest', age = 0, role = 'user' }) {
    return <h1>{name} ({role}) – {age}</h1>;
  }

  // Rename while destructuring
  function Profile({ name: fullName, age: years }) {
    return <h1>{fullName} – {years}</h1>;
  }

  // Nested destructuring
  function UserCard({ user: { name, address: { city } } }) {
    return <p>{name} from {city}</p>;
  }

  // Rest operator – get remaining props
  function Card({ title, children, ...rest }) {
    // rest = everything except title and children
    return <div {...rest}><h2>{title}</h2>{children}</div>;
  }

```

Array destructuring in map:

```

const users = [['Alice', 25], ['Bob', 30]];

users.map(([name, age]) => (

```

```
<p key={name}>{name} is {age}</p>
));
```

Object destructuring from API data:

```
const { data, loading, error } = useFetch('/api/users');

// Deep destructuring
const { user: { name, email }, token } = loginResponse;

// With rename + default
const { count: itemCount = 0, label: itemLabel = 'items' } = props;
```

Q8. How to Manage Session in React (Track Current User)

Method 1 — Context API (most common):

```
// AuthContext.jsx
const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  // Check session on app load
  useEffect(() => {
    const savedUser = localStorage.getItem('user');
    if (savedUser) {
      setUser(JSON.parse(savedUser));
    }
    setLoading(false);
  }, []);

  function login(userData, token) {
```

```

    setUser(userData);
    localStorage.setItem('user', JSON.stringify(userData));
    localStorage.setItem('token', token);
  }

  function logout() {
    setUser(null);
    localStorage.removeItem('user');
    localStorage.removeItem('token');
  }

  return (
    <AuthContext.Provider value={{ user, login, logout, loading }}>
      {children}
    </AuthContext.Provider>
  );
}

// Custom hook to use anywhere
export function useAuth() {
  return useContext(AuthContext);
}

// Wrap App
function App() {
  return (
    <AuthProvider>
      <Router>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/profile" element={
            <ProtectedRoute><Profile /></ProtectedRoute>
          } />
        </Routes>
      </Router>
    </AuthProvider>
  );
}

```

```

    </AuthProvider>
  );
}

// Use in any component
function Navbar() {
  const { user, logout } = useAuth();

  return (
    <nav>
      {user
        ? <>
          <span>Hello, {user.name}!</span>
          <button onClick={logout}>Logout</button>
        </>
        : <Link to="/login">Login</Link>
      }
    </nav>
  );
}

// Protected Route
function ProtectedRoute({ children }) {
  const { user, loading } = useAuth();
  if (loading) return <Spinner />;
  if (!user) return <Navigate to="/login" />;
  return children;
}

```

Method 2 — localStorage/sessionStorage:

```

// localStorage → persists after browser close
// sessionStorage → cleared when tab closes

// Save on login

```

```
localStorage.setItem('token', 'jwt-token-here');
localStorage.setItem('user', JSON.stringify({ id: 1, name: 'Alice'
})));

// Read anywhere
const token = localStorage.getItem('token');
const user  = JSON.parse(localStorage.getItem('user') || 'null');

// Clear on logout
localStorage.clear();
// or specific:
localStorage.removeItem('token');
```

Method 3 — Redux (large apps):

```
const authSlice = createSlice({
  name: 'auth',
  initialState: {
    user: JSON.parse(localStorage.getItem('user') || 'null'),
    token: localStorage.getItem('token') || null,
  },
  reducers: {
    loginSuccess: (state, action) => {
      state.user = action.payload.user;
      state.token = action.payload.token;
      localStorage.setItem('user',
JSON.stringify(action.payload.user));
      localStorage.setItem('token', action.payload.token);
    },
    logout: (state) => {
      state.user = null;
      state.token = null;
      localStorage.clear();
    }
  }
})
```

```
});

// Track who is logged in
function Dashboard() {
  const user = useSelector(state => state.auth.user);
  return <h1>Welcome back, {user?.name}!</h1>;
}
```

Q9. Pure Component

Re-renders **only when props or state actually change**. Same input = same output.

```
// Class – PureComponent does shallow comparison automatically
class UserCard extends React.PureComponent {
  render() {
    console.log('rendered');
    return <h1>{this.props.name}</h1>;
  }
}

// Functional – React.memo does the same
const UserCard = React.memo(({ name }) => {
  console.log('rendered');
  return <h1>{name}</h1>;
});

// Test
<UserCard name="Alice" /> // renders → "rendered"
<UserCard name="Alice" /> // SKIPS → same props ✅
<UserCard name="Bob" /> // renders → props changed

// Custom comparison function
const UserCard = React.memo(({ user }) => {
  return <h1>{user.name}</h1>;
});
```



```
}, (prevProps, nextProps) => {  
  // return true = skip render (props "equal")  
  // return false = re-render  
  return prevProps.user.id === nextProps.user.id;  
});
```

Pure → re-renders **ONLY when** props change

Non-Pure → re-renders **whenever** parent re-renders

Performance difference:

Parent renders **100** times

Non-Pure child → **100** re-renders **✗**

Pure child → **only when** props change **✓**

Q14. What are Hooks?

Hooks let functional components use React features that were only available in class components.

Rules of Hooks:

1. **Only call at** TOP LEVEL (**not** inside if/loop/**function**)
2. **Only call** inside React functions (**not** plain JS)

Built-in hooks:

```
// State  
const [count, setCount] = useState(0);  
  
// Side effects (API, timer, subscription)  
useEffect(() => {  
  fetchData();  
  return () => cleanup();  
}, [deps]);
```

```
// Context
const theme = useContext(ThemeContext);

// Complex state
const [state, dispatch] = useReducer(reducer, initial);

// Memoize value
const sorted = useMemo(() => list.sort(), [list]);

// Memoize function
const handleClick = useCallback(() => doSomething(id), [id]);

// DOM reference
const inputRef = useRef(null);
inputRef.current.focus();

// Custom Hook (reusable logic)
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch(url).then(r => r.json())
      .then(d => { setData(d); setLoading(false); });
  }, [url]);

  return { data, loading };
}

// Use anywhere
const { data, loading } = useFetch('/api/users');
```

Q15. Class Component vs Functional Component

	Class	Functional
Syntax	ES6 class	JS function
State	<code>this.state</code>	<code>useState</code>
Lifecycle	lifecycle methods	<code>useEffect</code>
<code>this</code> keyword	Required everywhere	Not needed
Boilerplate	More	Less
Performance	Slightly heavier	Lighter
Modern React	❌ Old way	✅ Preferred

// Same component – Class vs Functional

// — CLASS —

```
class Counter extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
    this.increment = this.increment.bind(this); // must bind!
  }

  increment() {
    this.setState({ count: this.state.count + 1 });
  }

  componentDidMount() {
    document.title = `Count: ${this.state.count}`;
  }

  componentDidUpdate() {
    document.title = `Count: ${this.state.count}`;
  }
}
```

```
componentWillUnmount() {
  document.title = 'App';
}

render() {
  return (
    <button onClick={this.increment}>
      {this.state.count}
    </button>
  );
}
}

// — FUNCTIONAL (same thing, much cleaner) —————
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `Count: ${count}`;
    return () => { document.title = 'App'; };
  }, [count]);

  return (
    <button onClick={() => setCount(c => c + 1)}>
      {count}
    </button>
  );
}
```

Q16. Why useEffect Causes Infinite Rendering & How to Avoid

The problem:

```
// ❌ INFINITE LOOP – no dependency array
useEffect(() => {
  setCount(count + 1); // setState → re-render → effect runs →
  setState...
});

// ❌ INFINITE LOOP – state updated inside effect with no deps
useEffect(() => {
  fetchData().then(data => setData(data)); // setData triggers re-
  render
                                     // which runs effect
  again...
});
```

Why it happens:

```
useEffect runs
  → updates state
  → state change triggers re-render
  → re-render runs useEffect again
  → updates state again
  → infinite loop ❌
```

How to fix — dependency array:

```
// ✅ Empty array [] → runs ONCE on mount only
useEffect(() => {
  fetchData().then(data => setData(data));
}, []); // never re-runs

// ✅ Specific deps → only re-runs when userId changes
useEffect(() => {
  fetchUser(userId);
}, [userId]); // only when userId changes
```

```
//  Correct deps – include everything used inside
useEffect(() => {
  fetchData(page, limit); // uses page and limit
}, [page, limit]); // list both as deps 
```

Common mistakes:

```
//  Object/array in deps – new reference every render
useEffect(() => {
  doSomething(options);
}, [options]); // options = {} → new object each render → infinite
loop!
```

```
//  Fix – use primitive values or useMemo
const stableOptions = useMemo(() => ({ page, limit }), [page,
limit]);
useEffect(() => {
  doSomething(stableOptions);
}, [stableOptions]);
```

```
//  Function in deps – new reference every render
useEffect(() => {
  fetchData();
}, [fetchData]); // fetchData recreated each render → infinite!
```

```
//  Fix – wrap function in useCallback
const fetchData = useCallback(() => {
  fetch('/api/data');
}, []); // stable reference now
```

```
useEffect(() => {
  fetchData();
}, [fetchData]); //  stable
```

Rules to avoid infinite loops:

1. Always provide dependency array []
2. Include all variables used inside effect in deps
3. Don't update state unconditionally inside effect
4. Use condition before setState inside effect
5. Memoize objects/arrays/functions used as deps

Q33. Axios vs Fetch

	fetch	axios
Built-in	✓ No install	✗ npm install
JSON parse	Manual <code>.json()</code>	Auto ✓
HTTP errors	NOT thrown (404, 500)	Auto thrown ✓
Timeout	✗ No built-in	✓ Yes
Interceptors	✗ No	✓ Yes
Cancel request	Complex	Simple
Progress	✗ No	✓ Yes
Old browsers	✗ Polyfill needed	✓ Works

```
// — FETCH —————  
  
async function getUser() {  
  const res = await fetch('/api/user');  
  
  // Must manually check status – 404 is NOT an error!  
  if (!res.ok) throw new Error('HTTP Error: ' + res.status);  
  
  // Must manually parse JSON  
  const data = await res.json();  
  return data;  
}
```

```

}

// POST with fetch
await fetch('/api/users', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ name: 'Alice' }) // must stringify
});

// — AXIOS —
async function getUser() {
  const { data } = await axios.get('/api/user');
  // auto-parses JSON ✓
  // auto-throws on 404/500 ✓
  return data;
}

// POST with axios
await axios.post('/api/users', { name: 'Alice' });
// auto-stringifies body ✓

// Axios interceptors (powerful – no fetch equivalent)
axios.interceptors.request.use(config => {
  config.headers.Authorization = `Bearer ${getToken()}`;
  return config; // auto-add token to EVERY request
});

axios.interceptors.response.use(
  res => res,
  err => {
    if (err.response.status === 401) logout();
    return Promise.reject(err);
  }
);

```



```
// Axios timeout
axios.get('/api/data', { timeout: 5000 }); // error after 5s
```

Q34. useContext vs Redux

	useContext	Redux
Setup	Minimal	More boilerplate
DevTools	✗ No	✓ Time-travel debug
Performance	All consumers re-render	Selective re-render
Async	Manual	Middleware (thunk/saga)
Middleware	✗ No	✓ Yes
Best for	Small/medium apps	Large complex apps
Bundle size	0kb (built-in)	Extra library

```
// — useContext —————
// Simple – good for theme, language, auth
const ThemeContext = createContext();

function App() {
  const [theme, setTheme] = useState('light');
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Page />
    </ThemeContext.Provider>
  );
}

function DeepChild() {
  const { theme, setTheme } = useContext(ThemeContext);
  return <button onClick={() => setTheme('dark')}>{theme}</button>;
}
```

```

// Problem: ENTIRE tree re-renders when context value changes
// Even components that don't use theme ❌

// — REDUX —————
// Each component subscribes to ONLY what it needs
function ThemeButton() {
  const theme = useSelector(state => state.theme); // only this
  const dispatch = useDispatch();
  // Only re-renders when theme changes, not other state ✅
}

function UserProfile() {
  const user = useSelector(state => state.user); // only this
  // Only re-renders when user changes ✅
}

```

Decision guide:

useContext ✅ when:

- Sharing theme, language, auth user
- Small to medium app
- Simple state (rarely changes)
- Don't want extra dependencies

Redux ✅ when:

- Large app with many features
- Multiple teams
- Complex async flows
- Need DevTools for debugging
- State changes frequently
- Need middleware (logging, analytics)

4/24/2026, 7:50:20 AM



vaisesika company UI

how to use this below json data using javascript functions

```
const response = {  
  name: "John Doe",  
  address: {  
    state: {  
      name: "California",  
      details : {  
        street: "123 Main St",  
        city: "Anytown",  
        state: "CA",  
        zip: "12345"  
      },  
    },  
  },  
}
```

like this way :-

if i call

findDetails(response, 'address.state.name')
'address.state.name'

what is EventLoop in js ? why it has been used

what are hooks ?

what is redux ?

how to write/create a customHook ?

4/28/2026, 12:55:49 PM



All Answers

Q1. findDetails — Access Nested JSON with Dot Notation String

```
const response = {
  name: "John Doe",
  address: {
    state: {
      name: "California",
      details: {
        street: "123 Main St",
        city: "Anytown",
        state: "CA",
        zip: "12345"
      }
    }
  }
};
```

```
// — Solution —————
function findDetails(obj, path) {
  // Step 1: split "address.state.name" → ["address", "state", "name"]
  const keys = path.split('.');

  // Step 2: reduce through keys, drilling into object each time
  return keys.reduce((current, key) => {
    // if current is null/undefined, return undefined (don't crash)
    if (current === null || current === undefined) return undefined;
    return current[key]; // drill one level deeper
  }, obj); // start from the root object
}
```

How it works step by step:

```
findDetails(response, 'address.state.name')
```

Step 1: split → ['address', 'state', 'name']

Step 2: reduce

start: obj = response (whole object)

key='address' → current = response['address']
= { state: { name: "California"... } }

key='state' → current = response['address']['state']
= { name: "California", details: {...} }

key='name' → current = response['address']['state']['name']
= "California"

Result: "California" 

All possible calls:

```
findDetails(response, 'name')
```

// → "John Doe"

```
findDetails(response, 'address.state.name')
```

// → "California"

```
findDetails(response, 'address.state.details.city')
```

// → "Anytown"

```
findDetails(response, 'address.state.details.zip')
```

// → "12345"

```
findDetails(response, 'address.state.details.street')
```

// → "123 Main St"

```
findDetails(response, 'address.state.details')
// → { street: "123 Main St", city: "Anytown", state: "CA", zip:
      "12345" }

findDetails(response, 'address.state.notExist')
// → undefined (doesn't crash) ✓
```

Extended version — with fallback & array support:

```
function findDetails(obj, path, fallback = undefined) {
  if (!obj || !path) return fallback;

  const keys = path.split('.');

  const result = keys.reduce((current, key) => {
    if (current === null || current === undefined) return undefined;

    // Support array index like 'users.0.name'
    if (Array.isArray(current) && !isNaN(key)) {
      return current[parseInt(key)];
    }

    return current[key];
  }, obj);

  // return fallback if result is undefined/null
  return result ?? fallback;
}

// Usage with fallback
findDetails(response, 'address.state.name', 'Unknown')
// → "California"

findDetails(response, 'address.state.phone', 'N/A')
// → "N/A" (fallback used since path doesn't exist)
```

```
// Array support
const data = {
  users: [
    { name: 'Alice' },
    { name: 'Bob' }
  ]
};
findDetails(data, 'users.0.name') // → "Alice"
findDetails(data, 'users.1.name') // → "Bob"
```

Set value at path too (bonus):

```
function setDetails(obj, path, value) {
  const keys = path.split('.');
  const lastKey = keys.pop(); // remove last key

  // navigate to parent
  const parent = keys.reduce((current, key) => {
    if (!current[key]) current[key] = {}; // create if missing
    return current[key];
  }, obj);

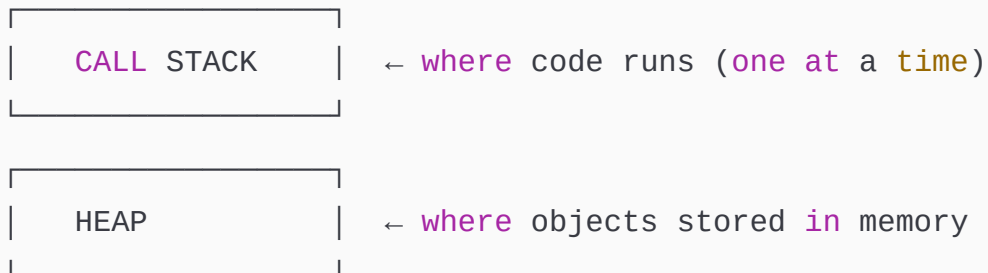
  parent[lastKey] = value; // set value
  return obj;
}

setDetails(response, 'address.state.name', 'Texas');
console.log(response.address.state.name); // "Texas"
```

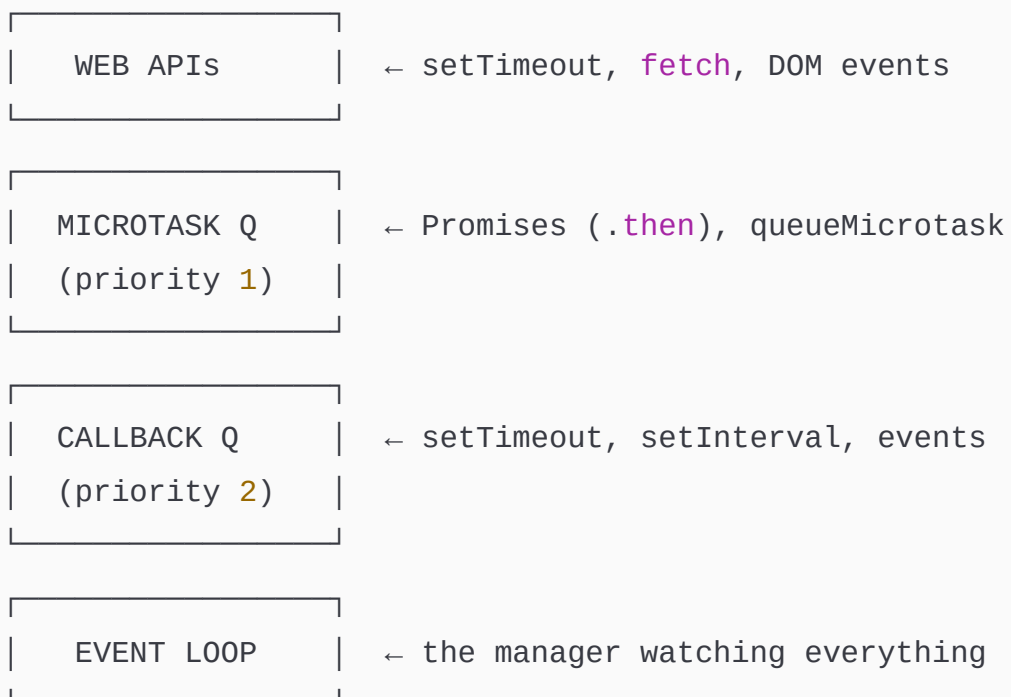
Q2. What is Event Loop in JS? Why is it used?

JavaScript is **single-threaded** — it can only do one thing at a time. The Event Loop is how JS handles async operations without blocking.

JS Engine has:



Browser/Node gives us:



Event Loop's job:

```
while (true) {  
  if (callStack is empty) {  
    if (microtaskQueue has tasks) {  
      push ALL microtasks to callStack // Promises first  
    } else if (callbackQueue has tasks) {  
      push ONE task to callStack // setTimeout next  
    }  
  }  
}
```



```
}  
}
```

Example — execution order:

```
console.log('1');           // sync → Call Stack → runs now  
  
setTimeout(() => {  
  console.log('4');         // async → Web API → Callback Queue  
}, 0);  
  
Promise.resolve()  
  .then(() => console.log('3')); // async → Microtask Queue  
  
console.log('2');           // sync → Call Stack → runs now  
  
// Output: 1 → 2 → 3 → 4  
  
// Why?  
// 1, 2 → synchronous, run immediately  
// Stack empties  
// Event Loop checks Microtask Q first → '3' runs  
// Event Loop checks Callback Q next → '4' runs
```

Visual flow:

```
console.log('1')    → Call Stack → runs → popped  
setTimeout(cb,0)    → Web API handles timer  
                   → after 0ms → cb goes to Callback Q  
Promise.then(cb2)   → Microtask Q  
console.log('2')    → Call Stack → runs → popped  
  
STACK IS EMPTY → Event Loop kicks in
```

```
Check Microtask Q → cb2 there → push to stack → '3' runs
Check Callback Q → cb there → push to stack → '4' runs
```

Why Event Loop is needed:

Without Event Loop:

```
fetch('/api/data') ← takes 2 seconds
// Browser FREEZES for 2 seconds
// User can't click, scroll, type ❌
```

With Event Loop:

```
fetch('/api/data') ← handed to Web API
// Browser continues other work
// User can interact freely ✅
// When fetch completes → callback queued
// Event Loop picks it up when stack empty
```

Q3. What are Hooks?

Hooks let **functional components** use React features (state, lifecycle, context) that only class components had before.

Before Hooks (React 15):

```
Need state?           → use class component
Need lifecycle?       → use class component
Need context?         → use class component
Simple display only? → use functional component
```

After Hooks (React 16.8+):

```
Everything in functional components ✅
Simpler, less code, reusable logic
```

All built-in hooks:

```
// 1. useState – local state
const [count, setCount] = useState(0);

// 2. useEffect – side effects (API, timer, subscription)
useEffect(() => {
  fetchData();
  return () => cleanup(); // runs on unmount
}, [dependency]);

// 3. useContext – read from context
const user = useContext(UserContext);

// 4. useReducer – complex state logic
const [state, dispatch] = useReducer(reducer, initialState);

// 5. useRef – DOM access / persist value without re-render
const inputRef = useRef(null);
inputRef.current.focus();

// 6. useMemo – cache expensive calculation
const sorted = useMemo(() => bigList.sort(), [bigList]);

// 7. useCallback – cache function reference
const handleClick = useCallback(() => {
  doSomething(id);
}, [id]);

// 8. useEffect – like useEffect but fires before paint
useLayoutEffect(() => {
  measureDOM();
}, []);

// 9. useId – generate unique IDs
const id = useId(); // "r1", "r2"...
```

```
// 10. useTransition – mark updates as non-urgent
const [isPending, startTransition] = useTransition();
startTransition(() => setResults(filter(query)));
```

Rules of Hooks:

```
// ❌ WRONG – inside condition
if (isLoggedIn) {
  const [user, setUser] = useState(null); // BREAKS!
}
```

```
// ❌ WRONG – inside loop
for (let i = 0; i < 3; i++) {
  const [val, setVal] = useState(0); // BREAKS!
}
```

```
// ❌ WRONG – inside regular JS function
function normalFn() {
  const [x, setX] = useState(0); // BREAKS!
}
```

```
// ✅ CORRECT – top level of component
function MyComponent() {
  const [user, setUser] = useState(null); // ✅
  const [count, setCount] = useState(0); // ✅
}
```

```
// ✅ CORRECT – inside custom hook
function useMyHook() {
  const [x, setX] = useState(0); // ✅
}
```

Q4. What is Redux?

Redux = **predictable global state management**. All state in ONE store. Only changes via actions through reducers.

Problem:

ComponentA (has **data**)

↓ props

ComponentB

↓ props

ComponentC

↓ props

ComponentD (needs **data**) ← prop drilling ❌

Solution (Redux):

ComponentA → dispatch(action) → Store

ComponentD → useSelector → Store

No prop drilling! ✅

Core flow:

User action

↓

dispatch({ type: 'ADD', payload: item })

↓

Reducer(currentState, action) → newState

↓

Store updates

↓

useSelector gets **new** state

↓

Component re-renders ✅

Full example:

```

// 1. Slice (Redux Toolkit)
import { createSlice } from '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: state => { state.value += 1; },
    decrement: state => { state.value -= 1; },
    addAmount: (state, action) => { state.value += action.payload; }
  }
});

export const { increment, decrement, addAmount } =
counterSlice.actions;
export default counterSlice.reducer;

// 2. Store
import { configureStore } from '@reduxjs/toolkit';
const store = configureStore({
  reducer: { counter: counterSlice.reducer }
});

// 3. Provide
<Provider store={store}><App /></Provider>

// 4. Use in component
function Counter() {
  const count = useSelector(state => state.counter.value);
  const dispatch = useDispatch();

  return (
    <>
      <h1>{count}</h1>
      <button onClick={() => dispatch(increment())}></button>
    </>
  );
}

```

```

    <button onClick={() => dispatch(decrement())}>-</button>
    <button onClick={() => dispatch(addAmount(5))}>+5</button>
  </>
);
}

```

Q5. How to Write a Custom Hook

A custom hook is a **reusable function** that starts with `use` and can call other hooks inside.

Why **custom** hooks?

Same logic needed **in** multiple components

→ extract **into custom** hook

→ reuse anywhere 

Examples:

useFetch → reuse API fetching logic

useLocalStorage → reuse localStorage logic

useDebounce → reuse debounce logic

useOnline → track online/offline status

Custom Hook 1 — useFetch (API fetching):

```

// hooks/useFetch.js
import { useState, useEffect } from 'react';

function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    // reset state on url change
    setLoading(true);

```

```

    setError(null);

    const controller = new AbortController(); // cancel on unmount

    async function fetchData() {
      try {
        const res = await fetch(url, { signal: controller.signal });
        if (!res.ok) throw new Error('HTTP Error: ' + res.status);
        const json = await res.json();
        setData(json);
      } catch (err) {
        if (err.name !== 'AbortError') { // ignore cancel errors
          setError(err.message);
        }
      } finally {
        setLoading(false);
      }
    }

    fetchData();

    return () => controller.abort(); // cleanup on unmount
  }, [url]); // re-fetch when url changes

  return { data, loading, error };
}

// Use in ANY component
function UserList() {
  const { data, loading, error } = useFetch('/api/users');

  if (loading) return <p>Loading...</p>;
  if (error)   return <p>Error: {error}</p>;
  return <ul>{data.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}

```



```
function PostList() {  
  const { data, loading } = useFetch('/api/posts'); // reuse!   
}
```

Custom Hook 2 — useLocalStorage:

```
function useLocalStorage(key, initialValue) {  
  // Initialize from localStorage or use initialValue  
  const [value, setValue] = useState(() => {  
    try {  
      const saved = localStorage.getItem(key);  
      return saved !== null ? JSON.parse(saved) : initialValue;  
    } catch {  
      return initialValue;  
    }  
  });  
  
  // Wrap setValue to also save to localStorage  
  function setStoredValue(newValue) {  
    try {  
      setValue(newValue);  
      localStorage.setItem(key, JSON.stringify(newValue));  
    } catch (err) {  
      console.error('localStorage error:', err);  
    }  
  }  
  
  return [value, setStoredValue];  
}  
  
// Usage – works exactly like useState  
// but value PERSISTS after page refresh   
function Settings() {  
  const [theme, setTheme] = useLocalStorage('theme', 'light');
```

```

const [lang, setLang] = useLocalStorage('lang', 'en');

return (
  <>
    <button onClick={() => setTheme('dark')}>Dark Mode</button>
    <p>Theme: {theme}</p> {/* survives refresh! */}
  </>
);
}

```

Custom Hook 3 — useDebounce:

```

function useDebounce(value, delay = 500) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => {
      setDebouncedValue(value); // update only after delay
    }, delay);

    return () => clearTimeout(timer); // cancel if value changes again
  }, [value, delay]);

  return debouncedValue;
}

```

// Usage

```

function SearchBar() {
  const [query, setQuery] = useState('');
  const debouncedQuery = useDebounce(query, 500);
  const { data } = useFetch(`/api/search?q=${debouncedQuery}`);

```

// API only called 500ms after user stops typing 

```

return (
  <input
    value={query}
    onChange={e => setQuery(e.target.value)}
    placeholder="Search..."
  />
);
}

```

Custom Hook 4 — useOnline (network status):

```

function useOnline() {
  const [isOnline, setIsOnline] = useState(navigator.onLine);

  useEffect(() => {
    function handleOnline() { setIsOnline(true); }
    function handleOffline() { setIsOnline(false); }

    window.addEventListener('online', handleOnline);
    window.addEventListener('offline', handleOffline);

    return () => {
      window.removeEventListener('online', handleOnline);
      window.removeEventListener('offline', handleOffline);
    };
  }, []);

  return isOnline;
}

// Usage
function App() {
  const isOnline = useOnline();
  return (
    <div>

```

```

    {!isOnline && (
      <div style={{ background: 'red', color: 'white', padding: 8
    }}>
      You are offline!
    </div>
  )}
  <MainContent />
</div>
);
}

```

Custom Hook 5 — usePrevious:

```

function usePrevious(value) {
  const ref = useRef();

  useEffect(() => {
    ref.current = value; // store current value after render
  }, [value]);

  return ref.current; // returns value from PREVIOUS render
}

// Usage
function Counter() {
  const [count, setCount] = useState(0);
  const prevCount = usePrevious(count);

  return (
    <div>
      <p>Current: {count}</p>
      <p>Previous: {prevCount}</p>
      <button onClick={() => setCount(c => c + 1)}></button>
    </div>
  )
}

```

```
);  
}
```

Rules for custom hooks:

- ✓ Name must start with 'use' (useMyHook)
 - React identifies it as a hook
 - Rules of hooks apply inside it
- ✓ Can call other hooks inside
 - useState, useEffect, useRef etc.
- ✓ Must follow hook rules
 - Top level only
 - Not inside conditions/loops
- ✓ Returns whatever is useful
 - value, array, object, function
 - whatever the consumer needs
- ✓ Each component gets its OWN state
 - hooks are not shared between components
 - only the LOGIC is shared

4/28/2026, 12:56:57 PM



Wipro

what is React ? is it library or framework ?

what is reconciliation ?

who will do reconciliation and diffing and update in UI ?

javascript vs typescript ?

what are the ways we can optimize performance in react UI

diff between useEffect and useLayoutEffect ?

what is redux ?

what is propsdrilling ?

what are Hooks you know ?

why we use useMemo and useCallback ?

only expensive calculation will store in useMemo or what ?

and what if value change in useMemo ? what happen ?

how to handle 100 of form fields in a form in your UI page in react ?

what is Lazy Import in react ?

5/8/2026, 5:53:53 PM



Wipro Interview Questions

Q1. What is React? Library or Framework?

React is a Library — not a framework.

Library vs Framework:

Library:

You **call** the library code

You **are in** control

Does **ONE specific** thing well
React → **only** handles UI (**View** layer)

Framework:

Framework calls your code

Framework **is in** control

Does EVERYTHING (routing, HTTP, forms, etc.)

Angular → routing + HTTP + forms + testing built-**in**

React **is** a library because:

- ✓ **Only** handles UI rendering
- ✓ You choose your own router (React Router)
- ✓ You choose your own state (Redux/Zustand)
- ✓ You choose your own HTTP (axios/**fetch**)
- ✓ You **are in** control **of** the structure

```
// React only does this – render UI
function App() {
  return <h1>Hello World</h1>;
}

// For everything else you pick your own tools:
// Routing      → React Router
// State        → Redux / Zustand / Context
// Forms        → React Hook Form / Formik
// HTTP         → axios / fetch / React Query
// Styling      → Tailwind / styled-components
```

Q2. What is Reconciliation?

The process React uses to **compare old Virtual DOM vs new Virtual DOM** and figure out the minimum changes needed to update the real DOM.

State changes



React creates **NEW** Virtual DOM tree



Compares **NEW** vs **OLD** Virtual DOM ← this **is** Reconciliation



Finds differences (diffing)



Updates **ONLY** changed parts **in** **real** DOM



Browser repaints

Diffing rules:

```
// Rule 1: Different element type → rebuild entire subtree
// Old:           New:
<div>           → <span>           // different type
  <Child />      <Child />      // Child unmounts + remounts!
</div>          </span>

// Rule 2: Same element type → update only changed attributes
// Old:           New:
<div className="old" /> → <div className="new" />
// React just updates className, keeps DOM node ✅

// Rule 3: Lists → use key to match elements
// Without key: React re-renders entire list
// With key:      React matches by key, updates only changed items
{items.map(item => <li key={item.id}>{item.name}</li>)}
```

Q3. Who Does Reconciliation, Diffing, and UI Update?

Three separate parts work together:

1. REACT (reconciler)

- Creates Virtual DOM
- Does diffing (finds what changed)
- React Fiber engine **handles** this
- **Async**, interruptible, prioritized

2. REACT DOM (renderer)

- Takes diff results **from** React
- Applies changes **to** real browser DOM
- appendChild, removeChild, etc.
- Sync, cannot be interrupted

3. BROWSER

- Takes updated real DOM
- Layout calculation
- Painting pixels **on** screen

// React does reconciliation (finding changes)

```
import React from 'react';
```

// ReactDOM applies changes to real DOM

```
import ReactDOM from 'react-dom/client';
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));
```

```
root.render(<App />);
```

// Flow:

// React → "I found these 3 nodes changed"

// ReactDOM → "Ok, I'll update those 3 DOM nodes"

// Browser → "Ok, I'll repaint those areas"

Q4. JavaScript vs TypeScript

	JavaScript	TypeScript
Type system	Dynamic (runtime)	Static (compile time)
Type errors	Found at runtime	Found while writing 
Learning curve	Easier	Steeper
Autocomplete	Basic	Excellent 
Large projects	Hard to maintain	Much easier 
Browser runs	Directly	Needs compilation
Optional	—	Types are optional

```
// — JAVASCRIPT —————  
  
function addNumbers(a, b) {  
  return a + b;  
}  
  
addNumbers(5, 10);           // 15   
addNumbers("5", 10);        // "510" ← bug! caught only at runtime   
addNumbers(5);               // NaN ← bug! no warning   
  
// — TYPESCRIPT —————  
  
function addNumbers(a: number, b: number): number {  
  return a + b;  
}  
  
addNumbers(5, 10);           // 15   
addNumbers("5", 10);        // ERROR at compile time  caught early!  
addNumbers(5);               // ERROR: missing argument   
  
// TypeScript in React  
interface UserProps {
```

```

    name: string;
    age: number;
    email?: string; // optional
  }

  function UserCard({ name, age, email }: UserProps) {
    return <div>{name} – {age}</div>;
  }

  <UserCard name="Alice" age={25} />           // ✅
  <UserCard name="Alice" age="25" />           // ERROR ✅
  <UserCard age={25} />                         // ERROR: missing name
  ✅

```

Q5. Ways to Optimize Performance in React UI

Level 1 — Prevent Unnecessary Re-renders

```

// React.memo – skip re-render if props same
const Card = React.memo(({ title }) => {
  return <div>{title}</div>;
});

// useCallback – stable function reference
const handleClick = useCallback(() => {
  doSomething(id);
}, [id]);

// useMemo – cache expensive value
const filtered = useMemo(() => {
  return bigList.filter(item => item.active);
}, [bigList]);

```

Level 2 — Load Less Code

```
// Lazy loading + code splitting
const Dashboard = lazy(() => import('./Dashboard'));
const Settings = lazy(() => import('./Settings'));

function App() {
  return (
    <Suspense fallback={<Spinner />}>
      <Routes>
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/settings" element={<Settings />} />
      </Routes>
    </Suspense>
  );
}
// Initial bundle: 2MB → 200KB 
```

Level 3 — Handle Large Lists

```
// Virtualization – only render visible items
import { FixedSizeList } from 'react-window';

function BigList({ items }) {
  return (
    <FixedSizeList
      height={600}
      itemCount={items.length} // even 100,000 items
      itemSize={50}
    >
      {({ index, style }) => (
        <div style={style}>{items[index].name}</div>
      )}
    </FixedSizeList>
  );
}
```

```
}  
// 100,000 items → only ~12 DOM nodes at any time 
```

Level 4 — Other Optimizations

```
// 1. Debounce search input  
const debouncedSearch = useCallback(  
  debounce(q => fetchResults(q), 500), []  
);  
  
// 2. useTransition – non-urgent updates  
const [isPending, startTransition] = useTransition();  
startTransition(() => {  
  setSearchResults(filter(query)); // won't block typing  
});  
  
// 3. Lazy load images  
  
  
// 4. Avoid new objects/arrays in JSX  
// ✗ new array every render  
<Component options={['a', 'b', 'c']} />  
  
//  stable reference  
const OPTIONS = ['a', 'b', 'c']; // outside component  
<Component options={OPTIONS} />  
  
// 5. Key prop for lists – helps reconciliation  
{items.map(item => <Item key={item.id} {...item} />)}
```

Q6. useEffect vs useLayoutEffect

	useEffect	useLayoutEffect
When it runs	AFTER browser paints	BEFORE browser paints
Blocking	Non-blocking (async)	Blocking (sync)
Use case	API calls, subscriptions	DOM measurements, animations
Performance	Better (default choice)	Can cause visual delay

```
function App() {
  const [width, setWidth] = useState(0);
  const divRef = useRef(null);

  // useEffect – runs AFTER paint
  // User might see old value for one frame (flash)
  useEffect(() => {
    console.log('3. useEffect runs (after paint)');
    // by now browser already painted old value
  }, []);

  // useLayoutEffect – runs BEFORE paint
  // No flash – measurement happens before user sees anything
  useLayoutEffect(() => {
    console.log('2. useLayoutEffect runs (before paint)');
    const w = divRef.current.getBoundingClientRect().width;
    setWidth(w); // safe! user won't see jump
  }, []);

  console.log('1. render');

  return <div ref={divRef}>Width: {width}</div>;
}

// Console output:
// 1. render
// 2. useLayoutEffect runs (before paint)
```

```
// [browser paints here]
// 3. useEffect runs (after paint)
```

When to use which:

useEffect (default — use 99% of time):

- ✓ API calls
- ✓ Setting up subscriptions
- ✓ Event listeners
- ✓ Timers

useLayoutEffect (only when needed):

- ✓ Measuring DOM elements (width, height, position)
- ✓ Animations that depend on DOM position
- ✓ Preventing visual flicker from DOM changes
- ✓ Tooltip positioning

Q7. What is Redux?

Already covered in detail in previous answers. Quick summary:

Redux = global state management

One store → all state lives here

Only changes through: dispatch(action) → reducer → new state

When to use:

- ✓ State shared across many components
- ✓ Large apps, multiple teams
- ✓ Complex state logic
- ✓ Need DevTools / time-travel debugging

Q8. What is Props Drilling?

Passing props through **many intermediate components** that don't need it, just to reach a deeply nested child.

```
// ❌ Props Drilling
function App() {
  const user = { name: "Alice", role: "admin" };
  return <Page user={user} />;
}

function Page({ user }) {
  return <Layout user={user} />; // Page doesn't use user!
}

function Layout({ user }) {
  return <Sidebar user={user} />; // Layout doesn't use user!
}

function Sidebar({ user }) {
  return <UserCard user={user} />; // Sidebar doesn't use user!
}

function UserCard({ user }) {
  return <h1>{user.name}</h1>; // ONLY this needs user
}

// user passed through 4 levels, 3 don't even need it ❌

// ✅ Solution 1: Context API
const UserContext = createContext();

function App() {
  const user = { name: "Alice", role: "admin" };
  return (
    <UserContext.Provider value={user}>
      <Page /> {/* no props needed! */}
    </UserContext.Provider>
  );
}
```



```

    );
}

function UserCard() {
  const user = useContext(UserContext); // direct access ✅
  return <h1>{user.name}</h1>;
}

// ✅ Solution 2: Redux
function UserCard() {
  const user = useSelector(state => state.user); // direct access ✅
  return <h1>{user.name}</h1>;
}

```

Q9. Hooks You Know

```

// Built-in React Hooks:

// 1. useState – local state
const [count, setCount] = useState(0);

// 2. useEffect – side effects
useEffect(() => { fetchData(); }, [dep]);

// 3. useContext – consume context
const theme = useContext(ThemeContext);

// 4. useReducer – complex state
const [state, dispatch] = useReducer(reducer, initial);

// 5. useRef – DOM access / persist value
const ref = useRef(null);

// 6. useMemo – cache expensive value

```

```

const result = useMemo(() => expensiveCalc(a, b), [a, b]);

// 7. useCallback – cache function
const fn = useCallback(() => doSomething(id), [id]);

// 8. useEffect – DOM measurement before paint
useLayoutEffect(() => { measureDOM(); }, []);

// 9. useTransition – mark non-urgent updates
const [isPending, startTransition] = useTransition();

// 10. useDeferredValue – defer a value update
const deferred = useDeferredValue(query);

// 11. useId – generate unique IDs
const id = useId();

// Custom hooks:
const { data, loading } = useFetch('/api/users');
const [val, setVal]      = useLocalStorage('key', default);
const isOnline           = useOnline();
const debouncedVal      = useDebounce(value, 500);

```

Q10. Why We Use useMemo and useCallback?

Both prevent **unnecessary recalculation/recreation** on every render.

```

function App() {
  const [count, setCount] = useState(0);
  const [text, setText]   = useState('');

  // ❌ Without useMemo
  // This runs on EVERY render, even when count changes
  // but bigList didn't change

```

```

const filtered = bigList.filter(item => item.active);

// ✅ With useMemo
// Only recalculates when bigList changes
const filtered = useMemo(() => {
  return bigList.filter(item => item.active);
}, [bigList]);

// ❌ Without useCallback
// New function created every render
// React.memo on Child is useless – onClick always "new"
const handleClick = () => doSomething(count);

// ✅ With useCallback
// Same function reference until count changes
const handleClick = useCallback(() => {
  doSomething(count);
}, [count]);

return <Child onClick={handleClick} items={filtered} />;
}

const Child = React.memo(({ onClick, items }) => {
  // Only re-renders if onClick or items actually changed
  console.log('Child rendered');
  return <button onClick={onClick}>Click</button>;
});

```

Q11. Only Expensive Calculations in useMemo? What if Value Changes?

Not only expensive — but yes, expensive is the main use case.

```
//  Good use – expensive calculation
const sortedAndFiltered = useMemo(() => {
  return bigArray
    .filter(item => item.active)
    .sort((a, b) => a.name.localeCompare(b.name))
    .map(item => ({ ...item, label: item.name.toUpperCase() }));
}, [bigArray]); // 10,000 items – expensive!

//  Good use – stable object reference for child component
const config = useMemo(() => ({
  page, limit, sortBy
}), [page, limit, sortBy]);
// Without this, new object every render → child always re-renders

//  Bad use – not expensive, not needed
const double = useMemo(() => count * 2, [count]);
// Simple math – useMemo overhead costs MORE than the calc itself!
// Just write: const double = count * 2;
```

What happens when value/deps change:

```
const result = useMemo(() => {
  console.log('calculating...');
  return expensiveCalc(a, b);
}, [a, b]);

// First render:
//   a=1, b=2 → runs expensiveCalc(1,2) → stores result
//   deps stored: [1, 2]

// Re-render, a and b SAME (a=1, b=2):
//   compares [1,2] vs [1,2] → same!
//   → SKIPS calculation → returns cached result 
```

```
// Re-render, a CHANGES (a=5, b=2):
//   compares [5,2] vs [1,2] → different!
//   → RUNS expensiveCalc(5,2) → stores new result
//   → returns new result
//   deps updated: [5, 2]

// Summary:
// deps same → return cache (no recalc)
// deps changed → recalculate → update cache → return new value
```

Q12. How to Handle 100 Form Fields in React?

Never create 100 `useState` calls. Use these approaches:

Approach 1 — Single State Object

```
function HugeForm() {
  // ONE state object for all fields
  const [formData, setFormData] = useState({
    firstName: '', lastName: '', email: '',
    phone: '', address: '', city: '',
    // ... 94 more fields
  });

  // ONE generic handler for ALL fields
  function handleChange(e) {
    const { name, value, type, checked } = e.target;
    setFormData(prev => ({
      ...prev,
      [name]: type === 'checkbox' ? checked : value
      // [name] = computed property = matches input's name attribute
    }));
  }
}
```

```

// Each input just needs name= matching the state key
return (
  <form>
    <input name="firstName" value={formData.firstName} onChange=
{handleChange} />
    <input name="lastName" value={formData.lastName} onChange=
{handleChange} />
    <input name="email" value={formData.email} onChange=
{handleChange} />
    {/* All 100 fields use same handleChange ✅ */}
  </form>
);
}

```

Approach 2 — React Hook Form (best for large forms)

```

import { useForm } from 'react-hook-form';

function HugeForm() {
  const {
    register,
    handleSubmit,
    watch,
    formState: { errors }
  } = useForm();

  function onSubmit(data) {
    console.log(data); // all 100 fields in one object
  }

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      {/* register connects input to form – no onChange needed! */}
      <input
        {...register('firstName', {

```

```

        required: 'First name required',
        minLength: { value: 2, message: 'Too short' }
    }}}
  />
  {errors.firstName && <p>{errors.firstName.message}</p>}

  <input
    {...register('email', {
      required: 'Email required',
      pattern: { value: /^\\S+@\\S+$/, message: 'Invalid email' }
    })}
  />

  <button type="submit">Submit</button>
</form>
);
}
// Benefits:
// ✓ No re-render on every keystroke (uncontrolled)
// ✓ Built-in validation
// ✓ Less code
// ✓ Great performance even with 100+ fields

```

Approach 3 — Dynamic fields from config

```

// Define fields as config array – easy to add/remove fields
const FIELD_CONFIG = [
  { name: 'firstName', label: 'First Name', type: 'text', required:
true },
  { name: 'lastName', label: 'Last Name', type: 'text', required:
true },
  { name: 'email', label: 'Email', type: 'email', required:
true },
  { name: 'phone', label: 'Phone', type: 'tel', required:
false },

```

```

    // ... define all 100 fields here
];

function DynamicForm() {
  const [formData, setFormData] = useState(
    // Initialize all fields to empty string automatically
    FIELD_CONFIG.reduce((acc, field) => {
      acc[field.name] = '';
      return acc;
    }, {})
  );

  function handleChange(e) {
    setFormData(prev => ({ ...prev, [e.target.name]: e.target.value
})));
  }

  return (
    <form>
      {FIELD_CONFIG.map(field => (
        <div key={field.name}>
          <label>{field.label}</label>
          <input
            type={field.type}
            name={field.name}
            value={formData[field.name]}
            onChange={handleChange}
            required={field.required}
          />
        </div>
      ))}
    </form>
  );
}

```



```
// Adding new field = add one line to FIELD_CONFIG ✅  
// No component changes needed ✅
```

Q13. What is Lazy Import in React?

Load component code **only when it's needed**, not on initial page load.

```
// ❌ Normal import – ALL code downloaded at startup  
import Dashboard from './Dashboard';  
import Settings from './Settings';  
import Reports from './Reports';  
// 3MB bundle downloaded even if user never visits these pages!  
  
// ✅ Lazy import – code downloaded only when component renders  
import { lazy, Suspense } from 'react';  
  
const Dashboard = lazy(() => import('./Dashboard'));  
const Settings = lazy(() => import('./Settings'));  
const Reports = lazy(() => import('./Reports'));
```

How it works:

```
import('./Dashboard')
```

↓

Webpack/Vite creates separate chunk file:

```
main.js          → 200KB (core app)  
Dashboard.chunk → 400KB (only when /dashboard visited)  
Settings.chunk  → 150KB (only when /settings visited)  
Reports.chunk   → 250KB (only when /reports visited)
```

User visits homepage:

Downloads: main.js (200KB) ✅ fast!

Dashboard/Settings/Reports NOT downloaded yet

User navigates to /dashboard:

Downloads: Dashboard.chunk (400KB) ← only now

Shows Suspense fallback while downloading

Then renders Dashboard

Full usage:

```
const Home      = lazy(() => import('./pages/Home'));
const Dashboard = lazy(() => import('./pages/Dashboard'));
const Settings  = lazy(() => import('./pages/Settings'));

// Suspense REQUIRED – shows fallback while chunk downloads
function App() {
  return (
    <BrowserRouter>
      <Suspense fallback={
        <div style={{ textAlign: 'center', padding: 50 }}>
          Loading...
        </div>
      }>
        <Routes>
          <Route path="/"          element={<Home />} />
          <Route path="/dashboard" element={<Dashboard />} />
          <Route path="/settings"  element={<Settings />} />
        </Routes>
      </Suspense>
    </BrowserRouter>
  );
}

// Lazy load on interaction (button click, hover)
const HeavyModal = lazy(() => import('./HeavyModal'));

function App() {
  const [show, setShow] = useState(false);
```

```

return (
  <>
    <button onClick={() => setShow(true)}>Open</button>
    {show && (
      <Suspense fallback={<p>Loading modal...</p>}>
        <HeavyModal />
        {/* Modal code only downloads when button clicked ✅ */}
      </Suspense>
    )}
  </>
);
}

```

Summary table:

Normal import → downloaded at app start (eager)
 Lazy import → downloaded when component first renders (lazy)

Use lazy for:

- ✅ Routes/pages (most common)
- ✅ Modals, drawers
- ✅ Admin panels
- ✅ Rarely used features
- ✅ Heavy third-party components (charts, maps, editors)

Don't lazy load:

- ❌ Components used immediately on landing page
- ❌ Very small components (overhead not worth it)
- ❌ Components needed for initial render

5/8/2026, 5:55:21 PM



-mphasis

What is event loop in js

- ☐ What is micro task
- ☐ and what is macro task
- ☐ What is closure and write closure example
- ☐ Synchronization vs asynchronization in js

5/9/2026, 6:28:20 PM

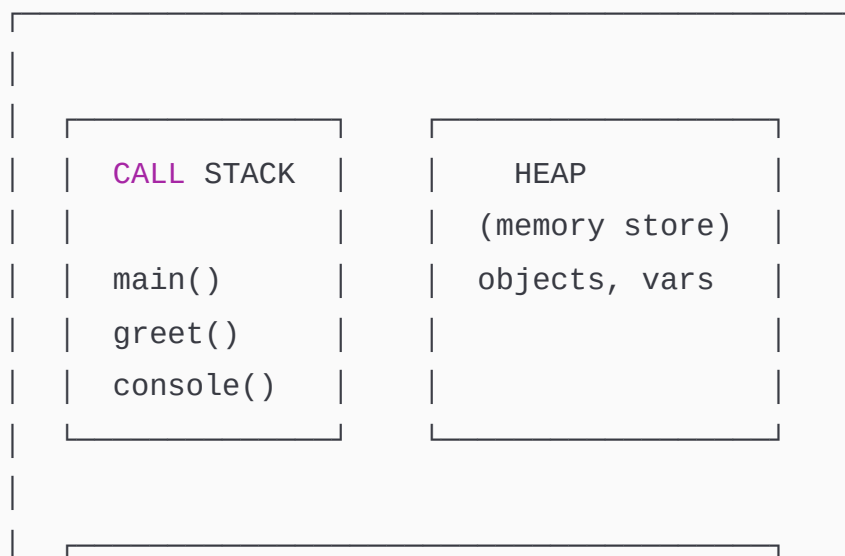


Mphasis Interview Questions

Q1. What is Event Loop in JS?

JavaScript is **single-threaded** — one thing at a time. The Event Loop is the mechanism that allows JS to handle async operations without blocking.

JS Engine components:





Event Loop algorithm:

```
while (true) {
  // 1. Run ALL synchronous code first
  if (callStack is not empty) {
    execute top of callStack
  }
  // 2. When stack empty, drain ALL microtasks
  else if (microtaskQueue is not empty) {
    push ALL microtasks to callStack one by one
  }
  // 3. Only then, take ONE macrotask
  else if (callbackQueue is not empty) {
    push ONE macrotask to callStack
    // then go back to step 2 (check microtasks again!)
  }
}
```

```
}  
}
```

Full example:

```
console.log('1 - sync');  
  
setTimeout(() => {  
  console.log('5 - macrotask (setTimeout)');  
}, 0);  
  
Promise.resolve()  
  .then(() => console.log('3 - microtask 1'))  
  .then(() => console.log('4 - microtask 2'));  
  
console.log('2 - sync');  
  
// Output:  
// 1 - sync          ← sync, runs immediately  
// 2 - sync          ← sync, runs immediately  
// 3 - microtask 1   ← microtask, runs before setTimeout  
// 4 - microtask 2   ← microtask chained, runs next  
// 5 - macrotask     ← macrotask, runs last
```

Step by step:

```
Step 1: console.log('1') → Call Stack → runs → "1 - sync"  
Step 2: setTimeout(cb) → Web API handles 0ms timer  
Step 3: Promise.then(cb) → Microtask Queue  
Step 4: console.log('2') → Call Stack → runs → "2 - sync"  
  
STACK IS EMPTY → Event Loop checks:  
Step 5: Microtask Q has tasks!  
→ push cb1 → runs → "3 - microtask 1"
```

```
→ .then chains another microtask
→ push cb2 → runs → "4 - microtask 2"
Microtask Q empty now
```

Step 6: Callback Q has setTimeout cb

```
→ push to stack → runs → "5 - macrotask"
```

Q2. What is Microtask?

Tasks that run **immediately after current code finishes** but **before any macrotasks**. Highest priority async tasks.

```
// What goes into Microtask Queue:
Promise.resolve().then(callback) // Promise handlers
async/await                      // under the hood uses Promises
queueMicrotask(callback)         // explicit microtask
MutationObserver                 // DOM change observer
```

Examples:

```
// Example 1 – Promise
console.log('start');

Promise.resolve('microtask result')
  .then(val => console.log(val));

console.log('end');

// Output:
// start
// end
// microtask result ← runs after sync, before macrotasks
```

```

// Example 2 – async/await (same as Promise under the hood)
async function fetchData() {
  console.log('async start');
  const result = await Promise.resolve('data');
  // everything AFTER await goes to Microtask Queue
  console.log('after await:', result);
}

console.log('before call');
fetchData();
console.log('after call');

// Output:
// before call
// async start
// after call
// after await: data ← microtask

// Example 3 – Microtasks drain completely before macrotasks
Promise.resolve()
  .then(() => {
    console.log('microtask 1');
    // adding another microtask from inside microtask
    Promise.resolve().then(() => console.log('microtask 3'));
  })
  .then(() => console.log('microtask 2'));

setTimeout(() => console.log('macrotask'), 0);

// Output:
// microtask 1
// microtask 2
// microtask 3 ← ALL microtasks done (including newly added)
// macrotask ← only THEN macrotask runs

```


Key rule:

After **EVERY** macrotask → drain **ALL** microtasks completely
Before **ANY** next macrotask → **check** microtask queue again

Microtask adds another microtask?
→ that also runs before next macrotask
→ microtask queue must be **COMPLETELY empty first**

Q3. What is Macrotask?

Tasks scheduled to run in **future event loop iterations**. Lower priority than microtasks.

```
// What goes into Callback Queue (Macrotask Queue):
setTimeout(callback, delay)      // timer
setInterval(callback, delay)     // repeated timer
setImmediate(callback)           // Node.js only
requestAnimationFrame(callback) // before next repaint
DOM events (click, keydown, etc.) // user interactions
fetch callbacks                  // after network response
MessageChannel                   // web workers
```

Examples:

```
// Example 1 – Multiple macrotasks
setTimeout(() => console.log('macro 1'), 0);
setTimeout(() => console.log('macro 2'), 0);
setTimeout(() => console.log('macro 3'), 0);

// Each setTimeout = separate macrotask
// Event Loop picks ONE at a time:
// Loop 1: macro 1 runs → check microtasks (none) → next loop
// Loop 2: macro 2 runs → check microtasks (none) → next loop
// Loop 3: macro 3 runs
```

```
// Output: macro 1 → macro 2 → macro 3
```

```
// Example 2 – setInterval
```

```
let count = 0;  
const id = setInterval(() => {  
  count++;  
  console.log('tick', count);  
  if (count === 3) clearInterval(id);  
}, 1000);
```

```
// Output (one per second):
```

```
// tick 1 (after 1s)
```

```
// tick 2 (after 2s)
```

```
// tick 3 (after 3s) ← cleared, stops
```

```
// Example 3 – DOM events are macrotasks
```

```
button.addEventListener('click', () => {  
  console.log('click event'); // macrotask when clicked  
  Promise.resolve().then(() => {  
    console.log('microtask inside click'); // microtask  
  });  
  setTimeout(() => {  
    console.log('setTimeout inside click'); // next macrotask  
  }, 0);  
});
```

```
// User clicks:
```

```
// click event ← macrotask (click)
```

```
// microtask inside click ← microtask (drained before next macro)
```

```
// setTimeout inside click ← next macrotask
```

Micro vs Macro — Side by Side

MICROTASK	MACROTASK
<code>Promise .then()</code>	<code>setTimeout</code>
<code>Promise .catch()</code>	<code>setInterval</code>
<code>async/await</code>	<code>setImmediate (Node)</code>
<code>queueMicrotask()</code>	<code>requestAnimationFrame</code>
<code>MutationObserver</code>	DOM events (click etc.)
	fetch (I/O callbacks)
Priority: HIGH 	Priority: LOW
Runs: ALL before macros	Runs: ONE per loop tick
Queue: drains completely	Queue: one at a time

```
// The golden order:
console.log('1. sync'); // sync

setTimeout(() => console.log('5. macro')); // macrotask

Promise.resolve()
  .then(() => console.log('3. micro 1')) // microtask
  .then(() => console.log('4. micro 2')); // microtask (chained)

console.log('2. sync'); // sync

// Always: sync → micro → macro
// Output: 1 → 2 → 3 → 4 → 5
```

Q4. What is Closure?

A closure is when an **inner function remembers variables from its outer function's scope** even after the outer function has finished executing.

```
function outer() {  
  let count = 0;           // outer variable  
  
  function inner() {       // inner function  
    count++;               // remembers 'count' from outer scope  
    console.log(count);  
  }  
  
  return inner;            // return inner WITHOUT calling it  
}  
  
const counter = outer(); // outer() finishes, but count stays alive!  
counter(); // 1   ← inner still has access to count  
counter(); // 2   ← count persists in closure  
counter(); // 3   ← count persists again
```

Why count doesn't disappear:

```
Normally: function finishes → variables garbage collected  
Closure:  inner function holds REFERENCE to outer variables  
          → garbage collector sees they're still referenced  
          → keeps them alive in memory ✅
```

Closure Examples:

Example 1 — Counter factory:

```
function makeCounter(startFrom = 0) {  
  let count = startFrom; // private variable  
  
  return {  
    // ...  
  }  
}
```

```

    increment: () => ++count,
    decrement: () => --count,
    reset:      () => { count = startFrom; return count; },
    getCount:   () => count
  };
}

const counter1 = makeCounter(0);
const counter2 = makeCounter(100); // separate closure!

counter1.increment(); // 1
counter1.increment(); // 2
counter2.increment(); // 101

console.log(counter1.getCount()); // 2    (independent)
console.log(counter2.getCount()); // 101  (independent)

// Each call to makeCounter creates its OWN closure
// counter1 and counter2 don't share state ✅

```

Example 2 — Data privacy (module pattern):

```

function createBankAccount(initialBalance) {
  let balance = initialBalance; // PRIVATE – can't access directly!
  let transactions = [];

  function log(type, amount) {
    transactions.push({ type, amount, balance });
  }

  return {
    deposit(amount) {
      if (amount <= 0) throw new Error('Amount must be positive');
      balance += amount;
      log('deposit', amount);
    }
  };
}

```

```

    return balance;
  },
  withdraw(amount) {
    if (amount > balance) throw new Error('Insufficient funds');
    balance -= amount;
    log('withdraw', amount);
    return balance;
  },
  getBalance() { return balance; },
  getHistory() { return [...transactions]; } // copy, not
reference
};
}

const account = createBankAccount(1000);
account.deposit(500);    // 1500
account.withdraw(200);  // 1300

console.log(account.getBalance()); // 1300
console.log(account.balance);      // undefined – PRIVATE! ✅
// Can't directly modify balance – must use deposit/withdraw

```

Example 3 — Closure in loops (classic interview question):

```

// ❌ WRONG – classic bug
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 1000);
}
// Output: 3, 3, 3 (not 0, 1, 2!)
// Why? var is function-scoped, ONE i shared by all callbacks
// By the time callbacks run, loop done, i = 3

// ✅ FIX 1 – closure with IIFE
for (var i = 0; i < 3; i++) {
  (function(j) {                // IIFE creates new scope

```

```

    setTimeout(() => console.log(j), 1000);
  })(i);           // capture current i as j
}
// Output: 0, 1, 2 ✅

// ✅ FIX 2 – use let (block-scoped, new i each iteration)
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 1000);
}
// Output: 0, 1, 2 ✅

```

Example 4 — Closure in React:

```

// React hooks use closures internally
function Counter() {
  const [count, setCount] = useState(0);

  // handleClick CLOSES OVER count
  // captures current value of count
  const handleClick = () => {
    console.log('current count:', count); // closure!
    setCount(count + 1);
  };

  // Stale closure bug:
  useEffect(() => {
    const timer = setInterval(() => {
      console.log(count); // always logs initial value! stale
      // closure ❌
    }, 1000);
    return () => clearInterval(timer);
  }, []); // [] = count never updated in closure

  // Fix stale closure:
  useEffect(() => {

```

```

    const timer = setInterval(() => {
      setCount(prev => prev + 1); // use functional update ✅
    }, 1000);
    return () => clearInterval(timer);
  }, []);

  return <button onClick={handleClick}>{count}</button>;
}

```

Real world uses of closure:

```

// 1. Memoization / caching
function memoize(fn) {
  const cache = {}; // closure keeps cache alive
  return function(...args) {
    const key = JSON.stringify(args);
    if (cache[key]) {
      console.log('from cache');
      return cache[key];
    }
    cache[key] = fn(...args);
    return cache[key];
  };
}

const expensiveCalc = memoize((n) => {
  // heavy calculation
  return n * n;
});

expensiveCalc(5); // calculates
expensiveCalc(5); // from cache ✅

// 2. Partial application / currying
function multiply(a) {

```



```

    return function(b) { // closes over a
      return a * b;
    };
  }
  const double = multiply(2);
  const triple = multiply(3);
  double(5); // 10
  triple(5); // 15

  // 3. Event handlers with data
  function attachHandler(element, data) {
    element.addEventListener('click', function() {
      console.log(data); // closes over data
    });
  }

```

Q5. Synchronous vs Asynchronous in JS

Synchronous

One task at a time. Next line **waits** for current line to finish.

```

// Sync – blocks everything until done
function syncTask() {
  console.log('Task 1 start');
  // imagine this takes 3 seconds (heavy computation)
  for (let i = 0; i < 1_000_000_000; i++) {} // blocks!
  console.log('Task 1 end');
}

console.log('Before');
syncTask(); // blocks here for 3 seconds
console.log('After'); // only runs after syncTask finishes

// Output:

```

```
// Before
// Task 1 start
// (3 seconds pass, UI frozen)
// Task 1 end
// After
```

Asynchronous

Start a task, **don't wait**, continue. Come back when done.

```
// Async – doesn't block
console.log('Before');

setTimeout(() => {
  console.log('Task done'); // runs after 3 seconds
}, 3000);

console.log('After'); // doesn't wait for setTimeout!

// Output:
// Before
// After           ← runs immediately, doesn't wait
// (3 seconds later)
// Task done
```

4 Ways to Write Async Code:

Way 1 — Callbacks:

```
function fetchUser(id, callback) {
  setTimeout(() => {
    const user = { id, name: 'Alice' }; // simulated API
    callback(null, user);                // (error, result)
  }, 1000);
}
```

```

}

fetchUser(1, (err, user) => {
  if (err) { console.error(err); return; }
  console.log(user); // { id: 1, name: 'Alice' }
});

console.log('this runs before callback!');

// Problem: Callback Hell
fetchUser(1, (err, user) => {
  fetchPosts(user.id, (err, posts) => {
    fetchComments(posts[0].id, (err, comments) => {
      // deeply nested, hard to read ❌
    });
  });
});

```

Way 2 — Promises:

```

function fetchUser(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (!id) reject(new Error('No ID'));
      else resolve({ id, name: 'Alice' });
    }, 1000);
  });
}

fetchUser(1)
  .then(user => { console.log(user); return fetchPosts(user.id); })
  .then(posts => { console.log(posts); })
  .catch(err => console.error(err))
  .finally(() => console.log('done'));

```

```
// Cleaner chaining  no callback hell
```

Way 3 — async/await (most readable):

```
async function loadData() {
  try {
    const user      = await fetchUser(1);
    const posts     = await fetchPosts(user.id);
    const comments = await fetchComments(posts[0].id);
    console.log(comments);
  } catch (err) {
    console.error(err);
  }
}

loadData();
console.log('runs immediately, doesnt wait for loadData');
```

Way 4 — Parallel async (faster):

```
async function loadAll() {
  //  Sequential – 3 seconds total
  const users  = await fetchUsers(); // 1 sec
  const posts  = await fetchPosts(); // 1 sec
  const photos = await fetchPhotos(); // 1 sec

  //  Parallel – 1 second total
  const [users, posts, photos] = await Promise.all([
    fetchUsers(), // all start at same time
    fetchPosts(),
    fetchPhotos()
  ]);
}
```

```
//  Fail safe – get all results even if some fail
const results = await Promise.allSettled([
  fetchUsers(),
  fetchPosts(),
  fetchPhotos()
]);
results.forEach(r => {
  if (r.status === 'fulfilled') console.log(r.value);
  else console.error(r.reason);
});
}
```

Sync vs Async comparison:

SYNCHRONOUS:

Task1  (3s)
 Task2  (1s)
 Task3  (0.5s)

Total: 4.5 seconds  blocking

ASYNCHRONOUS:

Task1  (3s)
 Task2  (1s) ← starts at same time
 Task3  (0.5s) ← starts at same time

Total: 3 seconds  non-blocking

In browser:

Sync heavy task → UI freezes, user can't interact ❌

Async heavy task → UI stays responsive ✅

Simple rule to remember:

Synchronous = cook **one** dish → serve → cook next
(**one at a time, in order**)

Asynchronous = **start all** dishes → serve **each when** ready
(concurrent, non-blocking)

Event Loop = the kitchen manager watching **all** dishes
and coordinating **when each is** served 🚀

5/9/2026, 6:29:35 PM



Bell Company (UI)

what is the output of this -

```
console.log("Try ", test);
```

```
const test = "JS is cool";
```

does const getting hoisted ?

what is call apply bind in js

Write the js code for this -

To find duplicate use SKU,

wayBill,carrier are same that object is duplicate, and sum of the quantity

```
const obj = [
```

```
[
```

```
{
```

```
"qty":1,
```

```
"SKU":"SXW12X",
```

```
"item": "Laptop",
```

```
"price": "1000",
```

```
"wayBill":123,
```

```
"carrier":"fedx"
```

```
},
```

```
{
```

```
"qty":2,
```

```
"SKU":"SXW13X",
```

```
"item": "Mouse",
```

```
"price": "200",
```

```
"wayBill":124,
```

```
"carrier":"fedx"
```

```
}
```

```
],
```

```
[
```

```
{
```

```
"qty":1,
"SKU":"SXW12X",
"item": "Laptop",
"price": "100",
"wayBill":123,
"carrier":"fedx"
},
{
"qty":3,
"SKU":"SXW12X",
"item": "Laptop",
"price": "100",
"wayBill":123,
"carrier":"dtdc"
},
{
"qty":1,
"SKU":"SXW15X",
"item": "Keyboard",
"price": "200",
"wayBill":125,
"carrier":"fedx"
}
]
];
```

Output -

```
/[{
"qty":2,
"SKU":"SXW12X",
"item": "Laptop",
"price": "100",
"wayBill":123,
"carrier":"fedx"
},
{
```



```
"wayBill":124,  
"carrier":"fedx"  
},  
{  
"wayBill":125,  
"carrier":"fedx"  
}  
];/
```

5/13/2026, 4:25:10 PM



Bell Company UI Questions

Q1. Output of this code?

```
console.log("Try ", test);  
const test = "JS is cool";
```

Output:

```
ReferenceError: Cannot access 'test' before initialization
```

Why?

Q2. Does const get Hoisted?

Yes — but differently from `var`.

ALL declarations are **hoisted** (moved to top).
But HOW they are hoisted **is** different:

`var` → hoisted + initialized **as undefined** (can access before)

<code>let</code>	→ hoisted + NOT initialized	(TDZ - cannot access)
<code>const</code>	→ hoisted + NOT initialized	(TDZ - cannot access)

Temporal Dead Zone (TDZ):

```
// What you write:
console.log("Try ", test); // line 1
const test = "JS is cool"; // line 2

// What JS engine sees after hoisting:
// const test; ← hoisted to top BUT not initialized (TDZ starts here)
// console.log("Try ", test); ← test exists but in TDZ → ERROR!
// test = "JS is cool"; ← TDZ ends here (initialization)
```

Comparison of all 3:

```
// — var —————
console.log(a); // undefined (no error, hoisted + initialized)
var a = "hello";
console.log(a); // "hello"

// — let —————
console.log(b); // ReferenceError: Cannot access before init
let b = "hello";

// — const —————
console.log(c); // ReferenceError: Cannot access before init
const c = "hello";

// — function declaration —————
greet(); // "Hello!" ✅ fully hoisted
function greet() { console.log("Hello!"); }
```

```
// — function expression —————  
sayBye();      // ReferenceError (const) or TypeError (var)  
const sayBye = function() { console.log("Bye!"); };
```

TDZ visualized:

const test;	← TDZ STARTS
TEMPORAL DEAD ZONE	
test exists in memory but cannot be accessed → ReferenceError	
test = "JS is cool";	← TDZ ENDS
console.log(test);	← safe now ✓

```
// Correct order:  
const test = "JS is cool"; // declare first  
console.log("Try ", test); // then use → "Try JS is cool" ✓
```

Q3. call, apply, bind in JS

All 3 are used to **set the value of** `this` explicitly.

The problem they solve:

```
const person = { name: "Alice" };  
  
function greet(greeting, punctuation) {  
  return `${greeting}, ${this.name}${punctuation}`;  
}  
  
greet("Hello", "!");
```

```
// this = window/undefined → "Hello, undefined!" ❌
```

```
// We need to tell greet: use person as 'this'
```

```
// That's what call/apply/bind do ✅
```

call — invoke immediately, args comma separated

```
function greet(greeting, punctuation) {
  return `${greeting}, ${this.name}${punctuation}`;
}

const person = { name: "Alice" };

// call(thisArg, arg1, arg2, ...)
const result = greet.call(person, "Hello", "!");
console.log(result); // "Hello, Alice!"

// Real world example
const user1 = { name: "Alice", age: 25 };
const user2 = { name: "Bob", age: 30 };

function introduce(city, country) {
  return `I am ${this.name}, ${this.age}, from ${city}, ${country}`;
}

introduce.call(user1, "Chennai", "India"); // "I am Alice, 25, from Chennai, India"
introduce.call(user2, "New York", "USA"); // "I am Bob, 30, from New York, USA"
```

apply — invoke immediately, args as ARRAY

```
function greet(greeting, punctuation) {
  return `${greeting}, ${this.name}${punctuation}`;
}
```

```

}

const person = { name: "Alice" };

// apply(thisArg, [arg1, arg2, ...])
const result = greet.apply(person, ["Hello", "!"]);
console.log(result); // "Hello, Alice!"

// When to use apply – when args are already in an array
const numbers = [5, 3, 8, 1, 9, 2];

// Find max using Math.max with spread
Math.max(...numbers); // modern way

// Old way – apply passes array as individual args
Math.max.apply(null, numbers); // 9

// Merge arrays
const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];
Array.prototype.push.apply(arr1, arr2);
console.log(arr1); // [1, 2, 3, 4, 5, 6]

```

bind — returns NEW function, doesn't call immediately

```

function greet(greeting, punctuation) {
  return `${greeting}, ${this.name}${punctuation}`;
}

const person = { name: "Alice" };

// bind(thisArg, arg1, arg2) → returns new function
const boundGreet = greet.bind(person, "Hello");
// NOT called yet – just creates new function with 'this' fixed

```

```

console.log(boundGreet("!")); // "Hello, Alice!"
console.log(boundGreet("?")); // "Hello, Alice?"

// Real world – React class components
class Button extends React.Component {
  constructor(props) {
    super(props);
    // bind in constructor so 'this' works in handler
    this.handleClick = this.handleClick.bind(this);
  }

  handleClick() {
    console.log(this.state); // 'this' = component ✅
  }
}

// Partial application with bind
function multiply(a, b) { return a * b; }

const double = multiply.bind(null, 2); // fix first arg as 2
const triple = multiply.bind(null, 3); // fix first arg as 3

double(5); // 10
double(8); // 16
triple(5); // 15

```

call vs apply vs bind — Summary

	call	apply	bind
Calls	Immediately	Immediately	Returns new fn
Args	Comma separated	Array	Comma separated

Use	Borrow methods	Array of args	Preset this/args

Memory trick:

call → Comma separated

apply → Array

bind → returns Bound function (call later)

```
function show(a, b) { console.log(this.name, a, b); }
const obj = { name: "Alice" };

show.call(obj, 1, 2); // Alice 1 2 (immediate, comma)
show.apply(obj, [1, 2]); // Alice 1 2 (immediate, array)
const fn = show.bind(obj, 1, 2); fn(); // Alice 1 2 (later)
```

Q4. Find Duplicates & Sum Quantity

Understanding the problem:

Duplicate = same SKU + same wayBill + same carrier

When duplicates found → merge them, SUM the qty

Non-duplicates → keep as is

Input: nested array of arrays → flatten first

Then group by key (SKU + wayBill + carrier)

Then merge duplicates

Step by step solution:

```
const obj = [
  [
```

```

    { qty:1, SKU:"SXW12X", item:"Laptop", price:"1000",
wayBill:123, carrier:"fedx" },
    { qty:2, SKU:"SXW13X", item:"Mouse", price:"200",
wayBill:124, carrier:"fedx" }
  ],
  [
    { qty:1, SKU:"SXW12X", item:"Laptop", price:"100",
wayBill:123, carrier:"fedx" },
    { qty:3, SKU:"SXW12X", item:"Laptop", price:"100",
wayBill:123, carrier:"dtdc" },
    { qty:1, SKU:"SXW15X", item:"Keyboard", price:"200",
wayBill:125, carrier:"fedx" }
  ]
];

```

```
function findDuplicatesAndSumQty(nestedArray) {
```

```

  // Step 1: Flatten nested array of arrays into single array
  const flat = nestedArray.flat();
  console.log("Flattened:", flat);
  /*
  [
    { qty:1, SKU:"SXW12X", wayBill:123, carrier:"fedx" }, ←
duplicate
    { qty:2, SKU:"SXW13X", wayBill:124, carrier:"fedx" }, ← unique
    { qty:1, SKU:"SXW12X", wayBill:123, carrier:"fedx" }, ←
duplicate
    { qty:3, SKU:"SXW12X", wayBill:123, carrier:"dtdc" }, ← unique
(diff carrier)
    { qty:1, SKU:"SXW15X", wayBill:125, carrier:"fedx" }, ← unique
  ]
  */

  // Step 2: Group by composite key (SKU + wayBill + carrier)
  const grouped = {};

```



```

flat.forEach(item => {
  // Create unique key from 3 fields
  const key = `${item.SKU}_${item.wayBill}_${item.carrier}`;
  // e.g. "SXW12X_123_fedx"

  if (!grouped[key]) {
    // First time seeing this key → store the full item
    grouped[key] = { ...item }; // copy so we don't mutate
original
  } else {
    // Already seen this key → it's a DUPLICATE → sum qty
    grouped[key].qty += item.qty;
  }
});

console.log("Grouped:", grouped);
/*
{
  "SXW12X_123_fedx": { qty:2, SKU:"SXW12X", item:"Laptop",
price:"100", wayBill:123, carrier:"fedx" },
  "SXW13X_124_fedx": { qty:2, SKU:"SXW13X", item:"Mouse",
price:"200", wayBill:124, carrier:"fedx" },
  "SXW12X_123_dtdc": { qty:3, SKU:"SXW12X", item:"Laptop",
price:"100", wayBill:123, carrier:"dtdc" },
  "SXW15X_125_fedx": { qty:1, SKU:"SXW15X",
item:"Keyboard",price:"200",wayBill:125, carrier:"fedx" },
}
*/

// Step 3: Convert grouped object back to array
const result = Object.values(grouped);

return result;
}

```

```
const result = findDuplicatesAndSumQty(obj);
console.log("Result:", JSON.stringify(result, null, 2));
```

Output:

```
[
  {
    qty: 2,           // ← 1 + 1 summed ✅
    SKU: "SXW12X",
    item: "Laptop",
    price: "100",
    wayBill: 123,
    carrier: "fedx"
  },
  {
    qty: 2,           // ← not duplicate, original qty kept
    SKU: "SXW13X",
    item: "Mouse",
    price: "200",
    wayBill: 124,
    carrier: "fedx"
  },
  {
    qty: 3,           // ← different carrier (dt dc), not duplicate
    SKU: "SXW12X",
    item: "Laptop",
    price: "100",
    wayBill: 123,
    carrier: "dt dc"
  },
  {
    qty: 1,
    SKU: "SXW15X",
    item: "Keyboard",
```

```

    price: "200",
    wayBill: 125,
    carrier: "fedx"
  }
]

```

Let me also find **ONLY** the duplicates (items that appeared more than once):

```

function findOnlyDuplicates(nestedArray) {
  const flat = nestedArray.flat();
  const grouped = {};
  const countMap = {}; // track how many times each key appeared

  flat.forEach(item => {
    const key = `${item.SKU}_${item.wayBill}_${item.carrier}`;

    countMap[key] = (countMap[key] || 0) + 1;

    if (!grouped[key]) {
      grouped[key] = { ...item };
    } else {
      grouped[key].qty += item.qty; // sum qty
    }
  });

  // Filter: only return items that appeared MORE than once
  return Object.entries(grouped)
    .filter(([key]) => countMap[key] > 1)
    .map([, value]) => value;
}

const duplicatesOnly = findOnlyDuplicates(obj);
console.log(duplicatesOnly);
// Only SXW12X_123_fedx had duplicates

```

```
// [{ qty: 2, SKU: "SXW12X", item: "Laptop", wayBill: 123, carrier: "fedx" }]
```

Tracing through the logic:

Input flat array:

Item 1: SKU=SXW12X, wayBill=123, carrier=fedx, qty=1

Item 2: SKU=SXW13X, wayBill=124, carrier=fedx, qty=2

Item 3: SKU=SXW12X, wayBill=123, carrier=fedx, qty=1 ← same as

Item 1!

Item 4: SKU=SXW12X, wayBill=123, carrier=dtcd, qty=3 ← diff carrier

Item 5: SKU=SXW15X, wayBill=125, carrier=fedx, qty=1

Processing:

Item 1 key = "SXW12X_123_fedx" → not in grouped → add {qty:1}

Item 2 key = "SXW13X_124_fedx" → not in grouped → add {qty:2}

Item 3 key = "SXW12X_123_fedx" → ALREADY in grouped → qty += 1 → {qty:2} ✓

Item 4 key = "SXW12X_123_dtdc" → not in grouped → add {qty:3}

Item 5 key = "SXW15X_125_fedx" → not in grouped → add {qty:1}

Result: 4 items, Item 1+3 merged with qty summed ✓

5/13/2026, 4:26:01 PM



a5econsulting - UI

what is reconciliation in react ?

what is props drilling ? how to avoid it ?

what is interceptor in react ?

we have a scenario like earlier we didnt had authorization token in backend in all 100 apis but now in latest it has added how we can update the same authorization token in all the axios call for backend api we can update it.

write this in js or react (try in both ways)

Given a string s consisting only of '(' and ')',

return the number of valid matched parentheses pairs.

```
console.log(countValidPairs("()()()")); // Output: 3
```

```
console.log(countValidPairs(")(")); // Output: 0
```

5/19/2026, 8:42:55 AM



a5econsulting UI Questions

Q1. What is Reconciliation in React?

Already covered in detail — quick summary:

State/Props change



React creates **NEW** Virtual DOM



Compares **NEW** vs **OLD** Virtual DOM ← Reconciliation



Finds differences (Diffing)

↓
Updates **ONLY** changed parts **in Real DOM**
↓
Browser repaints

```
function App() {  
  const [count, setCount] = useState(0);  
  return (  
    <div>  
      <h1>Title</h1>          {/* never changes → never touches DOM */}  
      <p>{count}</p>           {/* changes → only THIS updates in DOM */}  
      <button onClick={() => setCount(count + 1)}></button>  
    </div>  
  );  
}
```

// Without reconciliation → entire page re-renders on every click
// With reconciliation → only <p> text node updates 

Q2. Props Drilling — What & How to Avoid

What is it:

```
// ❌ Props Drilling – passing through components that DON'T need it  
function App() {  
  const [user] = useState({ name: "Alice", role: "admin" });  
  return <Page user={user} />;  
}  
  
function Page({ user }) {  
  return <Layout user={user} />; // Page doesn't use user!  
}
```

```
function Layout({ user }) {
  return <Sidebar user={user} />; // Layout doesn't use user!
}

function Sidebar({ user }) {
  return <Profile user={user} />; // Sidebar doesn't use user!
}

function Profile({ user }) {
  return <h1>{user.name}</h1>; // ONLY this needs it
}

// user passed through 4 levels, 3 don't need it ❌
```

How to avoid — 3 solutions:

Solution 1 — Context API:

```
// Create context
const UserContext = createContext(null);

// Provide at top level
function App() {
  const [user] = useState({ name: "Alice", role: "admin" });
  return (
    <UserContext.Provider value={user}>
      <Page /> { /* no props needed! */ }
    </UserContext.Provider>
  );
}

// Consume ANYWHERE without drilling
function Profile() {
  const user = useContext(UserContext); // direct access ✅
  return <h1>{user.name}</h1>;
}
```

```
}  
// Page, Layout, Sidebar don't need to pass anything 
```

Solution 2 — Redux:

```
// Store holds state globally  
function Profile() {  
  const user = useSelector(state => state.user); // direct   
  return <h1>{user.name}</h1>;  
}
```

Solution 3 — Component Composition:

```
// Pass components instead of data  
function App() {  
  const user = { name: "Alice" };  
  return (  
    <Layout>  
      <Profile user={user} /> {/* directly pass to where needed */}  
    </Layout>  
  );  
}  
  
function Layout({ children }) {  
  return <div className="layout">{children}</div>;  
  // Layout doesn't need to know about user at all   
}
```

Q3. What is Interceptor in React?

An interceptor is code that **runs before every request or after every response** automatically — without changing individual API calls.

Without Interceptor:

Every API call manually adds token:

```
axios.get('/api/users', { headers: { Authorization: token } })
axios.get('/api/products', { headers: { Authorization: token } })
axios.post('/api/orders', { headers: { Authorization: token } })
// 100 APIs × manual header = nightmare ❌
```

With Interceptor:

Set once → applies to ALL 100 APIs automatically ✅

```
axios.interceptors.request.use(config => {
  config.headers.Authorization = token;
  return config;
});
```

Q4. Adding Auth Token to All 100 Axios Calls

The Scenario:

Before: 100 APIs with no auth token

After: Backend now requires Authorization header on all 100 APIs

Goal: Update in ONE place, not touch all 100 calls

Solution 1 — Axios Interceptor (Best Way)

```
// api/axiosInstance.js
import axios from 'axios';

// Create ONE axios instance used everywhere
const axiosInstance = axios.create({
  baseURL: 'https://api.yourbackend.com',
  timeout: 10000,
});
```

```
// — REQUEST INTERCEPTOR —  
  
// Runs BEFORE every request is sent  
axiosInstance.interceptors.request.use(  
  (config) => {  
    // Get token from wherever you store it  
    const token = localStorage.getItem('authToken');  
    //           or sessionStorage.getItem('authToken')  
    //           or from Redux store  
    //           or from cookie  
  
    if (token) {  
      // Add to ALL requests automatically  
      config.headers.Authorization = `Bearer ${token}`;  
    }  
  
    console.log('Request:', config.method, config.url);  
    return config; // MUST return config  
  },  
  (error) => {  
    // Handle request error  
    return Promise.reject(error);  
  }  
);
```

```
// — RESPONSE INTERCEPTOR —  
  
// Runs AFTER every response is received  
axiosInstance.interceptors.response.use(  
  (response) => {  
    // Any 2xx response comes here  
    return response; // MUST return response  
  },  
  (error) => {  
    // Any non-2xx response comes here  
    if (error.response?.status === 401) {  
      // Token expired or invalid  
    }  
  }  
);
```

```

    console.log('Unauthorized! Redirecting to login...');
    localStorage.removeItem('authToken');
    window.location.href = '/login'; // redirect to login
  }

  if (error.response?.status === 403) {
    console.log('Forbidden! No permission.');
```

```

  }

  if (error.response?.status === 500) {
    console.log('Server error!');
```

```

  }

  return Promise.reject(error);
}
);

export default axiosInstance;
```

Use this instance everywhere:

```

// Before – each file imported axios directly
import axios from 'axios'; // ❌ no interceptor

// After – all files use axiosInstance
import axiosInstance from './api/axiosInstance'; // ✅ has
interceptor

// All 100 APIs just work – token added automatically
function getUsers() {
  return axiosInstance.get('/users'); // token auto-added ✅
}

function getProducts() {
  return axiosInstance.get('/products'); // token auto-added ✅
}
```

```
}

function createOrder(data) {
  return axiosInstance.post('/orders', data); // token auto-added ✅
}

// Changed axiosInstance.js → all 100 APIs updated instantly ✅
```

Solution 2 — React way with Context + Interceptor

```
// context/ApiContext.jsx
import React, { createContext, useContext, useEffect } from 'react';
import axios from 'axios';

const ApiContext = createContext(null);

export function ApiProvider({ children }) {
  const axiosInstance = axios.create({
    baseURL: 'https://api.yourbackend.com',
  });

  useEffect(() => {
    // Add request interceptor
    const requestInterceptor =
      axiosInstance.interceptors.request.use(
        (config) => {
          const token = localStorage.getItem('authToken');
          if (token) {
            config.headers.Authorization = `Bearer ${token}`;
          }
          return config;
        },
        (error) => Promise.reject(error)
      );
  });

  // Add response interceptor
```

```

    const responseInterceptor =
axiosInstance.interceptors.response.use(
    (response) => response,
    (error) => {
      if (error.response?.status === 401) {
        localStorage.removeItem('authToken');
        window.location.href = '/login';
      }
      return Promise.reject(error);
    }
  );

  // Cleanup interceptors on unmount
  return () => {
    axiosInstance.interceptors.request.eject(requestInterceptor);

    axiosInstance.interceptors.response.eject(responseInterceptor);
  };
}, []);

return (
  <ApiContext.Provider value={axiosInstance}>
    {children}
  </ApiContext.Provider>
);
}

// Custom hook to use anywhere
export function useApi() {
  return useContext(ApiContext);
}

// Wrap App
// index.js
function App() {

```

```

return (
  <ApiProvider>
    <Router>
      <Routes>...</Routes>
    </Router>
  </ApiProvider>
);
}

// Use in any component
function UserList() {
  const api = useApi(); // get axiosInstance with interceptor
  const [users, setUsers] = useState([]);

  useEffect(() => {
    api.get('/users') // token auto-added ✅
      .then(res => setUsers(res.data));
  }, []);

  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}

```

Solution 3 — Dynamic token (token changes after login)

```

// api/axiosInstance.js
import axios from 'axios';

const axiosInstance = axios.create({
  baseURL: 'https://api.yourbackend.com',
});

// Interceptor reads token DYNAMICALLY on each request
// So even if token changes (refresh), always gets latest
axiosInstance.interceptors.request.use((config) => {
  // Read fresh every time – not once at setup

```

```

const token = localStorage.getItem('authToken');

if (token) {
  config.headers.Authorization = `Bearer ${token}`;
}
return config;
});

// — Token Refresh Logic —————

let isRefreshing = false;
let failedQueue = [];

function processQueue(error, token = null) {
  failedQueue.forEach(({ resolve, reject }) => {
    if (error) reject(error);
    else resolve(token);
  });
  failedQueue = [];
}

axiosInstance.interceptors.response.use(
  response => response,
  async (error) => {
    const originalRequest = error.config;

    // Token expired (401) and not already retrying
    if (error.response?.status === 401 && !originalRequest._retry) {

      if (isRefreshing) {
        // Queue other requests while refreshing
        return new Promise((resolve, reject) => {
          failedQueue.push({ resolve, reject });
        }).then(token => {
          originalRequest.headers.Authorization = `Bearer ${token}`;
          return axiosInstance(originalRequest);
        });
      }
    }
  }
);

```

```

    });
  }

  originalRequest._retry = true;
  isRefreshing = true;

  try {
    // Call refresh token endpoint
    const refreshToken = localStorage.getItem('refreshToken');
    const { data } = await axios.post('/auth/refresh', {
refreshToken });

    // Save new token
    localStorage.setItem('authToken', data.accessToken);

    // Update header
    axiosInstance.defaults.headers.common['Authorization']
      = `Bearer ${data.accessToken}`;

    processQueue(null, data.accessToken);

    // Retry original request with new token
    return axiosInstance(originalRequest);

  } catch (err) {
    processQueue(err, null);
    localStorage.clear();
    window.location.href = '/login';
    return Promise.reject(err);
  } finally {
    isRefreshing = false;
  }
}

return Promise.reject(error);

```



```
    }  
  );  
  
  export default axiosInstance;
```

Q5. Count Valid Parentheses Pairs

Understanding the problem:

```
"()()()":  
  () → valid pair ✓ count = 1  
  () → valid pair ✓ count = 2  
  (( → open open  
  ) → closes one ✓ count = 3  
  leftover ( → invalid
```

Output: 3

```
"))((":  
  ) → no open to match ✗  
  ) → no open to match ✗  
  ( → open, waiting  
  ( → open, waiting  
  end → unmatched opens → invalid
```

Output: 0

JavaScript Solution

```
function countValidPairs(s) {  
  let openCount = 0; // tracks unmatched '('  
  let pairs = 0;     // counts valid matched pairs
```

```

for (let i = 0; i < s.length; i++) {
  const char = s[i];

  if (char === '(') {
    openCount++;           // found opening bracket, wait for match
  }
  else if (char === ')') {
    if (openCount > 0) {
      openCount--;         // matched! consume one open bracket
      pairs++;             // valid pair found ✅
    }
    // if openCount === 0, no open to match → skip (invalid)
  }
}

return pairs;
}

console.log(countValidPairs("()()()")); // Output: 3
console.log(countValidPairs("))(()));   // Output: 0

```

Tracing through step by step:

Input: `"()()()"`:

i=0	char='('	openCount=1	pairs=0	→ found open, waiting
i=1	char=')'	openCount=0	pairs=1	→ matched! ✅
i=2	char='('	openCount=1	pairs=1	→ found open, waiting
i=3	char=')'	openCount=0	pairs=2	→ matched! ✅
i=4	char='('	openCount=1	pairs=2	→ found open, waiting
i=5	char='('	openCount=2	pairs=2	→ found open, waiting
i=6	char=')'	openCount=1	pairs=3	→ matched! ✅

Result: 3 ✅

Input: `) ) ( (` :

`i=0 char=')'` `openCount=0` → no open to match, SKIP
`i=1 char=')'` `openCount=0` → no open to match, SKIP
`i=2 char='('` `openCount=1` → found open, waiting
`i=3 char='('` `openCount=2` → found open, waiting
End: `openCount=2` (unmatched opens, not counted)

Result: `0` 

React Solution — Interactive UI

```
import { useState } from 'react';

// Core logic – same function
function countValidPairs(s) {
  let openCount = 0;
  let pairs = 0;

  for (let char of s) {
    if (char === '(') {
      openCount++;
    } else if (char === ')') {
      if (openCount > 0) {
        openCount--;
        pairs++;
      }
    }
  }

  return pairs;
}
```

```

// Visualize which chars form valid pairs
function getCharDetails(s) {
  const result = s.split('').map(char => ({
    char,
    status: 'pending' // pending | matched | unmatched
  }));

  let openStack = []; // stores indices of unmatched '('

  for (let i = 0; i < result.length; i++) {
    if (result[i].char === '(') {
      openStack.push(i); // push index, wait for match
    } else if (result[i].char === ')') {
      if (openStack.length > 0) {
        const matchIdx = openStack.pop(); // get matching '('
        result[matchIdx].status = 'matched'; // mark '(' as matched
        result[i].status = 'matched'; // mark ')' as matched
      } else {
        result[i].status = 'unmatched'; // no open bracket → invalid
      }
    }
  }
}

// Remaining items in openStack are unmatched '('
openStack.forEach(idx => {
  result[idx].status = 'unmatched';
});

return result;
}

function ParenthesesCounter() {
  const [input, setInput] = useState("()()()()");

  const pairs = countValidPairs(input);

```

```

const details = getCharDetails(input);

// Color each character based on status
const colorMap = {
  matched: { background: '#e1f5ee', color: '#085041',
fontWeight: 'bold' },
  unmatched: { background: '#fcebeb', color: '#791f1f',
fontWeight: 'bold' },
  pending: { background: '#f5f5f5', color: '#888' }
};

return (
  <div style={{ padding: 24, fontFamily: 'monospace', maxWidth:
500 }}>
    <h2>Parentheses Pair Counter</h2>

    {/* Input */}
    <div style={{ marginBottom: 16 }}>
      <label>Input string:</label>
      <input
        value={input}
        onChange={e => setInput(e.target.value.replace(/^[^()]/g,
''))}
        placeholder="Type ( and ) only..."
        style={{
          display: 'block',
          marginTop: 8,
          padding: '8px 12px',
          fontSize: 18,
          letterSpacing: 4,
          borderRadius: 6,
          border: '1px solid #ddd',
          width: '100%'
        }}
      />

```

```

    <small style={{ color: '#999' }}>Only ( and )
allowed</small>
</div>

{/* Visual breakdown */}
<div style={{
  display: 'flex',
  gap: 4,
  flexWrap: 'wrap',
  marginBottom: 16
}}>
  {details.map((item, i) => (
    <span
      key={i}
      style={{
        ...colorMap[item.status],
        padding: '6px 10px',
        borderRadius: 4,
        fontSize: 20,
      }}
    >
      {item.char}
    </span>
  ))}
</div>

{/* Result */}
<div style={{
  padding: '12px 16px',
  background: '#e6f1fb',
  borderRadius: 8,
  fontSize: 18
}}>
  Valid pairs: <strong>{pairs}</strong>
</div>

```

```

    { /* Legend */ }
    <div style={{
      marginTop: 12,
      display: 'flex',
      gap: 12,
      fontSize: 13
    }}>
      <span style={{ ...colorMap.matched, padding: '2px 8px',
borderRadius: 4 }}>
        matched
      </span>
      <span style={{ ...colorMap.unmatched, padding: '2px 8px',
borderRadius: 4 }}>
        unmatched
      </span>
    </div>

    { /* Test cases */ }
    <div style={{ marginTop: 20 }}>
      <p style={{ color: '#666', fontSize: 13 }}>Test cases:</p>
      {[
        { s: "()()()", expected: 3 },
        { s: "))((", expected: 0 },
        { s: "()()", expected: 2 },
        { s: "((()))", expected: 3 },
        { s: "((((", expected: 0 },
      ]}.map(({ s, expected }) => (
        <div
          key={s}
          onClick={() => setInput(s)}
          style={{
            display: 'flex',
            justifyContent: 'space-between',
            padding: '6px 10px',

```

```

        marginBottom: 4,
        background: input === s ? '#e6f1fb' : '#f9f9f9',
        borderRadius: 6,
        cursor: 'pointer',
        fontSize: 14
    }}
    >
    <span>"{s}"</span>
    <span style={{ color: '#555' }}>
        expected: <strong>{expected}</strong>
        {countValidPairs(s) === expected ? '✅' : '❌'}
    </span>
</div>
    )})
</div>
</div>
);
}

export default ParenthesesCounter;

```

Edge cases covered:

```

console.log(countValidPairs("()()()")); // 3 ✅
console.log(countValidPairs(")()(")); // 0 ✅
console.log(countValidPairs("")); // 0 ✅ empty string
console.log(countValidPairs("((()"))); // 3 ✅ nested
console.log(countValidPairs("((((")); // 0 ✅ only opens
console.log(countValidPairs("))))"); // 0 ✅ only closes
console.log(countValidPairs("(()())")); // 3 ✅ mixed nesting
console.log(countValidPairs("()()()")); // 3 ✅ all valid

```

Algorithm summary:

Stack-based counting:

'(' found → openCount++ (push to mental stack)

)' found:

openCount > 0 → match! openCount--, pairs++

openCount = 0 → invalid, skip

Time: $O(n)$ – one pass through string

Space: $O(1)$ – just two counters

5/19/2026, 8:44:06 AM



wipro - UI

write js code to filter this below array with odd numbers and double it;

input - [34, 56, 76, 77, 79, 65, 15, 16, 14, 17]

how to create microfrontend (what are the steps we should take to create microfrontend)

where do u store your jwt token in react

diff between session storage and httpCookieOnly

user typing something in search bar and for each type making multiple api calls how can we avoid it and optimize it.

what are the way we optimize our React application

5/22/2026, 11:08:07 AM



Wipro UI Questions

Q1. Filter Odd Numbers and Double Them

```
const input = [34, 56, 76, 77, 79, 65, 15, 16, 14, 17];

// Method 1 – filter + map (chained)
const result = input
  .filter(num => num % 2 !== 0) // keep only odd numbers
  .map(num => num * 2);        // double each odd number

console.log(result); // [154, 158, 130, 30, 34]

// Tracing through:
// 34 → even → skip
// 56 → even → skip
```

```
// 76 → even → skip
// 77 → odd → keep → 77 * 2 = 154 ✓
// 79 → odd → keep → 79 * 2 = 158 ✓
// 65 → odd → keep → 65 * 2 = 130 ✓
// 15 → odd → keep → 15 * 2 = 30 ✓
// 16 → even → skip
// 14 → even → skip
// 17 → odd → keep → 17 * 2 = 34 ✓
```

Other methods:

```
// Method 2 – reduce (single pass, slightly more efficient)
const result2 = input.reduce((acc, num) => {
  if (num % 2 !== 0) { // if odd
    acc.push(num * 2); // double and add
  }
  return acc;
}, []);

console.log(result2); // [154, 158, 130, 30, 34]

// Method 3 – forEach
const result3 = [];
input.forEach(num => {
  if (num % 2 !== 0) result3.push(num * 2);
});

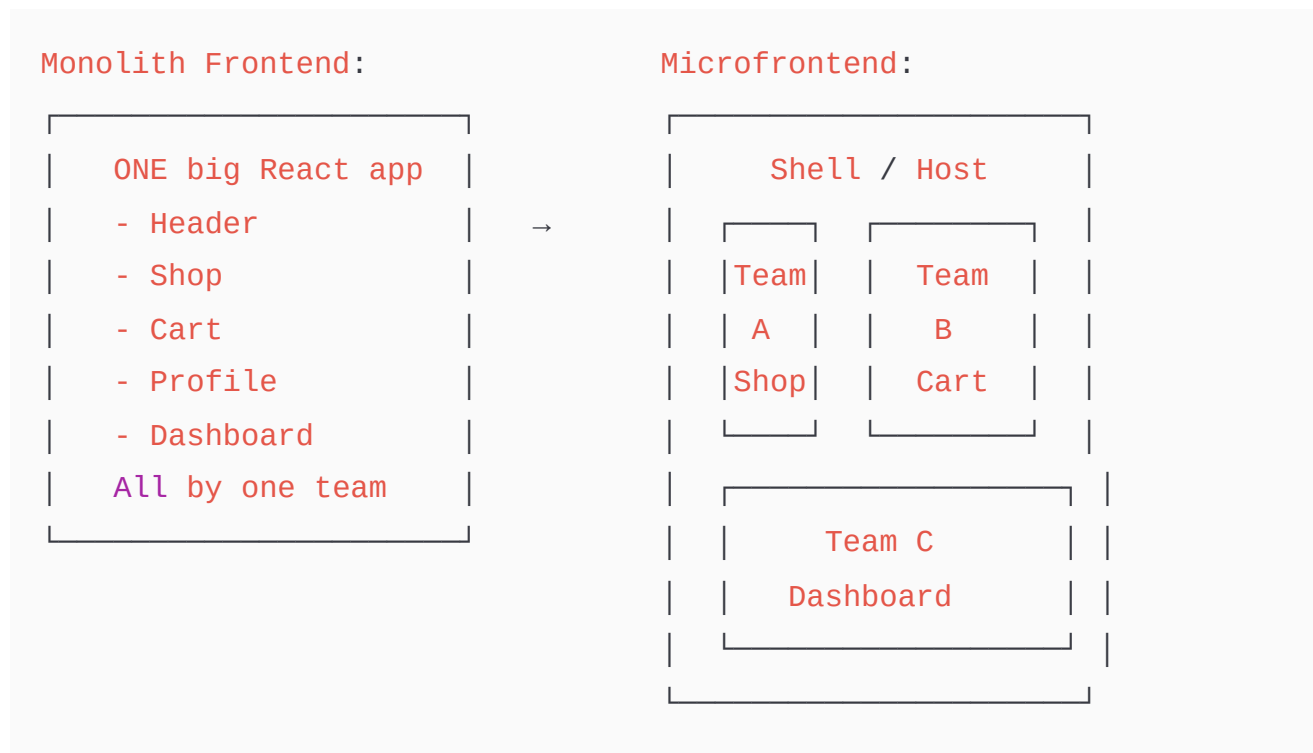
// Method 4 – flatMap (one step)
const result4 = input.flatMap(num =>
  num % 2 !== 0 ? [num * 2] : [] // return [] to skip, [value] to
  keep
);

console.log(result4); // [154, 158, 130, 30, 34]
```

```
// All 4 methods produce same output:  
// [154, 158, 130, 30, 34]
```

Q2. How to Create a Microfrontend — Steps

What is Microfrontend:



Step-by-Step Guide:

Step 1 — Plan & Divide the App

Identify independent features/domains:

Shell App	→ routing, layout, navigation	(Team Platform)
Auth MFE	→ login, register, logout	(Team Auth)
Shop MFE	→ products, search	(Team Shop)
Cart MFE	→ cart, checkout	(Team Commerce)
Profile MFE	→ user settings	(Team User)

Rules for splitting:

✅ Split by business domain

- ✓ Each MFE owned by ONE team
- ✓ Independent deployable
- ✓ Loose coupling between MFES

Step 2 — Choose Integration Approach

Option A: Module Federation (Webpack 5) – most popular

Each app exposes components

Host app imports them at runtime

No rebuild needed when child updates

Option B: iFrames – simplest but limited

Each MFE in separate iframe

Communication via postMessage

Option C: npm packages – for shared components

Build component library

Each MFE installs it

Need republish to update

Option D: Web Components – framework agnostic

Custom HTML elements

Works with any framework

Step 3 — Setup Module Federation (Webpack 5)

```
// Shop MFE – webpack.config.js (REMOTE – exposes components)
const ModuleFederationPlugin =
  require('webpack/lib/container/ModuleFederationPlugin');

module.exports = {
  plugins: [
    new ModuleFederationPlugin({
      name: 'shopMFE', // unique name
```

```

    filename: 'remoteEntry.js',    // entry point file
    exposes: {
      // what this MFE shares with others
      './ProductList': './src/components/ProductList',
      './SearchBar': './src/components/SearchBar',
    },
    shared: {
      // share React so each MFE doesn't bundle its own copy
      react: { singleton: true, requiredVersion: '^18.0.0' },
      'react-dom': { singleton: true, requiredVersion: '^18.0.0' }
    },
  },
  }),
],
};

// Shell App – webpack.config.js (HOST – consumes remotes)
module.exports = {
  plugins: [
    new ModuleFederationPlugin({
      name: 'shell',
      remotes: {
        // tell shell where each MFE lives
        shopMFE: 'shopMFE@http://localhost:3001/remoteEntry.js',
        cartMFE: 'cartMFE@http://localhost:3002/remoteEntry.js',
        profileMFE:
'profileMFE@http://localhost:3003/remoteEntry.js',
      },
      shared: {
        react: { singleton: true, requiredVersion: '^18.0.0' },
        'react-dom': { singleton: true, requiredVersion: '^18.0.0' }
      },
    }),
  ],
});

```

```
],  
};
```

Step 4 — Shell App Routes to MFEs

```
// Shell App — App.jsx  
import React, { lazy, Suspense } from 'react';  
import { BrowserRouter, Routes, Route } from 'react-router-dom';  
  
// Lazy load each MFE — downloaded only when route visited  
const ProductList = lazy(() => import('shopMFE/ProductList'));  
const Cart = lazy(() => import('cartMFE/Cart'));  
const Profile = lazy(() => import('profileMFE/Profile'));  
  
function Shell() {  
  return (  
    <BrowserRouter>  
      <nav>  
        <a href="/">Home</a>  
        <a href="/shop">Shop</a>  
        <a href="/cart">Cart</a>  
        <a href="/profile">Profile</a>  
      </nav>  
  
      <Suspense fallback={<div>Loading...</div>}>  
        <Routes>  
          <Route path="/shop" element={<ProductList />} />  
          <Route path="/cart" element={<Cart />} />  
          <Route path="/profile" element={<Profile />} />  
        </Routes>  
      </Suspense>  
    </BrowserRouter>  
  );  
}
```

Step 5 — Communication Between MFEs

```
// Method 1: Custom Events (loose coupling)
// Shop MFE – fire event when item added to cart
window.dispatchEvent(new CustomEvent('cart:add', {
  detail: { id: 1, name: 'Laptop', price: 1000 }
}));

// Cart MFE – listen for event
window.addEventListener('cart:add', (event) => {
  const item = event.detail;
  addToCart(item);
});

// Method 2: Shared State via URL
// Navigate with state in URL params
window.location.href = '/cart?item=123';

// Method 3: Shared localStorage
// Shop MFE
localStorage.setItem('selectedProduct', JSON.stringify(product));

// Cart MFE
const product = JSON.parse(localStorage.getItem('selectedProduct'));

// Method 4: Shared Redux Store (if same framework)
// Create shared store package
// Each MFE imports from '@company/shared-store'
```

Step 6 — Shared Design System / Component Library

Create shared npm package:

@company/ui-components	→ Button, Input, Modal, etc.
@company/utils	→ formatDate, formatCurrency, etc.
@company/api-client	→ axios instance with interceptors

Each MFE installs these as dependencies
Ensures consistent UI across all MFES ✓

Step 7 — Independent CI/CD Per MFE

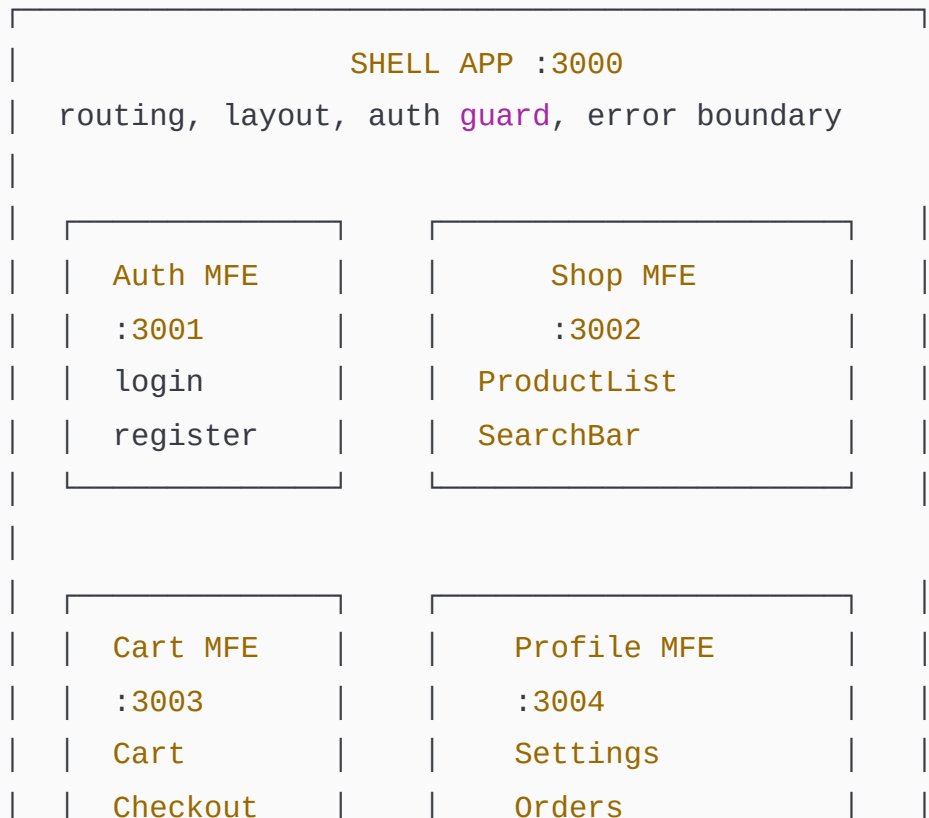
Each MFE has its own:

- ✓ Repository (or monorepo with separate pipelines)
- ✓ CI/CD pipeline
- ✓ Docker container
- ✓ Kubernetes deployment
- ✓ Own port (3001, 3002, 3003...)

Deploy Shop MFE → only Shop rebuilds and deploys

No need to rebuild Shell or other MFES ✓

Full Architecture:





Q3. Where to Store JWT Token in React

3 options — pros and cons:

	localStorage	httpOnly Cookie
XSS Attack	Vulnerable ❌	Safe ✅
CSRF Attack	Safe ✅	Vulnerable ❌
JS Access	Easy ✅	Cannot access ❌
Auto-send	Manual	Auto by browser
Expiry control	Manual	Browser handles
Best for	Simple apps	Production apps

Option 1 — localStorage (simple, less secure):

```

// After login
function handleLogin(token) {
  localStorage.setItem('authToken', token);
}

// On API calls
const token = localStorage.getItem('authToken');
axios.defaults.headers.common['Authorization'] = `Bearer ${token}`;

```

```
// On logout
localStorage.removeItem('authToken');

// Problem: XSS attack can steal token
// Any injected script can: localStorage.getItem('authToken') ❌
```

Option 2 — httpOnly Cookie (most secure):

```
// Backend sets the cookie (you can't do this from JS)
// Backend response header:
// Set-Cookie: token=xxx; HttpOnly; Secure; SameSite=Strict

// Frontend – you CANNOT read httpOnly cookie from JS
// That's the point! XSS can't steal it ✅
// Browser sends it automatically with every request ✅

// Axios must send credentials with requests
axios.defaults.withCredentials = true;

// CSRF protection needed:
// Add CSRF token in request header
axios.defaults.headers.common['X-CSRF-Token'] = getCsrfToken();
```

Option 3 — Memory (most secure for XSS, lost on refresh):

```
// Store in React state / module variable
let inMemoryToken = null;

function setToken(token) {
  inMemoryToken = token; // in memory only
}

function getToken() {
  return inMemoryToken;
}
```

```
}
```

```
// Combined approach: access token in memory + refresh in httpOnly cookie  
// Access token → short lived (15min) → store in memory  
// Refresh token → long lived (7days) → httpOnly cookie  
// When access expires → use refresh cookie to get new access token
```

Recommended approach:

Best practice:

Access Token → Memory (React state/context) – short lived

Refresh Token → httpOnly Cookie – long lived

Why:

Memory → XSS can't steal (not in DOM/localStorage)

httpOnly → JS can't read → XSS safe

Refresh in cookie → auto-renews access token silently

Q4. sessionStorage vs httpOnly Cookie

	sessionStorage	httpOnly Cookie
Set by	JavaScript	Server (Set-Cookie)
Read by JS	✓ Yes	✗ No (httpOnly)
Sent with req	✗ Manual only	✓ Auto by browser
Lifespan	Tab close = cleared	Expiry date / session
XSS safe	✗ No	✓ Yes
CSRF safe	✓ Yes	✗ Needs CSRF token
Cross-tab	✗ No	✓ Yes

Size limit	~5MB	~4KB	
Storage	Browser	Browser (server set)	

```
// — sessionStorage —————
// Set/Get/Remove from JavaScript
sessionStorage.setItem('token', 'jwt-here');
const token = sessionStorage.getItem('token');
sessionStorage.removeItem('token');

// Cleared automatically when:
// ✅ Browser tab is closed
// ✅ Browser is closed
// ❌ NOT cleared on page refresh (stays on refresh)

// — httpOnly Cookie —————
// Set by SERVER in response header:
// Set-Cookie: token=jwt-here; HttpOnly; Secure; SameSite=Strict;
// Max-Age=86400

// From JS — you CANNOT:
document.cookie; // won't show httpOnly cookies ❌

// Browser sends automatically with every same-origin request ✅
// Cannot be stolen by XSS ✅

// Flags explained:
// HttpOnly → JS cannot read
// Secure → HTTPS only
// SameSite=Strict → not sent on cross-site requests (CSRF
// protection)
// Max-Age → expiry in seconds
```

Q5. Avoid Multiple API Calls on Search (Debounce)

Problem:

```
User types "laptop":  
l      → API call ❌  
la     → API call ❌  
lap    → API call ❌  
lapt   → API call ❌  
lapto  → API call ❌  
laptop → API call ❌  
= 6 API calls for one search! ❌
```

Solution — Debouncing:

```
With 500ms debounce:  
l      → reset timer  
la     → reset timer  
lap    → reset timer  
lapt   → reset timer  
lapto  → reset timer  
laptop → reset timer  
(500ms passes with no typing)  
      → ONE API call ✅  
= 1 API call instead of 6! ✅
```

Plain JavaScript:

```
function debounce(fn, delay) {  
  let timer = null;  
  
  return function(...args) {  
    clearTimeout(timer); // cancel previous timer on every call  
    timer = setTimeout(() => {
```

```

    fn(...args);          // only fires after delay ms of silence
  }, delay);
};
}

// Usage
const debouncedSearch = debounce((query) => {
  fetch(`/api/search?q=${query}`)
    .then(res => res.json())
    .then(data => console.log(data));
}, 500);

// Attach to input
document.getElementById('search').addEventListener('input', (e) => {
  debouncedSearch(e.target.value);
});

```

React — useEffect way:

```

function SearchBar() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const [loading, setLoading] = useState(false);

  useEffect(() => {
    if (!query.trim()) {
      setResults([]);
      return;
    }

    const timer = setTimeout(async () => {
      try {
        setLoading(true);
        const res = await fetch(`/api/search?q=${query}`);
        const data = await res.json();

```

```

        setResults(data);
      } catch (err) {
        console.error(err);
      } finally {
        setLoading(false);
      }
    }, 500); // wait 500ms after last keystroke

    // Cleanup: cancel timer if user types again before 500ms
    return () => clearTimeout(timer);

  }, [query]); // re-runs on every query change

  return (
    <div>
      <input
        value={query}
        onChange={e => setQuery(e.target.value)}
        placeholder="Search..."
      />
      {loading && <p>Searching...</p>}
      <ul>
        {results.map(item => (
          <li key={item.id}>{item.name}</li>
        ))}
      </ul>
    </div>
  );
}

```

React — Custom useDebounce hook (cleanest):

```

// hooks/useDebounce.js
function useDebounce(value, delay = 500) {
  const [debouncedValue, setDebouncedValue] = useState(value);

```



```

useEffect(() => {
  const timer = setTimeout(() => {
    setDebounceValue(value); // update only after delay
  }, delay);

  return () => clearTimeout(timer); // cancel on next change
}, [value, delay]);

return debounceValue;
}

// SearchBar.jsx
function SearchBar() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);

  const debounceQuery = useDebounce(query, 500);

  // Only fires when debounceQuery changes (500ms after typing stops)
  useEffect(() => {
    if (!debounceQuery.trim()) return;

    fetch(`/api/search?q=${debounceQuery}`)
      .then(res => res.json())
      .then(data => setResults(data));

  }, [debounceQuery]); // NOT query – debounceQuery ✅

  return (
    <div>
      <input
        value={query}
        onChange={e => setQuery(e.target.value)}

```

```

        placeholder="Search..."
      />
      <ul>
        {results.map(item => <li key={item.id}>{item.name}</li>)}
      </ul>
    </div>
  );
}

```

Cancel previous API call too (AbortController):

```

function SearchBar() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);

  const debouncedQuery = useDebounce(query, 500);

  useEffect(() => {
    if (!debouncedQuery.trim()) return;

    // Cancel previous request if new one starts
    const controller = new AbortController();

    async function search() {
      try {
        const res = await fetch(
          `/api/search?q=${debouncedQuery}`,
          { signal: controller.signal } // attach cancel signal
        );
        const data = await res.json();
        setResults(data);
      } catch (err) {
        if (err.name !== 'AbortError') {
          console.error(err); // ignore cancel errors
        }
      }
    }
  });
}

```

```

    }
  }

  search();

  // Cancel this request when next search starts
  return () => controller.abort();

}, [debouncedQuery]);
}

```

Q6. Ways to Optimize React Application

Level 1 — Prevent Unnecessary Re-renders

```

// React.memo – skip re-render if props same
const ProductCard = React.memo(({ product }) => {
  return <div>{product.name}</div>;
});

// useMemo – cache expensive calculation
const sortedProducts = useMemo(() => {
  return products.sort((a, b) => a.price - b.price);
}, [products]); // only re-sorts when products changes

// useCallback – stable function reference
const handleAddToCart = useCallback((id) => {
  dispatch(addItem(id));
}, [dispatch]); // same function reference across renders

```

Level 2 — Load Less Code

```

// Code splitting + Lazy loading
const Dashboard = lazy(() => import('./pages/Dashboard'));

```

```

const Reports    = lazy(() => import('./pages/Reports'));

// Route-based splitting
<Suspense fallback={<Spinner />}>
  <Routes>
    <Route path="/dashboard" element={<Dashboard />} />
    <Route path="/reports"    element={<Reports />} />
  </Routes>
</Suspense>
// Each page only loads when user navigates there ✅

```

Level 3 — Handle Large Lists

```

// Virtualization – only render visible rows
import { FixedSizeList } from 'react-window';

function BigList({ items }) {
  return (
    <FixedSizeList
      height={600}
      itemCount={items.length} // 100,000 items
      itemSize={50}
    >
      {({ index, style }) => (
        <div style={style}>{items[index].name}</div>
      )}
    </FixedSizeList>
  );
  // Only ~12 DOM nodes at any time, not 100,000 ✅
}

```

Level 4 — Smart State Management

```

// Move state down – don't put everything at top
// ❌ Bad – parent re-renders affect ALL children
function App() {
  const [inputVal, setInputVal] = useState(''); // changes on every
  keypress
  return (
    <>
      <input onChange={e => setInputVal(e.target.value)} />
      <HeavyComponent /> {/* re-renders on every keypress! */}
    </>
  );
}

// ✅ Good – isolate state to where it's needed
function SearchInput() {
  const [inputVal, setInputVal] = useState('');
  return <input onChange={e => setInputVal(e.target.value)} />;
  // HeavyComponent never re-renders ✅
}

```

Level 5 — Network & Loading

```

// 1. React Query – caching API responses
const { data } = useQuery({
  queryKey: ['users'],
  queryFn: () => fetch('/api/users').then(r => r.json()),
  staleTime: 5 * 60 * 1000, // don't re-fetch for 5 minutes ✅
});

// 2. Debounce search
const debouncedSearch = useDebounce(query, 500);

// 3. Lazy load images


```

```
// 4. useTransition – non-urgent updates
const [isPending, startTransition] = useTransition();
startTransition(() => {
  setFilteredItems(heavyFilter(items)); // won't block typing ✅
});
```

Quick Checklist:

Re-renders:

- ✅ React.memo for pure components
- ✅ useMemo for expensive calculations
- ✅ useCallback for functions passed as props
- ✅ Move state down / co-locate state
- ✅ Avoid new objects/arrays in JSX props

Bundle size:

- ✅ React.lazy + Suspense for routes
- ✅ Tree shaking (import only what you need)
- ✅ Analyze bundle: webpack-bundle-analyzer

Large data:

- ✅ Virtualize long lists (react-window)
- ✅ Pagination or infinite scroll
- ✅ Debounce search input

Network:

- ✅ Cache API responses (React Query)
- ✅ AbortController – cancel stale requests
- ✅ Lazy load images
- ✅ CDN for static assets

Measure first:

- ✅ React DevTools Profiler → find slow components
- ✅ Lighthouse score



Chrome Performance tab

→ Optimize **ONLY** what's actually slow

→ Premature optimization = wasted effort **✗**

5/22/2026, 11:10:59 AM